Generative AI and Its Applications

Unit 3

(UE23CS342BA4)

This unit teaches how to **improve RAG systems** so that LLMs give **more accurate answers** using external documents.

Technical Explanation

Advanced RAG refers to improved **retrieval pipelines** that enhance:

- query understanding, document retrieval
- Ranking, context preparation before LLM generation.

ADVANCED RETRIEVAL-AUGMENTED GENERATION

Dr. Pooja Agarwal

Dept. of CSE, PES University, Bangalore

Acknowledgement

The slides are prepared from various resources from the Universities from abroad and India. Also, some material is taken from reliable resources from internet throughout this course.

Quick Recap

What is RAG?

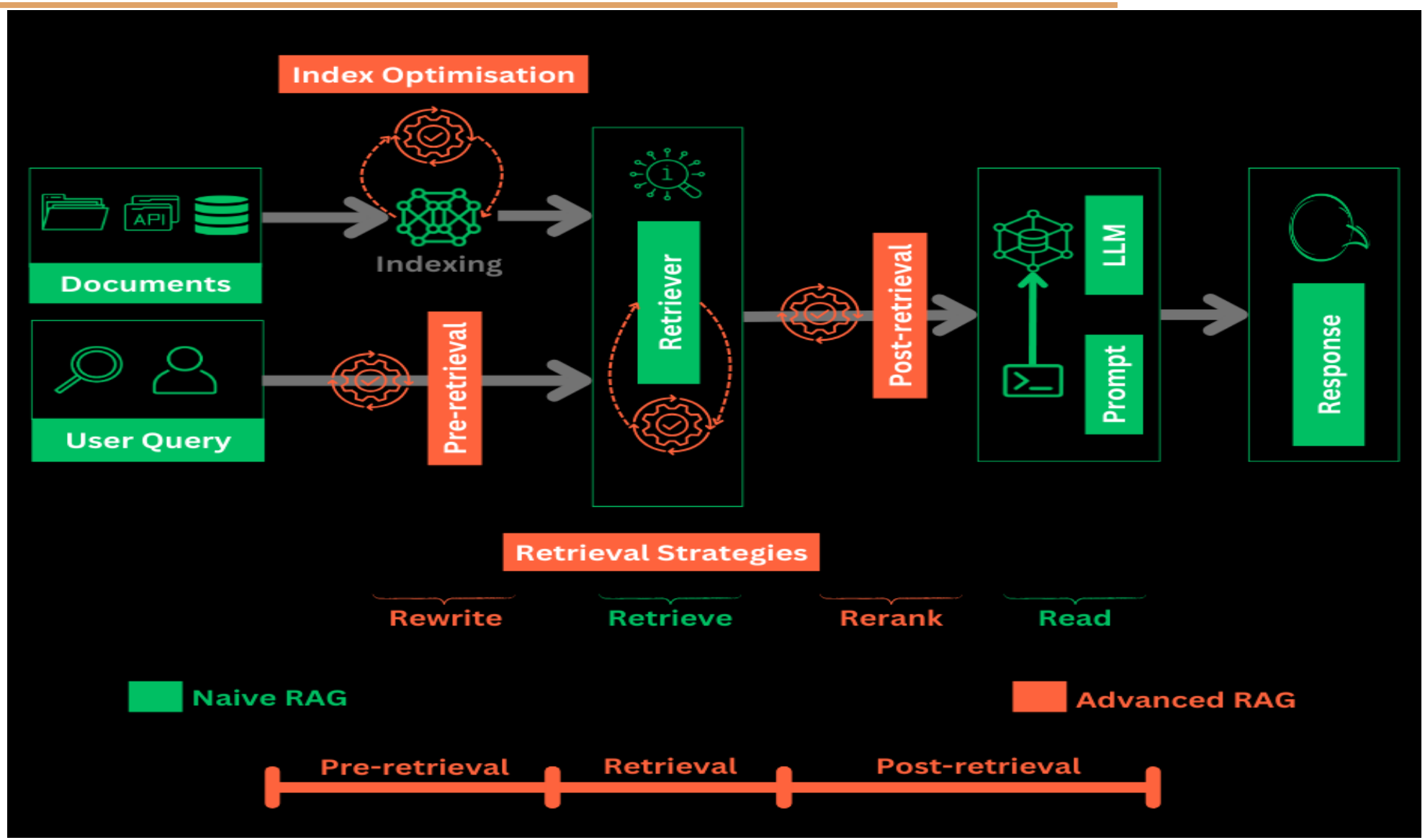
Basic RAG: Ingestion, Retrieval, Synthesis pipeline
Retrieval Algorithms: FLAT (brute-force), IVF (clustering), HNSW (graph-based), PQ (compression)
Ranking: BM25 for sparse retrieval, Cosine Similarity for dense retrieval

The Problem with Naive RAG

Issue	Description	Analogy
Irrelevant Retrieval	Top-k chunks often miss the mark	Searching for "apple" and getting fruit recipes when you wanted Apple Inc. stock prices
Context Overload	Too much irrelevant context confuses the LLM	Giving a student 10 textbooks to answer one simple question
Lost in the Middle	LLMs focus on start/end of context, ignore middle	Reading only first and last paragraphs of a long article
Hallucinations	LLM ignores context, invents answers	Student writing answers from memory instead of using provided notes
Latency Issues	Retrieval from millions of vectors is slow	Finding a needle in a haystack by looking at every straw

GenerativeAI and Its Applications

Quick Recap



The Evolution - From Naive to Advanced RAG

EVOLUTION

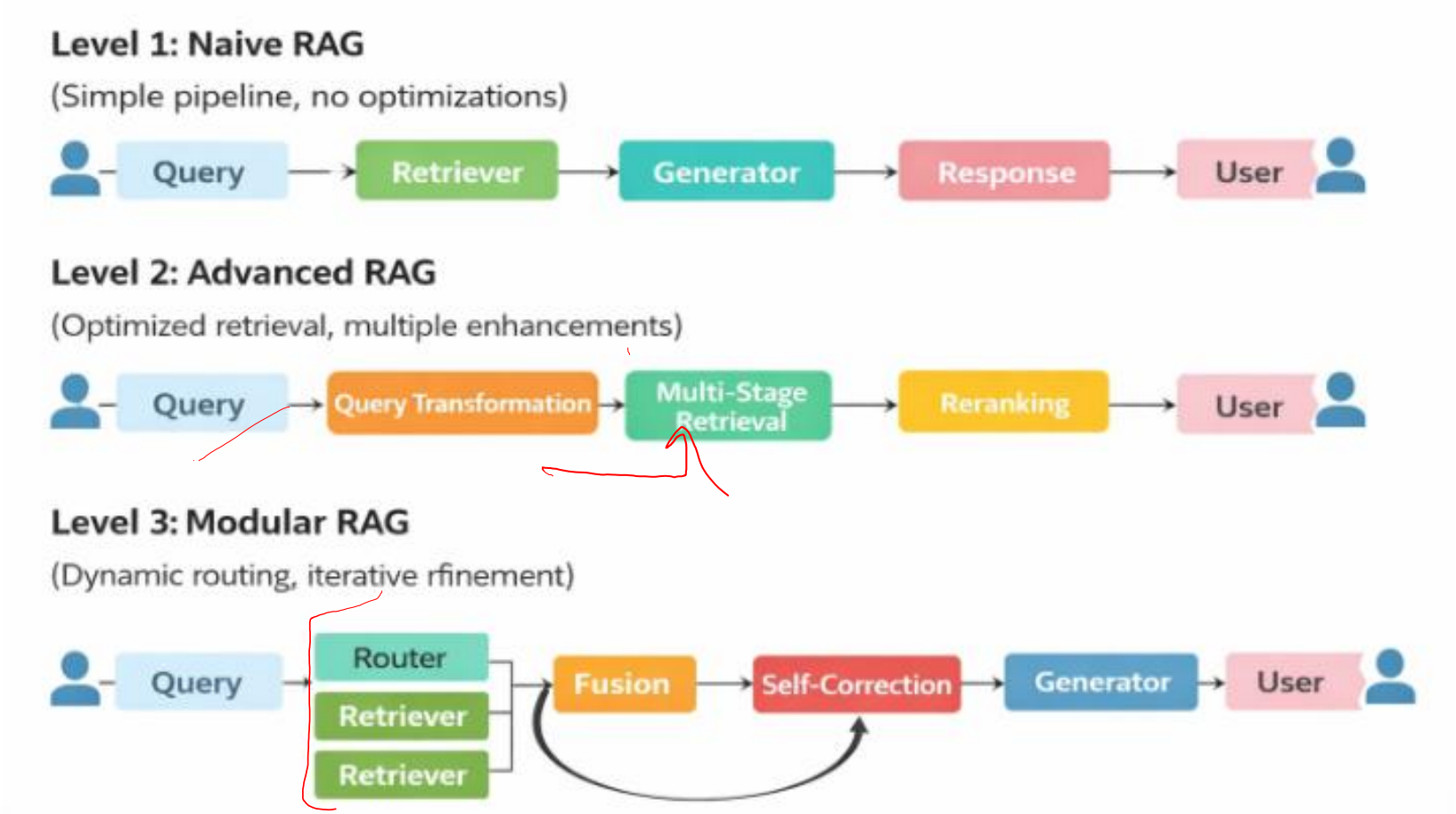
Level 1: Naive RAG
| Query → Retriever → Generator → Response
| (Simple pipeline, no optimizations)
|
Level 2: Advanced RAG
| Query → Query Transformation → Multi-Stage Retrieval → Reranking → Generator → Response (First dense search, then keyword search, then reranking)
| (Optimized retrieval, multiple enhancements)
|
Level 3: Modular RAG
| Query → Router → Multiple Retrievers → Fusion → Self-Correction → Generator
((Dense + Keyword + Graph))
| (Dynamic routing, iterative refinement)

Retrieval process:
1.Convert query → embedding
2.Search vector database
3.Find top relevant chunks

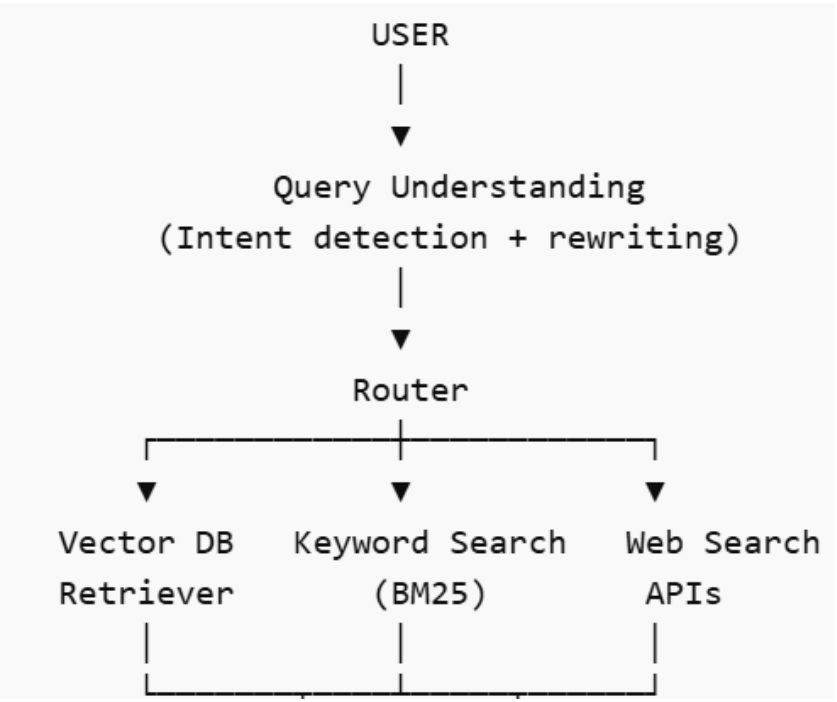
Examples of retrievers:
- **Vector Retriever** (FAISS, Pinecone search)
- **BM25 Retriever**
- **Hybrid Retriever**
- **Dense Retriever**

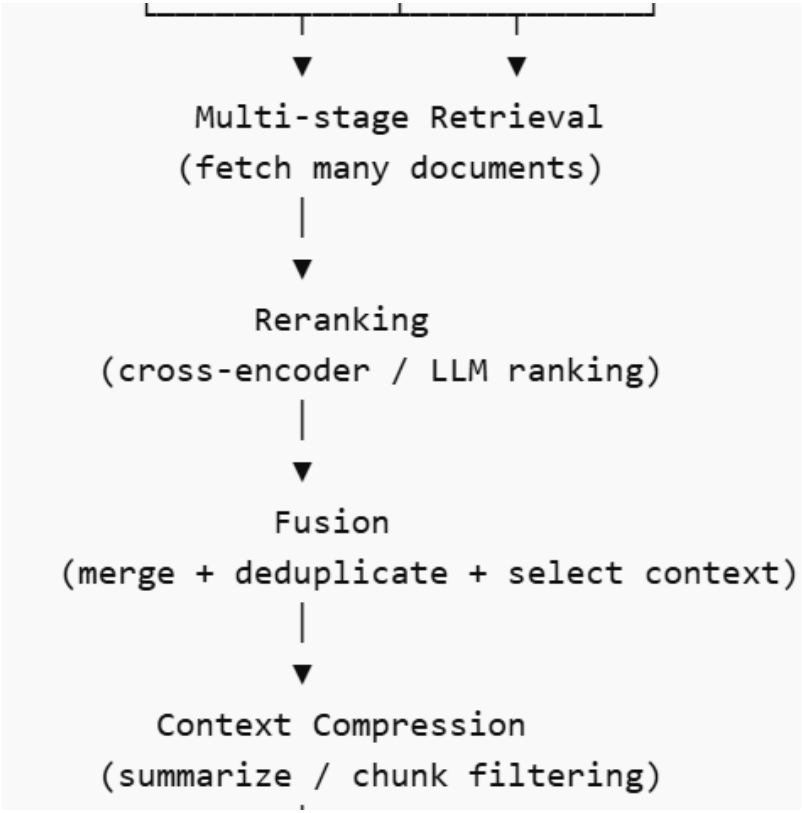
The Evolution - From Naive to Advanced RAG

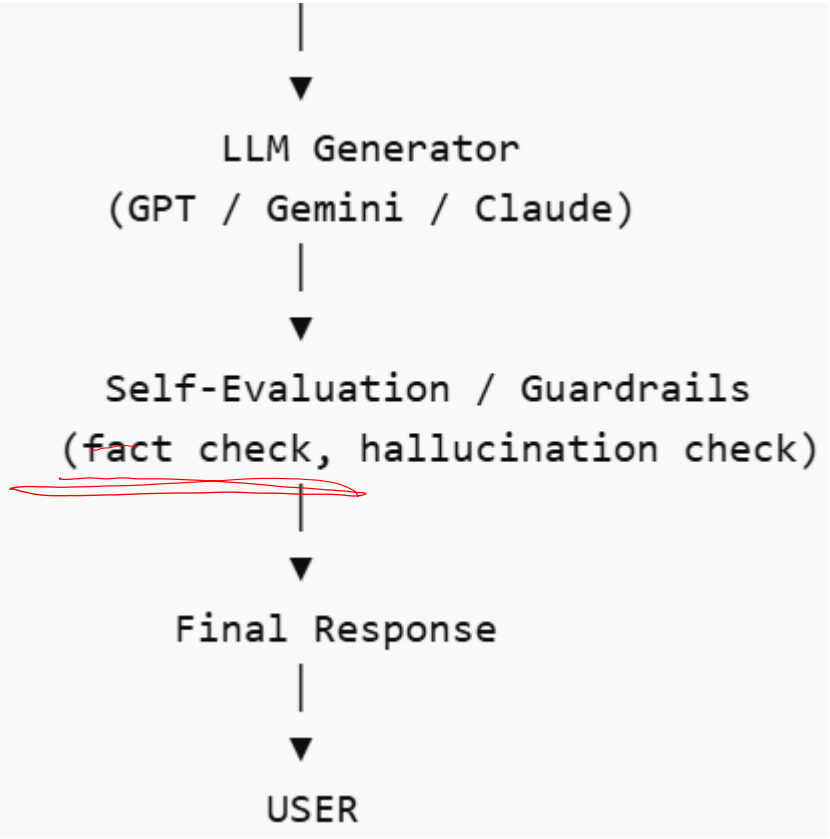
EVOLUTION



Modular RAG (Retrieval-Augmented Generation)







Advanced RAG (Retrieval-Augmented Generation)

Advanced RAG is an improved version of the traditional Retrieval-Augmented Generation system that enhances how information is **retrieved, filtered, and used** before generating responses with an LLM. It introduces **multiple optimization techniques** to improve **accuracy, relevance, and reasoning**.

Advanced RAG introduces:

- query transformation
- hybrid retrieval
- reranking
- context filtering
- multi-stage retrieval

Goal:

increase answer accuracy and reasoning ability

Instead of retrieving **10 random chunks**, system retrieves **top 3 highly relevant chunks**.

The Need for Advanced RAG - 5 Key Drivers

NEED FOR ADVANCED RAG?

1. Precision vs. Recall Trade-off
Naive RAG: High recall (finds many relevant docs), low precision (too much noise)
Advanced RAG: Balanced precision-recall through multi-stage filtering
2. Query Complexity
Real user queries are ambiguous, multi-intent, or poorly phrased
Need query understanding before retrieval
3. Scale Requirements
Production systems: Millions/Billions of documents
Need efficient indexing and retrieval (IVF, HNSW you learned)
4. Cost Optimization
LLM calls are expensive ($/token)
Better context = fewer tokens = lower cost
5. User Expectations
Users expect ChatGPT-like experience with factual accuracy
Need citations, verifiable answers

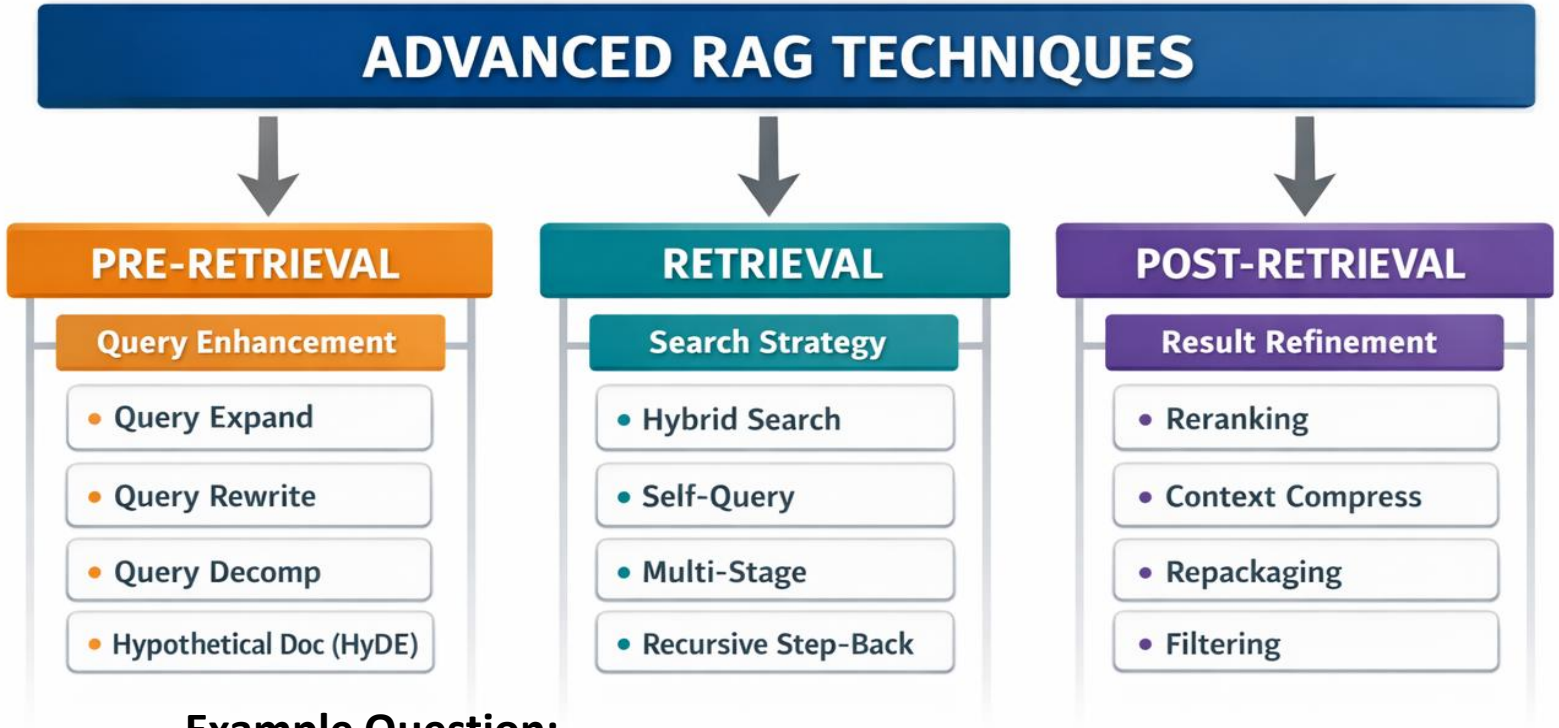
Problem	Explanation
Irrelevant documents	Retriever returns noisy chunks
Poor query understanding	User queries may be vague
Limited reasoning	LLM receives incomplete context
Scalability issues	Large document collections

GenerativeAI and Its Applications

Advanced RAG techniques flowchart diagram

PIPELINE:

User Query
↓
Query Transformation
↓
Multi-Stage Retrieval
↓
Reranking / Filtering
↓
Context Selection
↓
LLM Generation
↓
Final Response



Example Question:
"Applications of AI in cancer detection"
Steps:
1 Query expanded
2 Retrieve research papers
3 Rerank most relevant ones
4 LLM summarizes

GenerativeAI and Its Applications

Key Techniques in Advanced RAG

Query Transformation

The query is modified or expanded before retrieval.

Examples:

- Query rewriting
- Query Expansion
- Query decomposition
- Step-back prompting

Example:

User Query

"Impact of AI on healthcare"

Rewritten queries:

- AI applications in healthcare
- Benefits of AI in medical diagnosis
- AI-based medical systems

Key Techniques in Advanced RAG

Hybrid Retrieval Combines **two types of search**.

Method	Purpose
Sparse Retrieval (BM25)	Keyword matching
Dense Retrieval (Embeddings)	Semantic similarity

This improves **recall and precision**.

Example
Query:
"car engine problems"
Dense search retrieves:
•automobile engine failure
Sparse search retrieves:
•car engine repair manual.

Key Techniques in Advanced RAG

Hybrid Retrieval Combines **two types of search**.

Method	Purpose
Sparse Retrieval (BM25)	Keyword matching
Dense Retrieval (Embeddings)	Semantic similarity

This improves **recall and precision**.

Multi-Stage Retrieval

Retrieval happens in **multiple filtering stages**.

Stage 1 → Fast retrieval (BM25 / ANN)

Stage 2 → Reranking **using** cross-encoder Model (Top 10 docs)

Stage 3 → Context Selection - Select best chunks, Remove Duplicates (Top 5 docs)

This reduces **noise in retrieved documents**.

GenerativeAI and Its Applications

Key Techniques in Advanced RAG

Reranking

Retrieved documents are **rescored using better models**.

Reranking models include:

- Cross-encoder models
- LLM rerankers
- Cohere reranker

Purpose: Select the **most relevant top-k documents**.

.

Context Enrichment

System retrieves **small chunks** but adds **surrounding context**.

Methods:

- Sentence window retrieval
- Parent-child chunk retrieval
- Auto-merging retriever

This improves **LLM reasoning ability**.

Retrieved sentence:
"AI detects tumors using CNN."
System also adds **previous and next sentences**.

Recursive Retrieval

System retrieves information **multiple times using new queries**.

Query → Retrieve documents

↓

Extract entities

↓

Generate new queries

↓

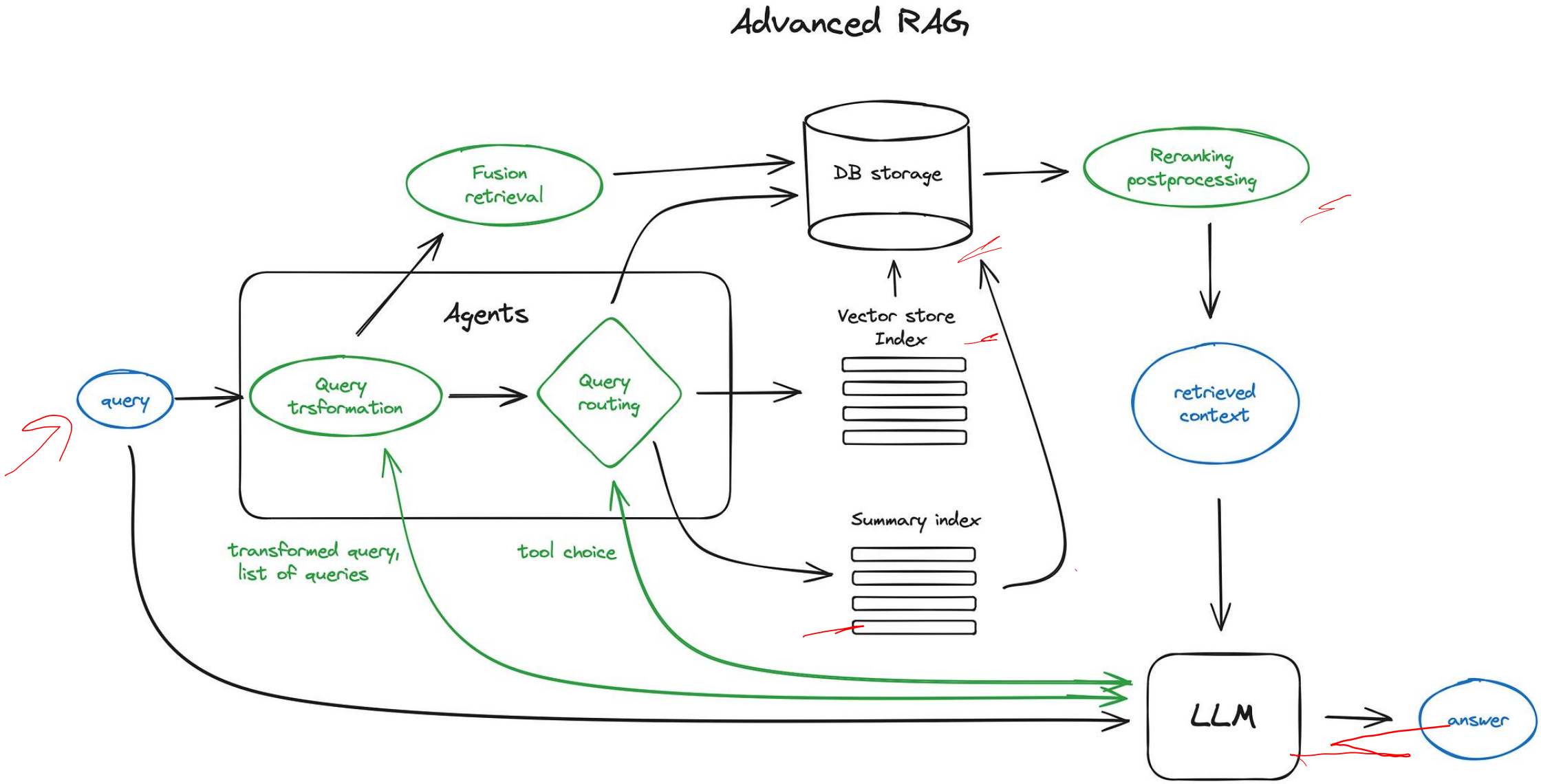
Retrieve more documents

Used in **complex knowledge systems**.

Query:
COVID vaccine side effects
Round 1:
Retrieve vaccine papers
Round 2:
Search Pfizer trials
Round 3:
Search FDA reports

GenerativeAI and Its Applications

The Advanced RAG Pipeline - High Level



GenerativeAI and Its Applications

Chunking: Large documents are **split into smaller pieces**.

- Transformer models have fixed input sequence length and even if the input context window is large, the vector of a sentence or a few better represents their semantic meaning than a vector averaged over a few pages of text (depends on the model too, but true in general).
- **Chunk your data** is spliting the initial documents in chunks of some size without loosing their meaning (splitting your text in sentences or in paragraphs, not cutting a single sentence in two parts). There are various text splitter implementations capable of this task.

Consider: Encoder models like BERT-based Sentence Transformers take 512 tokens at most, OpenAI ada-002 is capable of handling longer sequences like 8191 tokens, but the compromise here is enough context for the LLM to reason upon vs specific enough text embedding in order to efficiently execute search upon. Chunks are typically: 100 – 500 words

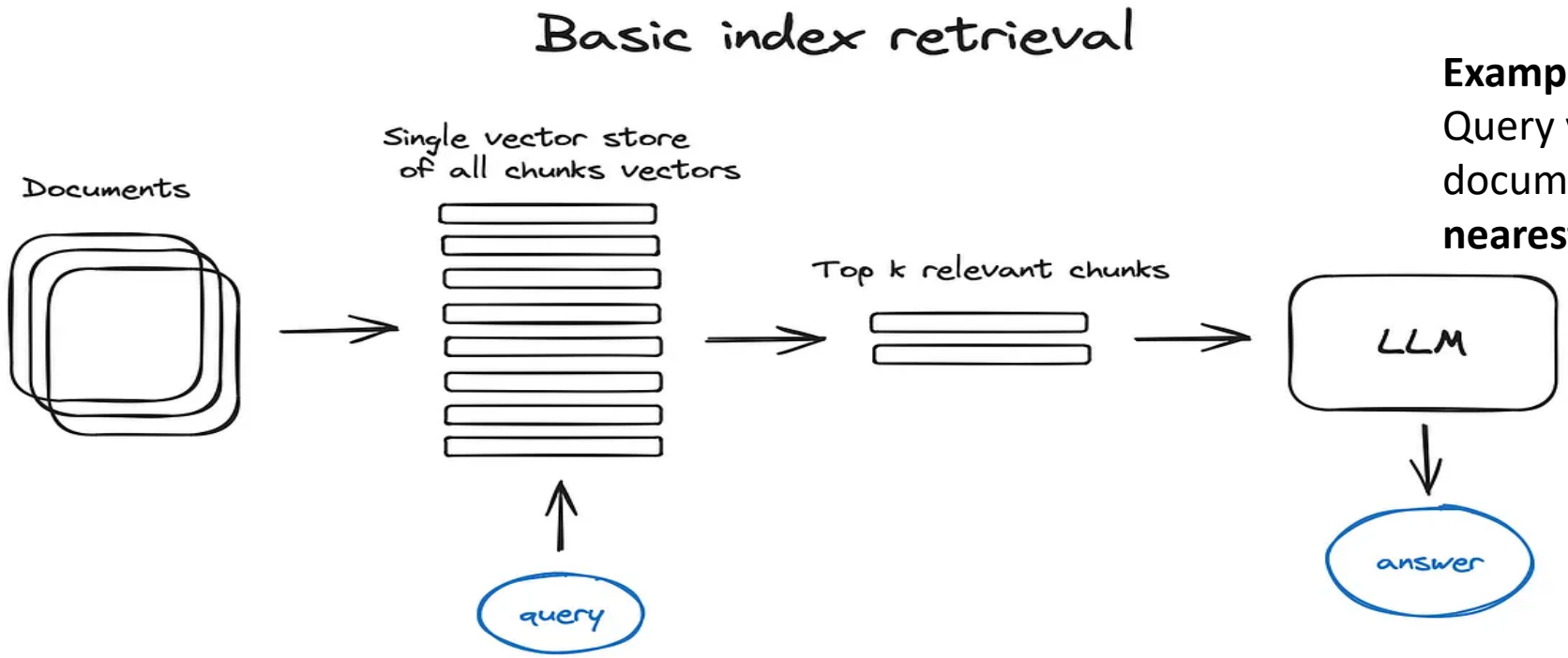
Example: Research paper split into: Introduction chunk, Method chunk, Results chunk

- The next step is to choose a **model to embed our chunks. The search optimised models** like bge-large or E5 embeddings family can be chosen.

GenerativeAI and Its Applications

Search index- Vector store index

- The crucial part of the RAG pipeline is the search index, storing your vectorized content we got in the previous step. The naivest implementation uses a flat index which is a brute force distance calculation between the query vector and all the chunks' vectors.
- A proper search index, optimized for efficient retrieval on 10000+ elements scales is a **vector index like faiss, nmslib or annoy**, using some Approximate Nearest Neighbours implementation like clustring, trees or HNSW algorithm.
- Depending on your index choice, data and search needs you can also store metadata along with vectors and then use metadata filters to search for information within some dates or sources for example.



Example

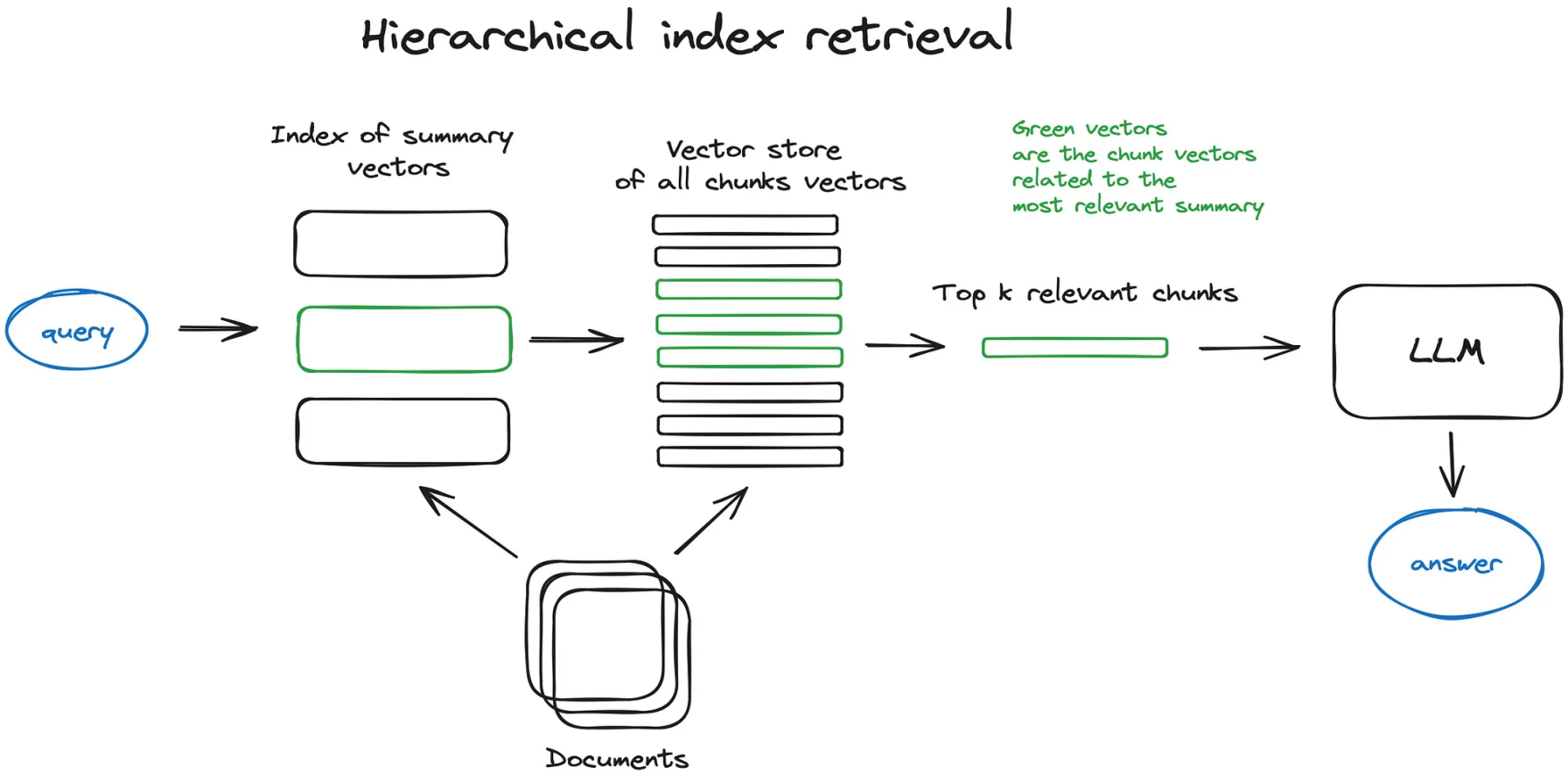
Query vector compared with document vectors to find **nearest matches**.

GenerativeAI and Its Applications

Search index: Hierarchical indices

- In case you have many documents to retrieve from, you need to be able to efficiently search inside them, find relevant information and synthesize it in a single answer with references to the sources.
- An efficient way to do that in case of a large database is to **create two indices: one composed of summaries (Summary index) and the other one composed of document chunks (Chunk index)**, and to search in two steps, first filtering out the relevant docs by summaries and then searching just inside this relevant group.

Query:
Search research papers.
Process
First find **relevant paper**, then retrieve **paragraphs inside it**.



Search index- Hypothetical Questions and HyDE (Hypothetical Document Embedding)

LLM generates **a hypothetical answer**, then search using that.

- Another approach is to ask an LLM to **generate a question for each chunk and embed these questions in vectors**, at runtime performing query search against this index of question vectors (replacing chunks vectors with questions vectors in our index) and then after retrieval route to original text chunks and send them as the context for the LLM to get an answer.
- This approach improves search quality due to a **higher semantic similarity between query and hypothetical question** compared to what we'd have for an actual chunk.
- There is also the reversed logic apporach called HyDE — you ask an LLM to generate a hypothetical response given the query and then use its vector along with the query vector to enhance search quality.

1 Query → generate hypothetical document
2 Convert it to embedding
3 Use embedding for retrieval

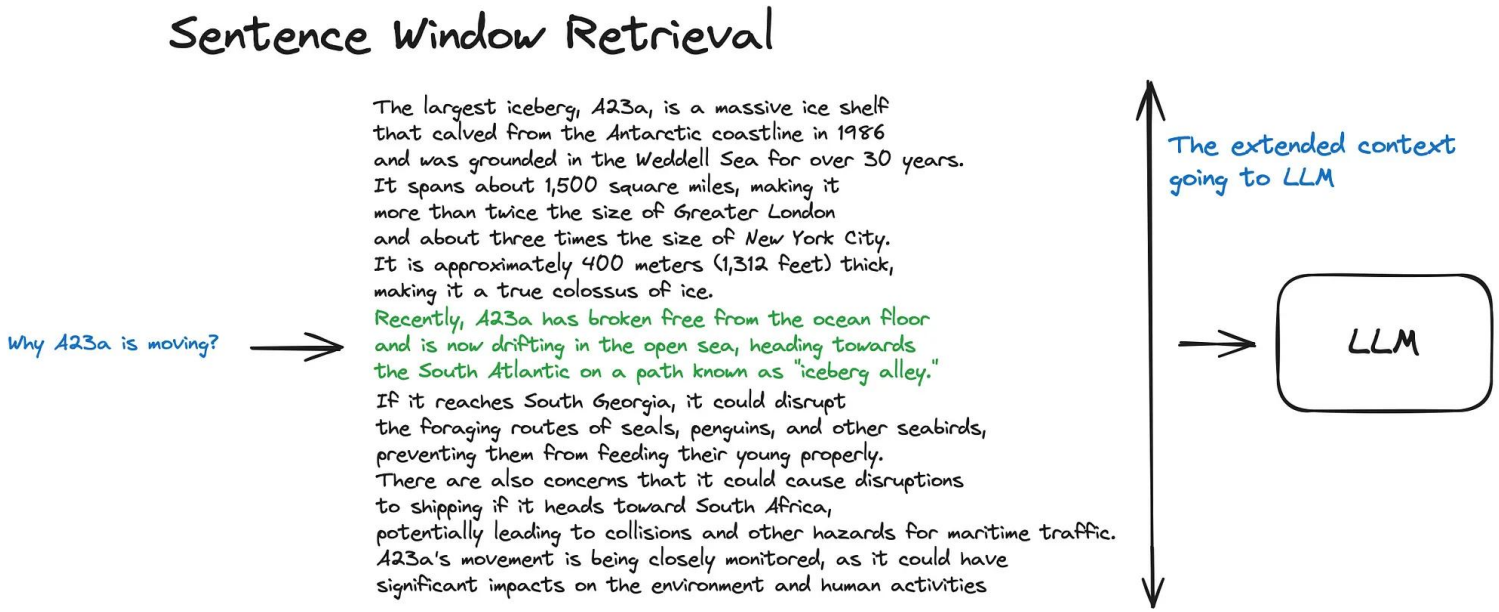
This improves semantic similarity.

GenerativeAI and Its Applications

Search index- Context enrichment (Retrieve **one sentence**, then extend surrounding context.**)**

- **The concept here is to retrieve smaller chunks for better search quality, but add up surrounding context for LLM to reason upon.**
- There are two options — to expand context by sentences around the smaller retrieved chunk or to split documents recursively into a number of larger parent chunks, containing smaller child chunks.

1.Sentence Window Retrieval

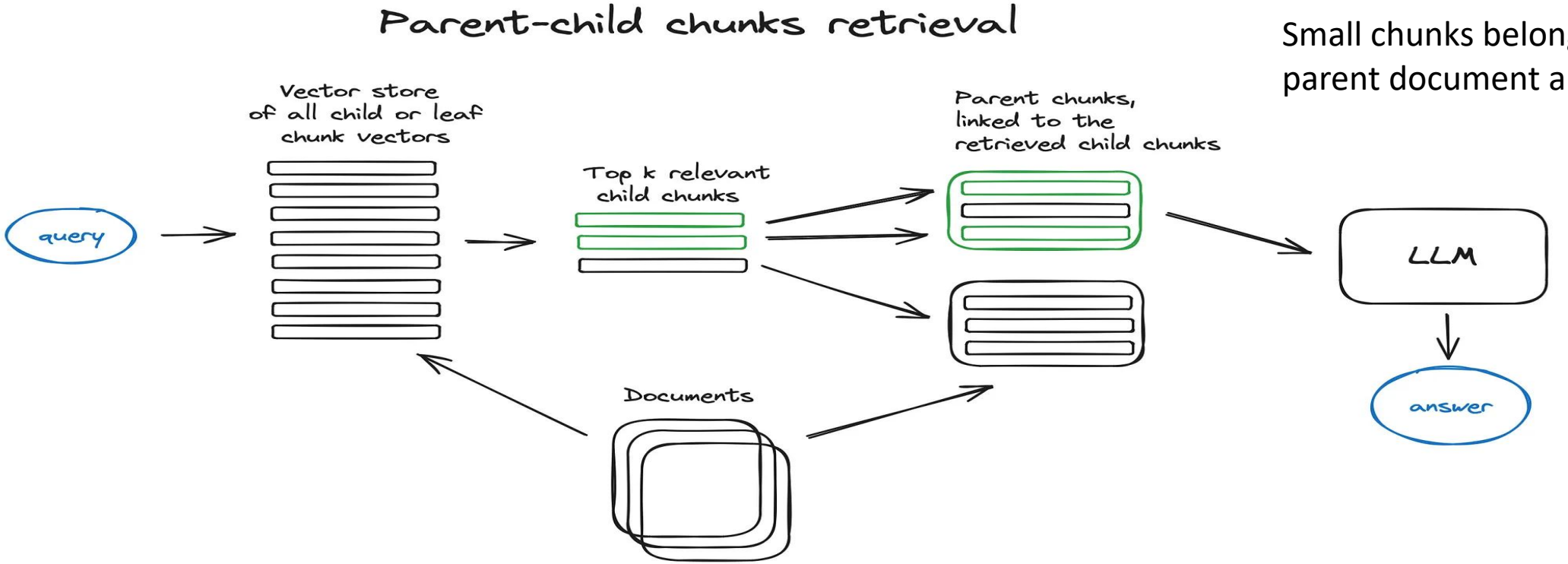


- In this scheme each sentence in a document is embedded separately which provides great accuracy of the query to context cosine distance search.
- In order to better reason upon the found context after fetching the most relevant single sentence **we extend the context window by *k* sentences before and after the retrieved sentence** and then send this extended context to LLM.

GenerativeAI and Its Applications

Search index- Context enrichment 2. Auto-merging Retriever (aka Parent Document Retriever)

- The idea here is pretty much similar to Sentence Window Retriever: to search for more granular pieces of information and then to extend the context window before feeding said context to an LLM for reasoning. Documents are split into smaller child chunks referring to larger parent chunks.



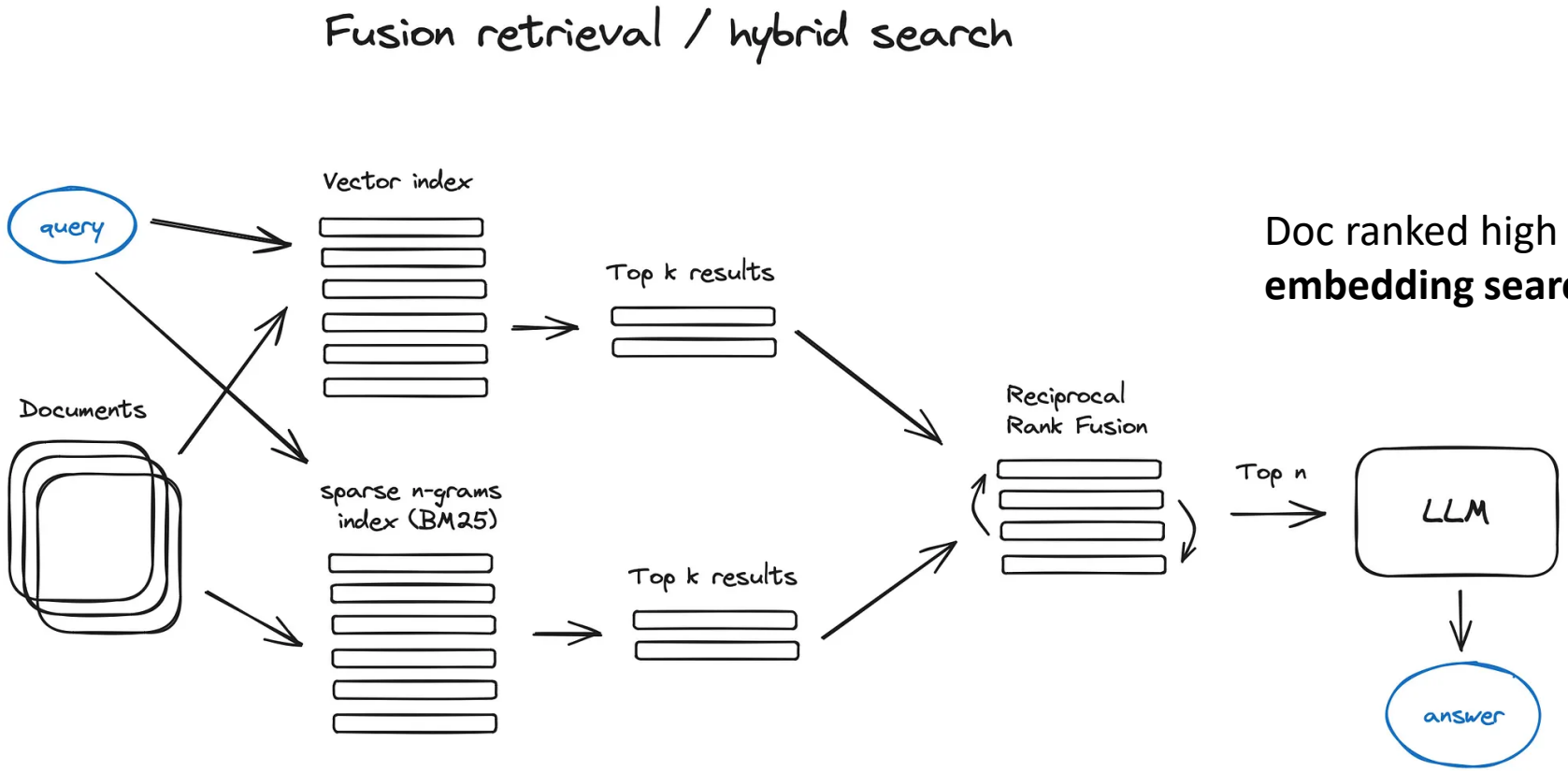
Small chunks belonging to same parent document are merged.

- Fetch smaller chunks during retrieval first, then if more than *n* chunks in top *k* retrieved chunks are linked to the same parent node (larger chunk)
- We replace the context fed to the LLM by this parent node — works like auto merging a few retrieved chunks into a larger parent chunk, hence the method name.

GenerativeAI and Its Applications

Fusion retrieval or hybrid search

- To properly combine the retrieved results with different similarity scores — this problem is usually solved with the help of the **Reciprocal Rank Fusion algorithm**, reranking the retrieved results for the final output.



Doc ranked high by **both keyword and embedding search** becomes top result.

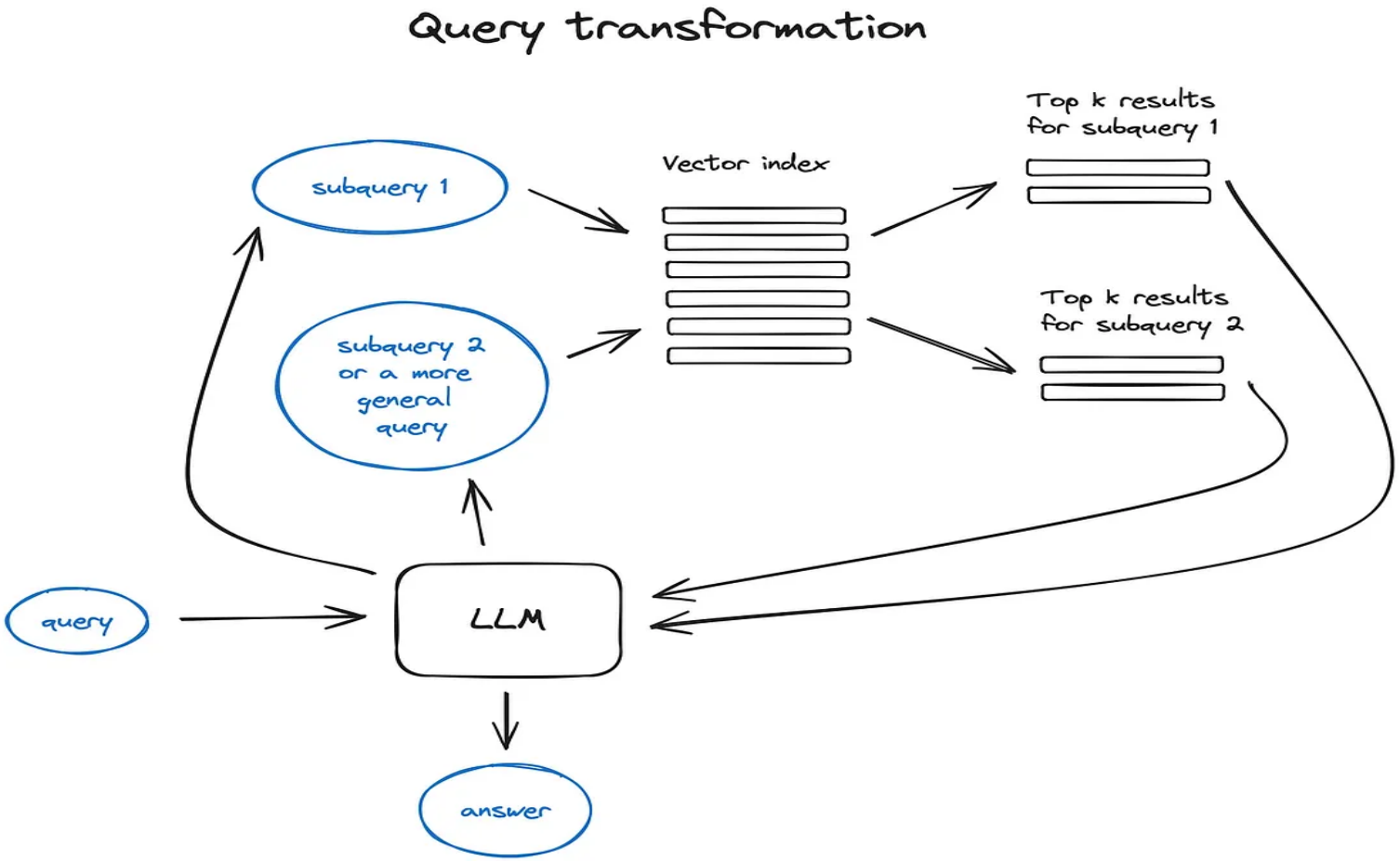
- Hybrid or fusion search usually provides better retrieval results as two complementary search algorithms are combined, taking into account both semantic similarity and keyword matching between the query and the stored documents.

Reranking & filtering

- Now we got our retrieval results with any of the algorithms described above, it is time to refine them through filtering, re-ranking or some transformation.
- In LlamaIndex there is a variety of available **Postprocessors**, **filtering out results based on similarity score, keywords, metadata or reranking them with other models** like an LLM, sentence-transformer cross-encoder, Cohere reranking endpoint or based on metadata like date recency — basically, all you could imagine.
- This is the final step before feeding our retrieved context to LLM in order to get the resulting answer.

GenerativeAI and Its Applications

Query transformations/ Decomposition



If the query is complex, LLM can decompose it into several sub queries.

Anology:

if you ask:

— *"What framework has more stars on Github, Langchain or LlamaIndex?",*

— *"How many stars does Langchain have on Github?"*

— *"How many stars does Llamaindex have on Github?"*

They would be executed in parallel and then the retrieved context would be combined in a single prompt for LLM to synthesize a final answer to the initial query.

USE CASES OF ARAG?

Company	Use Case	Naive RAG Problem	Advanced RAG Solution
JPMorgan	Financial Document Analysis	Missed nuanced regulatory requirements	Query expansion + Multi-stage retrieval
Merck	Medical Literature Q&A	Retrieved irrelevant drug studies	Reranking + Domain-specific embeddings
Uber	Customer Support	High latency, irrelevant answers	Hybrid search + Caching
Notion	AI Q&A over notes (Knowledge Search)	Couldn't handle complex queries	Query rewriting + Recursive retrieval

RETRIEVAL ENHANCEMENT TECHNIQUES - INTRODUCTION

WHAT IS RETREIVAL ENHANCEMENT ?

A set of techniques applied during the retrieval phase to improve the quality, relevance, and comprehensiveness of retrieved documents before they reach the LLM.

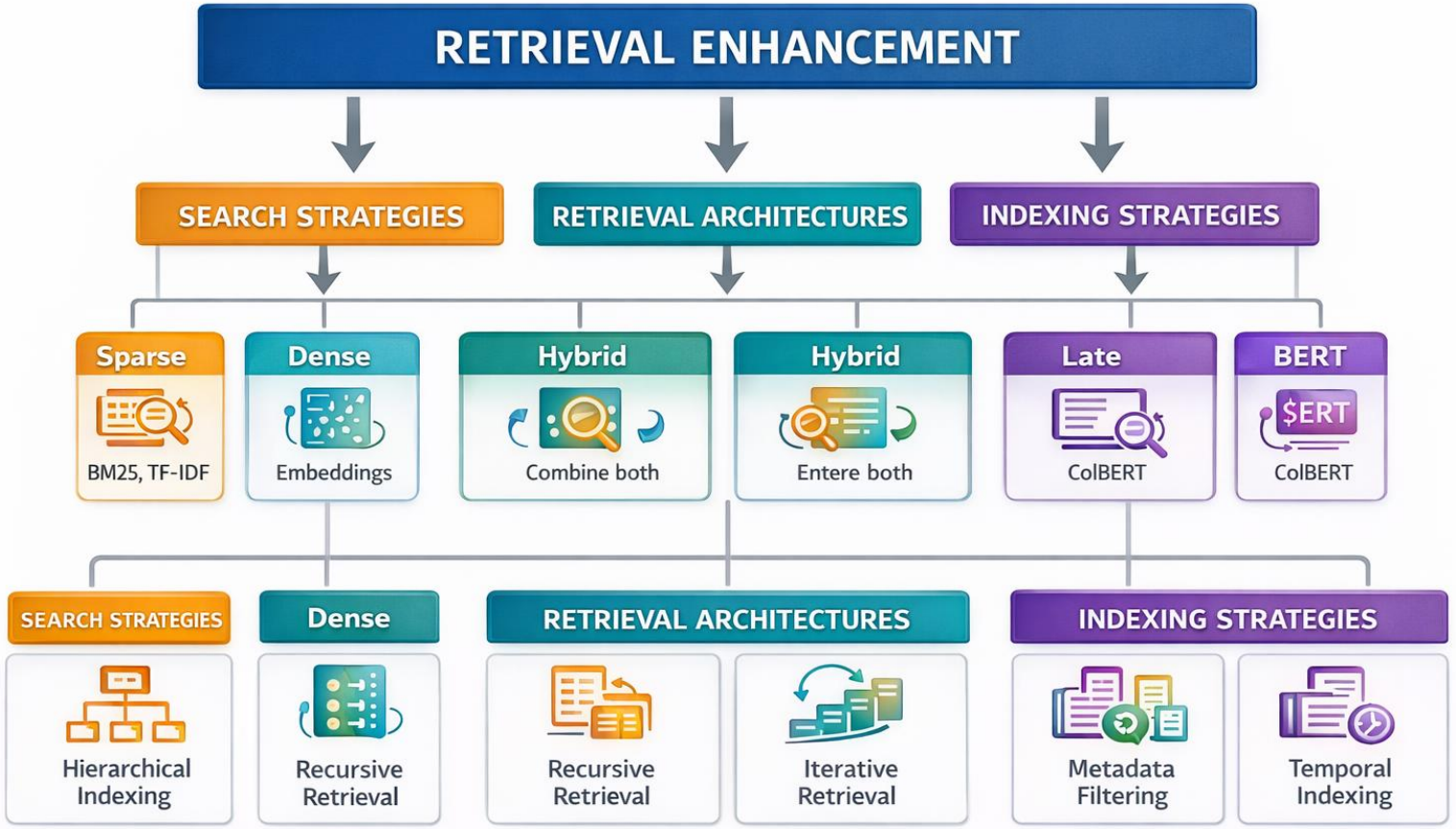
Retrieval Enhancement Techniques are methods used to improve the quality, relevance, and accuracy of information retrieved in Retrieval-Augmented Generation (RAG) systems. In traditional retrieval systems, the model searches for documents based on a simple query, which may lead to irrelevant or incomplete results. Retrieval enhancement techniques address this limitation by improving how queries are processed and how documents are searched and selected.

Why Retrieval Matters in RAG

Issue	Example
Hallucination	LLM invents facts
Outdated knowledge	Model trained in 2023
Can Not Access private data	Company docs unavailable

RETRIEVAL ENHANCEMENT TECHNIQUES - INTRODUCTION

RETREIVAL ENHANCEMENT Landscape?



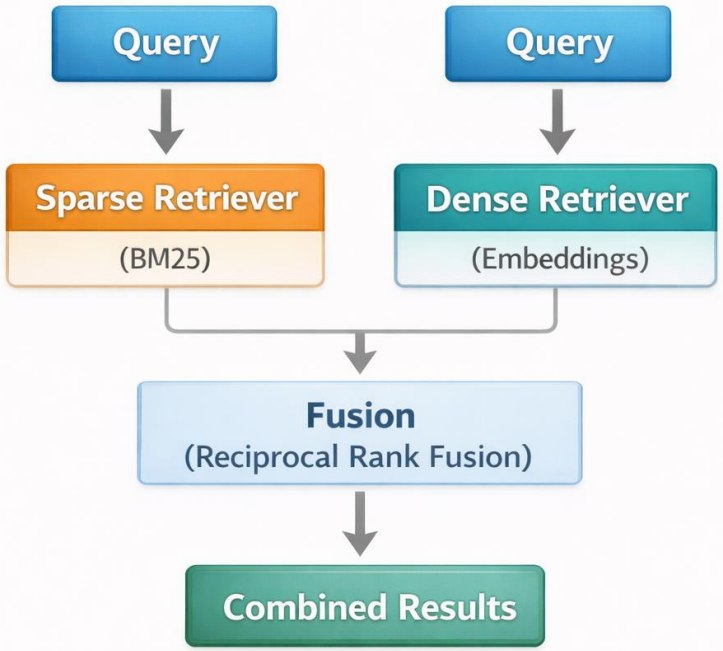
Sparse vs Dense Retrieval -

Sparse Retrieval (BM25, TF-IDF):
[cat, sat, on, mat] → {cat:1, sat:1, on:1, mat:1}
(Bag of words, high-dimensional sparse vector)
Based on: Exact keyword matching
Strength: Precision, interpretability
Weakness: Vocabulary mismatch ("car" vs "automobile")
Analogy: Library card catalog - looks for exact book titles

Dense Retrieval (Embeddings)
[cat, sat, on, mat] → [0.23, -0.45, 0.67, ...] (dense vector)
Based on: Semantic similarity
Strength: Understands concepts, synonyms
Weakness: Can miss exact matches, less interpretable
Analogy: Librarian who understands concepts - finds books about "felines" when you ask for "cats"

Feature	Sparse Retrieval	Dense Retrieval
Representation	Bag of Words	Embeddings
Example	BM25	BERT / E5
Strength	Exact matches	Semantic similarity
Weakness	Vocabulary mismatch	May miss keywords

Hybrid retrieval flowchart process?



Why Hybrid?

- BM25 catches **exact terms** (product codes, names)
- Embeddings catch **semantic meaning** (concepts, synonyms)
- Together it is a comprehensive retrieval

Hybrid scoring:

```
Final Score = RRF(BM25 rank) + RRF(Dense rank)

If document ranks high in both → very relevant
```

Multi-Stage Retrieval process?

Stage 1: Coarse Retrieval
| Query → IVF/HNSW → Top 100 candidates
| (Fast, approximate, high recall)
|
Stage 2: Fine Re-ranking
| Top 100 → Cross-encoder → Top 10 candidates
| (Slower, precise, high precision)
|
Stage 3: Context Selection
| Top 10 → Deduplication → Final 5 chunks
| (Quality control)

Analogy –

1.)Medical Diagnosis:

Stage 1 (Triage Nurse): Quick assessment → 100 possible conditions
Stage 2 (General Doctor): Detailed check → 10 likely conditions
Stage 3 (Specialist): Final diagnosis → 1 confirmed condition

2.)Google Search Pipeline

1. BM25 retrieves 1000 docs
2. BERT reranker scores them
3. Top 10 results shown

Multi-Stage Retrieval

Recursive Retrieval Retrieval process? System repeatedly retrieves deeper documents.

Retrieve → Extract entities → Retrieve more based on entities → Repeat

Query: "What were the side effects of the COVID vaccine trials in 2020?"

Round 1:

— Retrieve: Documents about COVID vaccines
— Extract: "Pfizer", "Moderna", "Phase 3 trials", "adverse events"
— Identify gaps: Need specific trial data

Round 2:

— New queries: "Pfizer Phase 3 adverse events 2020", "Moderna clinical trial safety data"
— Retrieve: Specific trial documents
— Identify gaps: Need regulatory reports

Round 3:

— New query: "FDA adverse event reporting system COVID 2020"
— Retrieve: Regulatory safety reports
Final: Combine all retrieved context

Query
↓
Retrieve docs
↓
Extract entities
↓
New queries
↓
Retrieve again

Step-Back Retrieval Retrieval process? Ask a **more general question first**.

The Problem: LLMs often fail on questions requiring high-level reasoning

Original Query: "What caused the temperature anomaly in February 2023?"

Step 1: Step-back to concept
└── "What factors influence temperature variations?"

Step 2: Retrieve general principles
└── Documents about climate patterns, El Niño, greenhouse effects

Step 3: Original retrieval
└── Specific documents about Feb 2023 weather data

Step 4: Combine both
└── General principles + Specific data → Better reasoning

Why does this work:
Original query = specific facts
Step-back query = conceptual knowledge
LLM needs both for reasoning

LLM generates a higher-level query

Example:
Original:
Why temperature anomaly Feb 2023?
Step-back:
What factors influence temperature?

Self-Querying Retrieval Retrieval process?

LLM extracts metadata filters from natural language query

Query: "Show me documents about machine learning published by Google in 2023"

Step 1: LLM parses query into:

```
{
  "query": "machine learning",
  "filter": {
    "author_org": "Google",
    "year": {"$gte": 2023, "$lt": 2024}
  }
}
```

Step 2: Retrieve using:

- Semantic search on "machine learning"
- Metadata filter applied

ADVANTAGES:

- Handles complex natural language
- Reduces irrelevant results
- Enables structured search over unstructured queries

ANOLOGY:

"AI papers by Microsoft after 2022"

Parsed to:

query = "AI papers"

filter = {author_org:"Microsoft", year>2022}

GenerativeAI and Its Applications

Retrieval Enhancement - Summary Table

Technique	When to Use	Pros	Cons
Hybrid Search	Always in production	Balanced results	More complex
Multi-stage	High precision needed	Very accurate	Slower, costlier
Recursive	Multi-hop questions	Deep understanding	Multiple LLM calls
Step-Back	Reasoning questions	Better conceptual	May over-generalize
Self-Query	Structured data + NLP	Precise filtering	Requires schema knowledge

Metrics - How We Measure Advanced RAG

Retrieval Metrics:

Metric	Formula	What it Measures
Recall@k	(relevant docs retrieved) #relevant retrieved / #total relevant	Did we find all good chunks?
Precision@k	(correctness of retrieved docs) #relevant retrieved / k	Are retrieved chunks actually good?
MRR (Mean Reciprocal Rank)	(ranking quality) 1/rank_first_relevant	How early is first good result?
NDCG@k	(ranking effectiveness) Normalized Discounted Cumulative Gain	Ranking quality (position matters)

WHY BM25?

User Query
↓
BM25 retrieves top 20 chunks
↓
Dense reranker selects top 5
↓
LLM generates answer

Introduction to Sparse Retrieval?

Analogy:
Sparse retrieval is like a **library card catalog** - each card lists exactly which books contain which words. To find books about "quantum computing," you look under "quantum" and "computing" cards.

A family of retrieval methods that represent documents and queries as sparse vectors (mostly zeros) based on term occurrence.

Key Characteristics:

- **Vocabulary-based:** Each dimension = a word/term
- **Interpretable:** We know exactly why a document matched
- **Efficient:** Inverted indexes enable fast lookup
- **Precise:** Exact matches are guaranteed

BM25 - Quick Recap from Previous Class

BM25 Scoring Formula

$$Score(q, d) = \sum_{t \in q} IDF(t) \times \frac{TF(t, d) \times (k_1 + 1)}{TF(t, d) + k_1 \left(1 - b + b \times \frac{|d|}{avgdl}\right)}$$

Where

- q= query
- d= document
- t= query term
- $TF(t, d)$ =term frequency of term $t$in document d
- $|d|$= length of the document
- $avgdl$= average document length in the collection
- k_1 =term frequency scaling parameter (usually **1.2–2.0**)

IDF Formula

$$IDF(t) = \log\left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1\right)$$

Where:

- N= total number of documents
- n_t =number of documents containing term t

BM25 - The Intuition Behind Parameters

IDF Formula (BM25)

$$IDF(t) = \log\left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1\right)$$

Where:

- t= term
- N= total number of documents in the collection
- n_t =number of documents containing term t

Meaning

- If a term appears in **many documents**, $n_t$is large → **IDF decreases** (less important term).
- If a term appears in **few documents**, $n_t$is small → **IDF increases** (more important term).

Example

If:

- $N = 1000$documents
- $n_t = 10$documents contain the term

$$IDF(t) = \log\left(\frac{1000 - 10 + 0.5}{10 + 0.5} + 1\right)$$

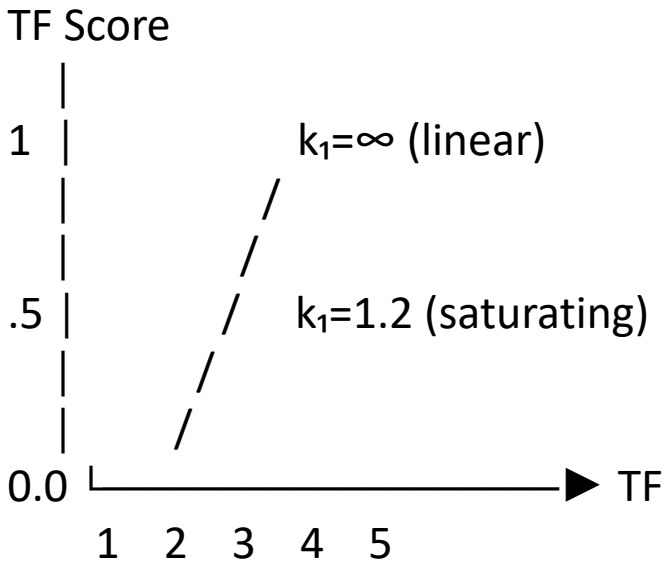
$$IDF(t) \approx \log(95.29) \approx 4.55$$

So the term receives a **high importance weight**.

Effect of k_1:

- $k_1 = 0$: Binary model (TF = 0 or 1 only)
- $k_1 \to \infty$: No saturation (TF grows linearly)
- $k_1 = 1.2$-2.0: Typical range (saturates after 2-3 occurrences)

Visual Representation:



The b Parameter (Length Normalization)

Formula Component: $(1 - b + b \times (|d|/avgdl))$

Effect of b:
- b = 0: No length normalization (ignore document length)
- b = 1: Full length normalization (penalize longer docs)
- b = 0.75: Typical default (balanced)

Example Calculation:
avgdl = 100 words
|d| = 200 words (twice average)

b = 0: $1 - 0 + 0 \times 2.0 = 1.0$ (no penalty)
b = 0.75: $1 - 0.75 + 0.75 \times 2.0 = 0.25 + 1.5 = 1.75$ (penalty factor)
b = 1.0: $1 - 1 + 1 \times 2.0 = 2.0$ (max penalty)

TF Score Multiplier = $(k_1 + 1)/(f + k_1 \times length\_factor)$
Higher length_factor = Lower TF contribution

Analogy - Shopping:
A 1000-page book mentioning "apple" 10 times is less about apples than a 10-page pamphlet mentioning "apple" 5 times. Length normalization captures this.

BM25 - Complete Numerical Example

Corpus: 1,000 documents
Query: "quantum computing applications"
Document d: 150 words, contains:
"quantum": 4 times
"computing": 3 times
"applications": 2 times
Average document length (avgdl): 120 words
BM25 parameters: $k_1 = 1.5$, $b = 0.75$

Document length: |d| = 150
Average length: avgdl = 120

Term "quantum": appears in 10 documents
Term "computing": appears in 50 documents
Term "applications": appears in 200 documents

- **Calculate IDF for each term**
- **Calculate Length Normalization Factor**
- **Calculate TF Component for each term**
- **Calculate Final BM25 Score**
- **Give Interpretation of each term in terms of appearance and weightage**

Step 1: Calculate IDF for each term

Corpus: 1,000 documents
Query: "quantum computing applications"
Document d: 150 words, contains:
"quantum": 4 times
"computing": 3 times
"applications": 2 times
Average document length (avgdl): 120 words
BM25 parameters: $k_1 = 1.5$, $b = 0.75$

(Leaving 1 for convenience as this the smoothing factor)

Term "quantum": appears in 10 documents
IDF(quantum) = log((1000 - 10 + 0.5)/(10 + 0.5))
= log(990.5/10.5)
= log(94.33)
= 4.55

Term "computing": appears in 50 documents
IDF(computing) = log((1000 - 50 + 0.5)/(50 + 0.5))
= log(950.5/50.5)
= log(18.82)
= 2.93

Term "applications": appears in 200 documents
IDF(applications) = log((1000 - 200 + 0.5)/(200 + 0.5))
= log(800.5/200.5)
= log(3.99)
= 1.38

Step 2: Calculate Length Normalization Factor

Document length: |d| = 150
Average length: avgdl = 120

Length factor = 1 - b + b × (|d|/avgdl)
= 1 - 0.75 + 0.75 × (150/120)
= 0.25 + 0.75 × 1.25
= 0.25 + 0.9375
= 1.1875

Step 3: Calculate TF Component for each term:

For "quantum": f = 4
TF(quantum) = (f × (k_1 + 1)) / (f + k_1 × length_factor)
= (4 × (1.5 + 1)) / (4 + 1.5 × 1.1875)
= (4 × 2.5) / (4 + 1.78125)
= 10 / 5.78125
= 1.73

For "computing": f = 3
TF(computing) = (3 × 2.5) / (3 + 1.78125)
= 7.5 / 4.78125
= 1.57

For "applications": f = 2
TF(applications) = (2 × 2.5) / (2 + 1.78125)
= 5 / 3.78125
= 1.32

Step 4: Calculate Final BM25 Score

Score(q,d) = Σ [IDF(t) × TF(t,d)]

Score = (4.55 × 1.73) + (2.93 × 1.57) + (1.38 × 1.32)
= 7.87 + 4.60 + 1.82
= 14.29

Interpretation:
7.87 from "quantum" (rare term, high weight)
4.60 from "computing" (moderately rare)
1.82 from "applications" (common term, low weight)

Scenario 1: Increase "quantum" frequency from 4 to 10

TF(quantum) new = (10 × 2.5) / (10 + 1.78125)
= 25 / 11.78125
= 2.12 (was 1.73)

New contribution = 4.55 × 2.12 = 9.65 (was 7.87)
Total new score = 9.65 + 4.60 + 1.82 = 16.07

Observation: 150% increase in frequency → only 12% increase in score
Reason: TF SATURATION!

Long documents are penalized.
Example:
150 word doc → higher score
300 word doc → lower score.

Increasing frequency from 4 → 10 does **not increase score proportionally**.
Reason:
BM25 prevents keyword stuffing.

Scenario 2: Document length increases to 300 words

New length factor = 1 - 0.75 + 0.75 × (300/120)
= 0.25 + 0.75 × 2.5
= 0.25 + 1.875
= 2.125

TF(quantum) new = (4 × 2.5) / (4 + 1.5 × 2.125)
= 10 / (4 + 3.1875)
= 10 / 7.1875
= 1.39 (was 1.73)

Parameter **k1**
Controls TF saturation.
Parameter **b**
Controls document length normalization.

New quantum contribution = 4.55 × 1.39 = 6.32 (was 7.87)
Total new score = 6.32 + 3.68 + 1.45 = 11.45 (was 14.29)

Observation: Doubling length → 20% score decrease
Reason: LENGTH NORMALIZATION!

GenerativeAI and Its Applications

BM25 - Parameter Tuning Guide

Effect of k_1 (Term Frequency Saturation)

k_1 Value	Behavior	Use Case
0 - 0.5	Strong saturation	Short documents, many repeated terms
1.2 - 1.5	Moderate saturation (default)	General web search
2.0+	Weak saturation	Long documents, need term frequency signal

Effect of b (Length Normalization)

b Value	Behavior	Use Case
0	No length normalization	All documents same length
0.5 - 0.75	Moderate normalization (default)	Mixed document lengths
1.0	Full normalization	Very varied lengths

Web search:
$k_1 = 1.2$, $b = 0.75$
Academic papers:
$k_1 = 1.5$, $b = 0.5$ (length matters less)
Social media:
$k_1 = 0.5$, $b = 1.0$ (short, varied)

GenerativeAI and Its Applications

BM25 vs TF-IDF - Numerical Comparison

TF-IDF Formula: Score = Σ [TF × IDF]
where TF = raw frequency (no saturation)

Term "quantum": TF = 4, IDF = 4.55 → Contribution = 18.2
Term "computing": TF = 3, IDF = 2.93 → Contribution = 8.79
Term "applications": TF = 2, IDF = 1.38 → Contribution = 2.76

TF-IDF Score = 18.2 + 8.79 + 2.76 = 29.75

BM25 Score (from earlier) = 14.29

BM25 still uses TF and IDF, but TF is replaced by a **normalized TF function** that prevents keyword stuffing and corrects for document length.

Term	Frequency	TF-IDF Weight	BM25 Weight	Why Different?
quantum	4	18.2	7.87	BM25 saturates high frequencies
computing	3	8.79	4.60	BM25 saturates
applications	2	2.76	1.82	BM25 saturates
TOTAL		**29.75**	**14.29**	BM25 more conservative

BM25 with Query Term Weighting

Advanced: Queries have term weights too

If a term appears **multiple times in query**, its importance increases.

Extended BM25 Scoring Formula

$$Score(q, d) = \sum_{t \in q} IDF(t) \times \frac{TF(t, d) \times (k_1 + 1)}{TF(t, d) + k_1 \left(1 - b + b \times \frac{|\, d \,|}{avgdl}\right)} \times \frac{(k_3 + 1)\, qtf}{k_3 + qtf}$$

Where:

- q= query
- d= document
- t= query term
- $TF(t, d)$ =term frequency of term $t$in document d
- qtf= term frequency in the **query**
- $IDF(t)$ =inverse document frequency of term t
- $|\, d \,|$= document length
- $avgdl$= average document length in the collection
- k_1 =term frequency scaling parameter (typically **1.2 – 2.0**)
- k_3 =query term frequency parameter
- b= document length normalization parameter (**0.75 commonly**)

IDF Formula

$$IDF(t) = \log\left(\frac{N - n_t + 0.5}{n_t + 0.5} + 1\right)$$

Where:

- N= total number of documents
- n_t =number of documents containing term t

BM25 with Query Term Weighting

Advanced: Queries have term weights too

Query

"quantum computing quantum algorithms"

The term **“quantum” appears twice** in the query.

So,

$$qtf = 2$$

Step 1: Query Term Frequency Factor

The query factor in **Extended BM25** is:

$$\frac{(k_3 + 1) \times qtf}{k_3 + qtf}$$

Case 1: Large k_3)Typical Implementation)

Let

$$k_3 = 1000$$

$$\frac{(1000 + 1) \times 2}{1000 + 2} = \frac{2002}{1002} \approx 1.998 \sim 2$$

Case 2: Smaller k_3

Let

$$k_3 = 1.2$$

$$\frac{(1.2 + 1) \times 2}{1.2 + 2} = \frac{2.2 \times 2}{3.2} = 1.375$$

BM25 with Query Term Weighting

Advanced: Queries have term weights too

Step 2: Document Scoring Example

Parameter	Value
IDF(quantum)	4.55
TF normalization	1.73
Query factor	1.375

Without Query Frequency Factor

$$Score = 4.55 \times 1.73$$
$$Score = 7.87$$

With Query Frequency Factor

$$Score = 4.55 \times 1.73 \times 1.375$$
$$Score = 10.82$$

- When **query terms repeat**, BM25 can increase their importance.
- **Large** k_3 →query frequency effect is minimal.
- **Smaller** k_3 →repeated terms influence ranking more.

"quantum" appears twice in query (qtf = 2)
With k_3 = 1000 (large): factor ≈ 1 (qtf ≈ 1 effectively)
With k_3 = 1.2: factor = (1.2 + 1) × 2 / (1.2 + 2) = 2.2 × 2 / 3.2 = 1.375
Quantum contribution becomes: 4.55 × 1.73 × 1.375 = 10.82 (was 7.87)

GenerativeAI and Its Applications

BM25 Numerical Exercise

Problem:
Given a corpus of 500 documents, query = "neural networks training", document d has:

- Length = 200 words
- Contains "neural": 5 times (appears in 20 docs)
- Contains "networks": 4 times (appears in 25 docs)
- Contains "training": 3 times (appears in 100 docs)
- Average document length = 150 words
- $k_1 = 1.2$, $b = 0.75$

Calculate:

1. IDF for each term
2. Length normalization factor
3. TF component for each term
4. Final BM25 score

BM25 Numerical Exercise

Step 1: IDF Calculation

$$IDF(neural) = log\big((500 - 20 + 0.5)/(20 + 0.5)\big)$$
$$= log(23.44) = 3.15$$
$$IDF(networks) = log(18.65) = 2.93$$
$$IDF(training) = log(3.98) = 1.38$$

Step 2: Length Normalization

$$Length\ factor = 1 - b + b \times \frac{|\ d\ |}{avgdl}$$
$$= 0.25 + 0.75 \times (200/150)$$
$$= 1.25$$

Step 3: TF Components

$$TF(neural) = \frac{5 \times 2.2}{5 + 1.2 \times 1.25} = 1.69$$
$$TF(networks) = 1.60$$
$$TF(training) = 1.47$$

Step 4: Final BM25 Score

$$Score$$
$$= 3.15 \times 1.69 + 2.93 \times 1.60 + 1.38 \times 1.47$$
$$Score = 5.32 + 4.69 + 2.03$$
$$Final\ Score = 12.04$$

THANK YOU

Generative AI and Its Applications

Unit 3

(UE23CS342BA4)

Dense Retrieval and Hybrid Retrieval

Dr. Pooja Agarwal

Dept. of CSE, PES University, Bangalore

Acknowledgement

The slides are prepared from various resources from the Universities from abroad and India. Also, some material is taken from reliable resources from internet throughout this course.

The Vocabulary Mismatch Problem

The Fundamental Challenge in Information Retrieval!

Query: "How to fix a car that won't start?"

Traditional Sparse Retrieval (BM25) looks for:
"fix" AND "car" AND "won't" AND "start"

But relevant documents might say:
"automobile engine troubleshooting guide" No match!
"vehicle ignition problems solutions" No match!
"jump-start dead battery procedure" No match!
"starter motor replacement steps" No match!

THEREFORE Result: ZERO relevant documents retrieved!

Sparse retrieval fails because keywords differ.

Analogy:
Sparse retrieval is like a librarian who only looks for exact book titles. If you ask for "automobile repair," they ignore books about "car maintenance" because the words don't match exactly.

What is Dense Retrieval?

Dense Retrieval!

It Explains - **how modern search systems find relevant documents** using AI instead of just keywords.

Dense Retrieval is a technique that represents queries and documents as dense vectors (embeddings) in a continuous semantic space, where similarity is measured by vector distance rather than keyword matching.

Text → Neural Encoder → Vector

"cat" → [0.23, -0.45, 0.67, 0.12, ..., 0.34] (768 dimensions)
"feline" → [0.21, -0.44, 0.65, 0.15, ..., 0.36] (similar vectors!)

Vectors with similar meaning are close together.

GenerativeAI and Its Applications

What is Dense Retrieval?

Dense Retrieval!

Dense Retrieval is a technique that represents queries and documents as dense vectors (embeddings) in a continuous semantic space, where similarity is measured by vector distance rather than keyword matching.

Step-by-Step Steps:

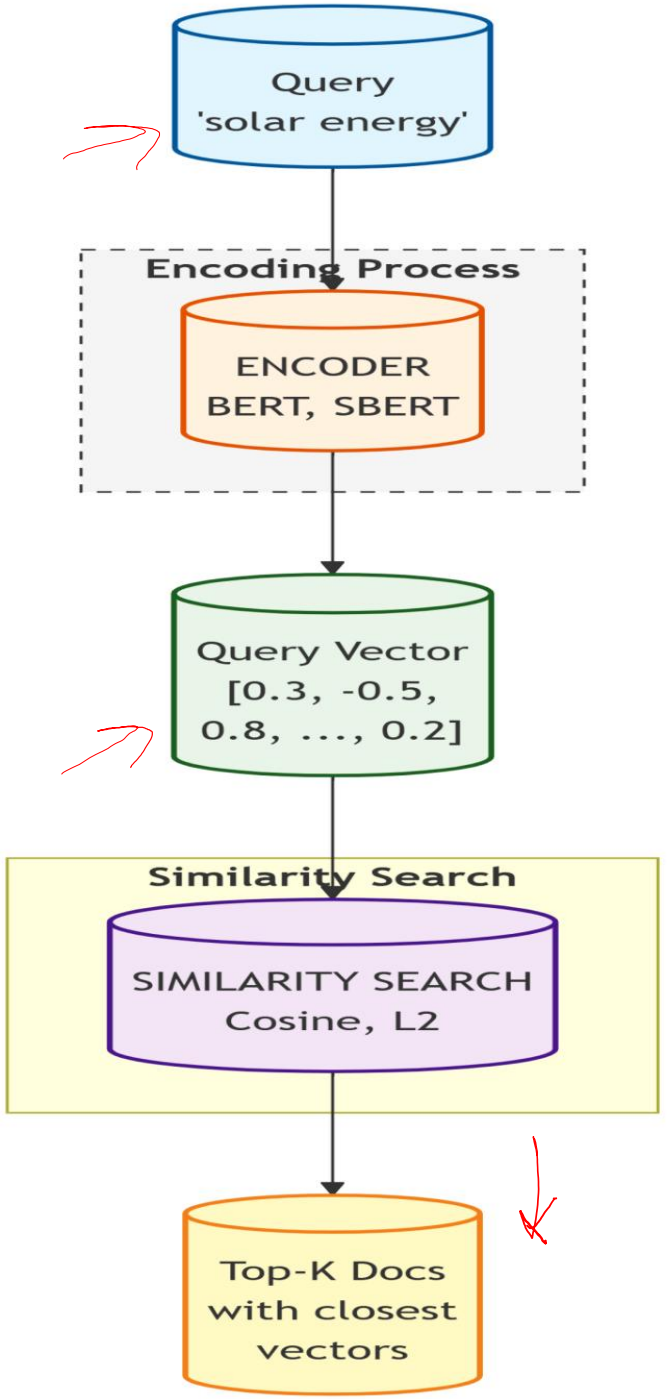
1. Encode documents into embeddings
2. Store embeddings in vector database
3. Encode query
4. Compute similarity
5. Retrieve top-K documents

Architecture:

Query → Encoder → Vector

Docs → Encoder → Vectors

Similarity → Ranking



1.Encode query → dense vector

2.Encode all documents → dense vectors (pre-computed)

3.Compare query vector to all document vectors

4.Return documents with highest similarity scores

What is Dense Retrieval?

Dense V/S Sparse Overview!

Aspect	Sparse Retrieval (BM25)	Dense Retrieval (Embeddings)
Representation	Bag-of-Words / Term Frequency	Semantic Vector Embeddings
Dimensions	Vocabulary size (50K – 1M)	Fixed small vector (384 – 768)
Vector Structure	Very sparse (≈99.9% zeros)	Dense (almost all values non-zero)
Matching Method	Exact keyword matching	Semantic similarity matching
Synonym Handling	Misses synonyms	Captures synonyms
Polysemy (multiple meanings)	Often confused	Context-aware understanding
Interpretability	High (terms visible)	Low (latent features)
Training Required	No training needed	Requires trained model
Speed	Very fast	Slower (but optimized with ANN search)

Query: "bank"
Sparse retrieves:
- Documents with "bank" (both river bank AND financial bank)

Dense with context:
- "I went to the bank to deposit money" → Financial context
- "The river bank eroded" → Geographic context

GenerativeAI and Its Applications

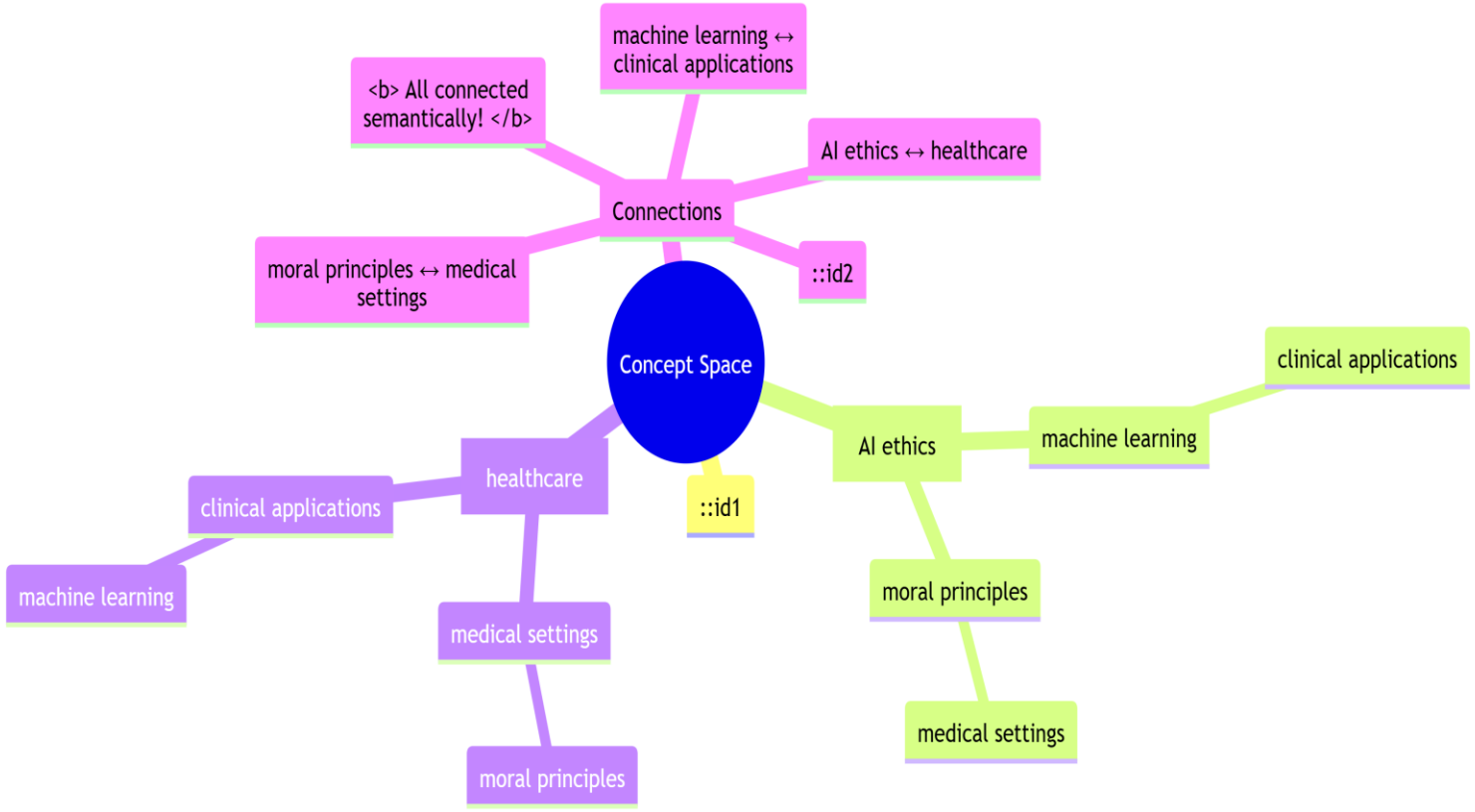
Why Dense Retrieval? The Semantic Gap

Query: "AI ethics guidelines for healthcare"

Documents that should match but don't contain these exact words:

1. "Artificial intelligence moral principles in medical settings"
2. "Machine learning ethical frameworks for clinical applications"
3. "Responsible AI deployment in hospitals: best practices"

BM25 Score: 0 (no common terms!)
Dense Score: High (similar meaning!)



Why Dense Retrieval? The Semantic Gap

Words with **similar meaning form clusters**.

Advantages:

- Understands synonyms
- Understands context
- Supports semantic search

EXAMPLE:

car – automobile – vehicle
dog – puppy – canine
doctor – physician

GenerativeAI and Its Applications

Similarity Measurement

document = "The cat sat on the mat"

↓

Embedding Model (BERT/SBERT)

↓

Vector d = [0.23, -0.56, 0.78, 0.12, ..., -0.34] $\in \mathbb{R}^{768}$

Similarity Metrics:

1.Cosine Similarity (Most common)

$$\cos(\theta) = \frac{q \cdot d}{\|q\| \, \|d\|} = \frac{\Sigma(q\_i \times d\_i)}{\sqrt{(\Sigma q\_i^2)} \times \sqrt{(\Sigma d\_i^2)}}$$

Range: [-1, 1] → Higher = More similar

Value	Meaning
1	identical
0.8	highly similar
0	unrelated

2.Euclidean Distance (L2)

$$L2(q,d) = \sqrt{[\Sigma(q_i - d_i)^2]}$$

Range: [0, ∞) → Lower = More similar

3.Dot Product (when vectors are normalized)

$$dot(q,d) = \Sigma(q_i \times d_i)$$

Range: [-1, 1] → Higher = More similar

LETS SOLVE:

Dense Retrieval Numerical Example

2D vectors (simplified for demonstration)

Query: "solar power" [.8, .6]

•Document 1: "renewable energy" [.9, .4]

•Document 2: "financial investment" [.1, .2]

Step 1: Get Embeddings

GenerativeAI and Its Applications

Similarity Measurement

cosine similarity formula:

$$\cos(\theta) = (A \cdot B) / (||A|| \, ||B||)$$

Where

- $A \cdot B$ = dot product
- $||A||$ = magnitude of vector A

Step 1: Given Embeddings

Text	Vector
Query q = "solar power"	**[0.8 , 0.6]**
Doc1 d_1 = "renewable energy"	**[0.9 , 0.4]**
Doc2 d_2 = "financial investment"	**[0.1 , 0.2]**

Step 2: Compute Similarity with Document 1

Dot Product

$$q \cdot d_1 = (0.8 \times 0.9) + (0.6 \times 0.4)$$
$$= 0.72 + 0.24 = 0.96$$

Vector Magnitudes

$$|| q ||= \sqrt{0.8^2 + 0.6^2}$$
$$= \sqrt{0.64 + 0.36} = \sqrt{1.00} = 1.00$$
$$|| d_1 ||= \sqrt{0.9^2 + 0.4^2}$$
$$= \sqrt{0.81 + 0.16} = \sqrt{0.97} = 0.985$$

Cosine Similarity

$$cos(\theta_1) = \frac{0.96}{1.00 \times 0.985}$$
$$cos(\theta_1) = 0.975$$

=High similarity

Step 3: Compute Similarity with Document 2

Dot Product

$$q \cdot d_2 = (0.8 \times 0.1) + (0.6 \times 0.2)$$
$$= 0.08 + 0.12 = 0.20$$

Magnitude

$$|| d_2 || = \sqrt{0.1^2 + 0.2^2}$$
$$= \sqrt{0.01 + 0.04}$$
$$= \sqrt{0.05} = 0.224$$

Cosine Similarity

$$cos(\theta_2) = \frac{0.20}{1.00 \times 0.224}$$
$$cos(\theta_2) = 0.893$$

=Less similarity

GenerativeAI and Its Applications

Similarity Measurement

Document	Cosine Similarity	Relevance
Doc1 – renewable energy	**0.975**	Most Relevant
Doc2 – financial investment	**0.893**	Less Relevant

Doc1 (0.975) > Doc2 (0.893)

Therefore **"renewable energy"** is **more relevant to the query "solar power"**.

GenerativeAI and Its Applications

Training Dense Retrievers

How Do We Get Good Embeddings?

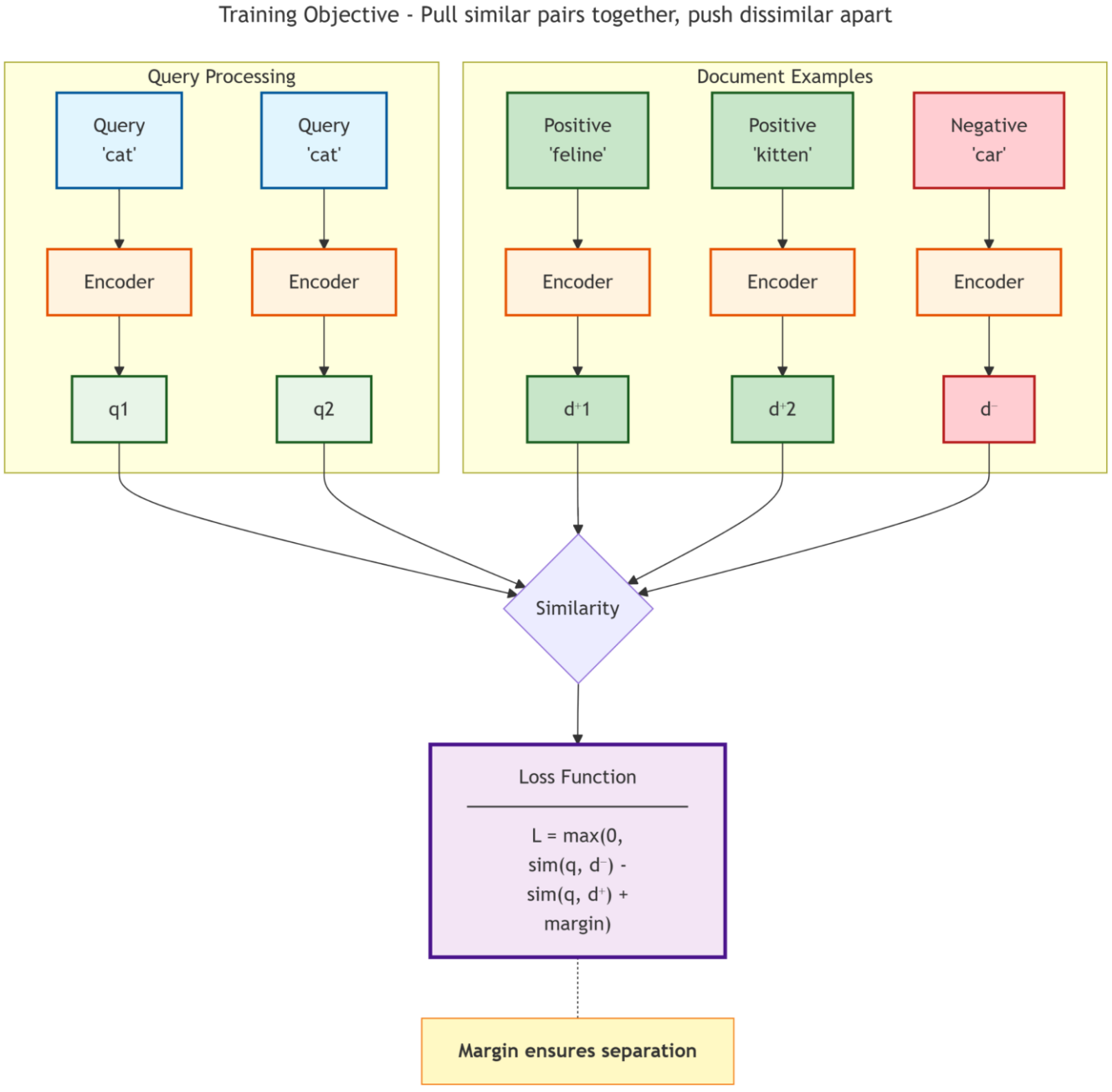
Triplet / Margin Ranking Loss (Used in Dense Retrieval Training)

Loss = max(0, cos(q,d⁻) - cos(q,d⁺) + margin)

This loss function is used to **train dense retrieval models like SBERT** so that:

- Relevant documents are **closer to the query**

Symbol	Meaning
q	Query embedding
d^+	Positive document (relevant)
d^-	Negative document (irrelevant)
cos(q,d)	Cosine similarity between query and document
margin	Minimum gap required between positive and negative similarity



How do embeddings learn meaning?

Training Dense Retrievers

How Do We Get Good Embeddings?

Example Calculation (Classroom Example)

Assume:

Value	Number
cos(q, d^+)	0.85
cos(q, d^-)	0.60
margin	0.20

Step 1: Substitute

```
Loss = max(0, 0.60 - 0.85 + 0.20)
```

Step 2: Solve

```
Loss = max(0, -0.05)
Loss = 0
```

Good training example

The model already satisfies the margin.

Value	Number
cos(q, d^+)	0.70
cos(q, d^-)	0.65
margin	0.20

```
Loss = max(0, 0.65 - 0.70 + 0.20)
Loss = max(0, 0.15)
Loss = 0.15
```

Bad example

The model must **adjust embeddings** so that:

```
cos(q,d+) increases
cos(q,d-) decreases
```

Dense Retrieval Similarity

Problem

A query and two documents are represented by **2-dimensional embeddings**.

Text	Vector
Query **q**	[0.6 , 0.8]
Document $\mathbf{d_1}$	[0.7 , 0.7]
Document $\mathbf{d_2}$	[0.2 , 0.1]

Task

1. Compute the **cosine similarity** between **q and $\mathbf{d_1}$**
2. Compute the **cosine similarity** between **q and $\mathbf{d_2}$**
3. Which document will the **dense retriever rank higher**?

Dense Retrieval Similarity Problem **Dense Retrieval Similarity**

Cosine Similarity Formula

cos(θ) = (A · B) / (||A|| ||B||)

Step 1: Compute Dot Products

For **Document 1**

$$q \cdot d_1 = (0.6 \times 0.7) + (0.8 \times 0.7)$$
$$= 0.42 + 0.56 = 0.98$$

For **Document 2**

$$q \cdot d_2 = (0.6 \times 0.2) + (0.8 \times 0.1)$$
$$= 0.12 + 0.08 = 0.20$$

Step 2: Compute Vector Magnitudes

Query

$$|| q ||= \sqrt{0.6^2 + 0.8^2}$$
$$= \sqrt{0.36 + 0.64} = 1$$

Document 1

$$|| d_1 ||= \sqrt{0.7^2 + 0.7^2}$$
$$= \sqrt{0.49 + 0.49} = \sqrt{0.98} = 0.99$$

Document 2

$$|| d_2 ||= \sqrt{0.2^2 + 0.1^2}$$
$$= \sqrt{0.04 + 0.01} = \sqrt{0.05} = 0.224$$

Step 3: Cosine Similarities

Document 1

$$cos(q, d_1) = 0.98/(1 \times 0.99)$$
$$cos(q, d_1) \approx 0.99$$

Document 2

$$cos(q, d_2) = 0.20/(1 \times 0.224)$$
$$cos(q, d_2) \approx 0.89$$

Document $\mathbf{d_1}$ is more semantically similar to the query.

Dense Retrieval Similarity Problem **Dense Retrieval Similarity**

Triplet Loss Problem

Value	Number
$\cos(q,d^+)$	0.82
$\cos(q,d^-)$	0.71
margin	0.2

Compute the **training loss**.

Dense Retrieval Similarity Problem **Dense Retrieval Similarity**

Triplet Loss Problem

Loss=max(0,0.71−0.82+0.2)

$$Loss = max(0, 0.09)$$

$$Loss = 0.09$$

Model must learn more.

SBERT (SENTENCE-BERT) ?

Sentence-BERT (SBERT) is a modification of the BERT architecture that uses siamese network structures to derive semantically meaningful sentence embeddings that can be compared using cosine similarity.

The Problem with BERT:

BERT was designed for pair classification:
Input: [CLS] sentence A [SEP] sentence B [SEP] → similarity score

Problem: To find most similar sentence among 10,000, you'd need:
10,000 forward passes through BERT → IMPOSSIBLY SLOW!

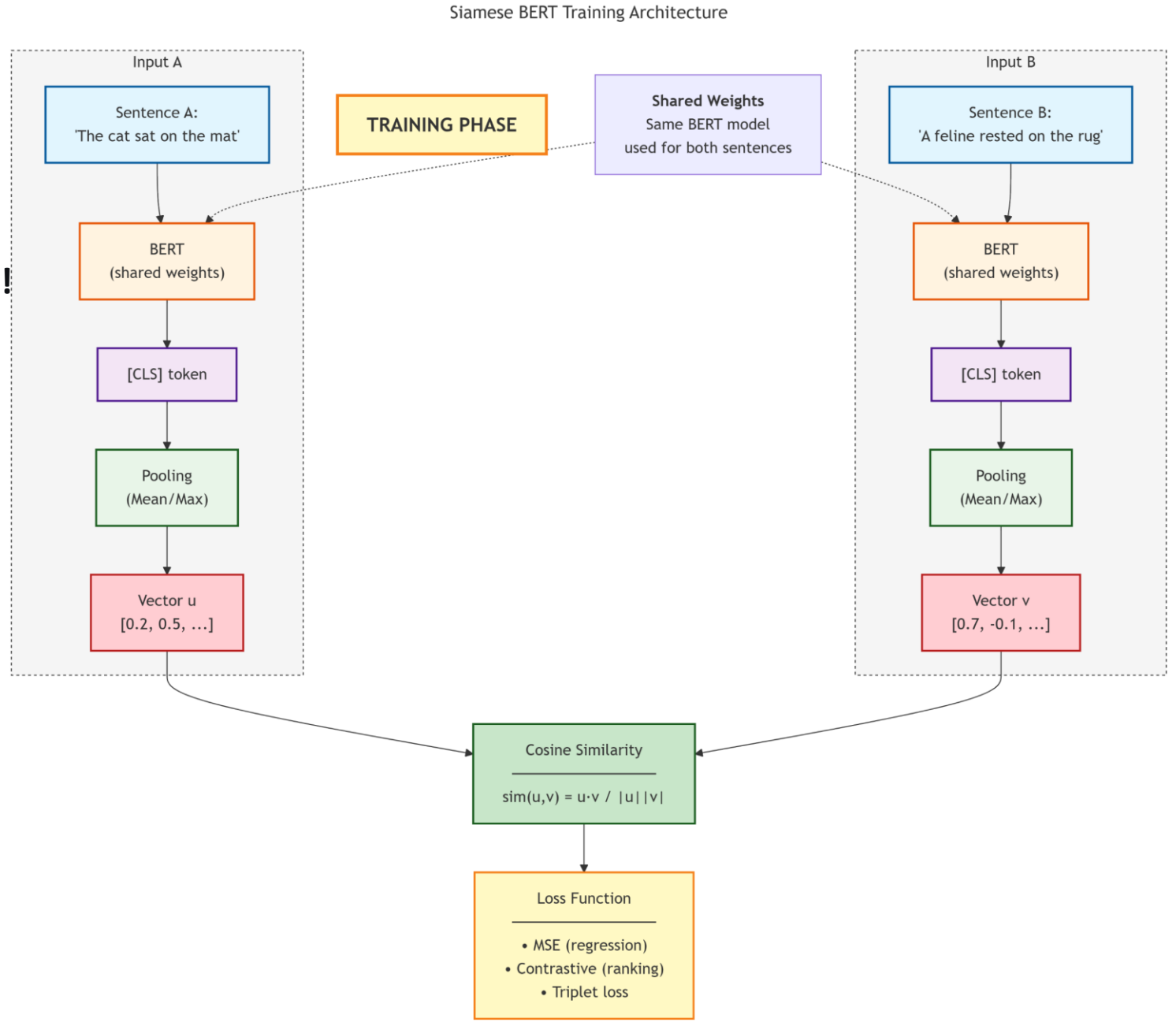
SBERT Solution: Pre-compute embeddings for all sentences once:
s1 → [0.2, -0.5, 0.7, ...]
s2 → [0.3, -0.4, 0.6, ...]
s3 → [-0.1, 0.8, -0.2, ...]
...
Then just compare vectors with cosine similarity!

GenerativeAI and Its Applications

SBERT (SENTENCE-BERT)

SBERT (SENTENCE-BERT) ?

Key Feature: Same BERT weights for both sentences!



SBERT Pooling Strategies?

How do we get a single vector from BERT's output?

BERT produces **one embedding per token**.

Example sentence:

`"The cat sat on the mat"`

BERT output:

`[token1_vec, token2_vec, token3_vec, ..., tokenN_vec]`

Each token vector has **768 dimensions**.

Need to be achieved: Convert these **N vectors → 1 sentence vector**

Pooling Method	Idea	Formula	Notes
CLS Token Pooling	Use the special `[CLS]` token embedding	Sentence = CLS vector	Simple but sometimes unstable
Mean Pooling	Average all token embeddings	Mean(token vectors)	Most common in SBERT
Max Pooling	Take maximum value across tokens	Max(token vectors)	Captures strongest signals

SBERT Pooling Strategies?

Mean Pooling Formula

SentenceEmbedding = (token1 + token2 + ... + tokenN) / N

This creates a **single vector representing the entire sentence**.

Example

Sentence:

`"Machine learning is powerful"`

Token embeddings (simplified 3-dimensional):

Token	Vector
Machine	[0.4, 0.6, 0.2]
learning	[0.5, 0.7, 0.3]
powerful	[0.6, 0.8, 0.4]

Mean Pooling

```
Sentence vector =([0.4,0.6,0.2] + [0.5,0.7,0.3] + [0.6,0.8,0.4]) / 3
= [1.5 , 2.1 , 0.9] / 3
= [0.5 , 0.7 , 0.3]
```

Final **sentence embedding**

`[0.5 , 0.7 , 0.3]`

Method	Performance	Usage
CLS Pooling	Good	Used in original BERT
Mean Pooling	Best	Default in SBERT
Max Pooling	Moderate	Used in some NLP tasks

SBERT Training Objectives?

SBERT learns sentence embeddings by training on **sentence pairs or triplets**.
There are **three major training objectives**.

1.) Classification Objective: Predict whether two sentences are similar or not

Example:
Sentence A:
"I love programming"
Sentence B:
"Coding is my passion"
Label = **1 (similar)**
Model Input Representation:
`[u ; v ; |u − v|]`
where
- **u** = embedding of sentence A
- **v** = embedding of sentence B
- **|u−v|** = absolute difference

Loss Function:

$$\text{Loss} = \text{CrossEntropy}(\text{softmax}(W \cdot [u ; v ; |u - v|]), \text{label})$$

Explanation :
The model **classifies sentence pairs into categories** such as:

Label	Meaning
0	Not similar
1	Similar

SBERT Training Objectives?

SBERT learns sentence embeddings by training on **sentence pairs or triplets**.
There are **three major training objectives**.

2. Regression Objective

Goal: **Predict a similarity score between two sentences**
Example:
Sentence A: "The weather is nice"
Sentence B: "It's sunny today"
Target Similarity Score:0.8

Model computes **cosine similarity between embeddings**.
Cosine Similarity:
$\cos(\theta) = (u \cdot v) / (||u|| \, ||v||)$
Loss Function:

```
Loss = MSE( cosine_sim(u,v), target )
```

Term	Meaning
MSE	Mean Squared Error
target	Ground truth similarity

Pair	Target
Sentence A vs B	0.8

The model learns to **predict similarity scores directly**.

SBERT Training Objectives?

3. Triplet Objective (Most Important for Retrieval)

Goal:

Learn embeddings so that

Anchor is closer to Positive than Negative

Example:

Anchor: "Machine learning is fascinating"

Positive: "AI and deep learning are interesting"

Negative: "I need to buy groceries"

Triplet Loss:

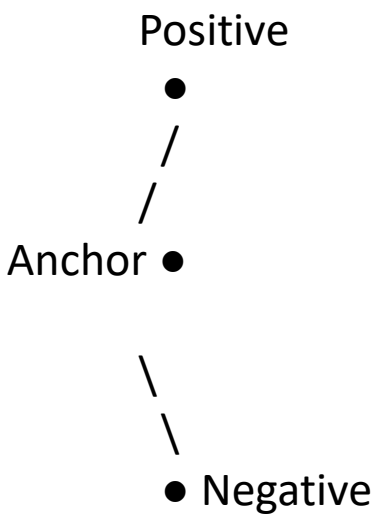
```
Loss = max(0, cos(anchor,negative) – cos(anchor,positive) + margin)
```

Margin usually:

`0.1 – 0.3`

Interpretation:

Relationship	Desired
cos(anchor,positive)	HIGH
cos(anchor,negative)	LOW



distance(anchor,positive) < distance(anchor,negative)

SBERT Training Objectives?

3. Triplet Objective (Most Important for Retrieval)

Goal:

Learn embeddings so that

Anchor is closer to Positive than Negative

Example:

Anchor: "Machine learning is fascinating"

Positive: "AI and deep learning are interesting"

Negative: "I need to buy groceries"

Triplet Loss:

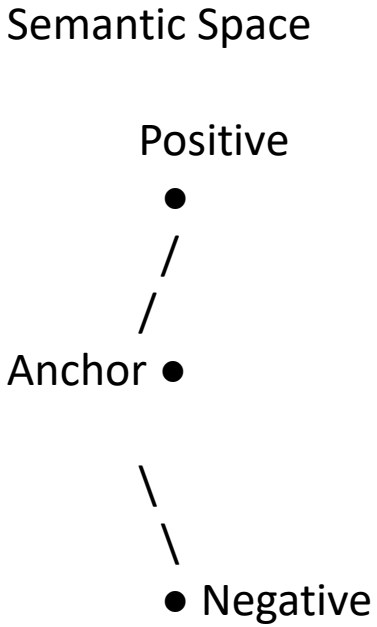
```
Loss = max(0, cos(anchor,negative) – cos(anchor,positive) + margin)
```

Margin usually:

`0.1 – 0.3`

Interpretation:

Relationship	Desired
cos(anchor,positive)	HIGH
cos(anchor,negative)	LOW



distance(anchor,positive) < distance(anchor,negative)

SBERT Training Problem!

Training Data

Anchor (a): "deep learning" → [0.9, 0.8]
Positive (p): "neural networks" → [0.8, 0.7]
Negative (n): "grocery shopping" → [0.1, 0.2]
margin = 0.3

Step 1: Cosine Similarity

Calculate Norms:

$$\|a\| = \sqrt{(0.9^2 + 0.8^2)} = \sqrt{(0.81 + 0.64)} = \sqrt{1.45} = 1.204$$

$$\|p\| = \sqrt{(0.8^2 + 0.7^2)} = \sqrt{(0.64 + 0.49)} = \sqrt{1.13} = 1.063$$

$$\|n\| = \sqrt{(0.1^2 + 0.2^2)} = \sqrt{(0.01 + 0.04)} = \sqrt{0.05} = 0.224$$

Calculate: cos(a,p)

$$\cos(a,p) = (0.9\times0.8 + 0.8\times0.7) / (1.204 \times 1.063) = (0.72 + 0.56) / 1.279 = 1.28 / 1.279 \approx 0.999 \approx 1.00$$

Calculate: cos(a,n)

$$\cos(a,n) = (0.9\times0.1 + 0.8\times0.2) / (1.204 \times 0.224) = (0.09 + 0.16) / 0.269 = 0.25 / 0.269 \approx 0.93$$

SBERT Training Problem!

Training Data

Anchor (a): "deep learning" → [0.9, 0.8]
Positive (p): "neural networks" → [0.8, 0.7]
Negative (n): "grocery shopping" → [0.1, 0.2]
margin = 0.3

Step 2: Triplet Loss

Triplet loss formula:
Loss = max(0, cos(a,n) - cos(a,p) + margin)
Substitute values:
Loss = max(0, 0.93 - 1.00 + 0.3)
= max(0, 0.23)
= 0.23
Since **loss > 0**, the model must update weights.

Step 3: Backpropagation Intuition

The optimizer updates SBERT so that:
cos(a,p) → increases
cos(a,n) → decreases
Graphically:
Before training
a ---- p (close)
a ---- n (too close)

After training
a ---- p (very close)
a ---------- n (far away)

Triplet loss trains SBERT so that semantically similar sentences have **higher cosine similarity** than unrelated sentences by at least a **margin**.

SBERT Inference - Finding Similar Sentences!

Problem: Find sentences similar to query in 1M document corpus

Step 1: Pre-compute Document Embeddings (One-time cost)

Document 1: "Quantum computing uses qubits" → [0.23, -0.45, 0.67, ...]

Document 2: "Machine learning algorithms" → [-0.12, 0.78, -0.34, ...]

Document 3: "Climate change effects" → [0.56, 0.23, -0.89, ...]

...

Document 1M: "..." → [...]

Time: ~1 second per 1000 docs = ~1000 seconds total

SBERT Inference - Finding Similar Sentences!

Step 2: Encode Query (At search time)

Query: "What is quantum computing?" → q = [0.25, -0.40, 0.70, ...]

Time: ~0.01 seconds

Step 3: Similarity Search

For each doc i:

score_i = cosine_similarity(q, doc_i)

Time with flat index: O(n × d) = 1M × 768 operations ≈ 0.1 seconds

Time with IVF/HNSW: O(log n) ≈ 0.001 seconds!

SBERT Numerical Example

We have **3 documents** and **1 query** represented as embeddings.
Doc1: "AI and machine learning" → $d_1 = [0.9, 0.7]$
Doc2: "Climate change effects" → $d_2 = [0.2, 0.9]$
Doc3: "Neural networks deep learning" → $d_3 = [0.8, 0.6]$

Query: "artificial intelligence" → $q = [0.85, 0.65]$

QUESTION: **Find the most similar document using cosine similarity.**

SBERT Numerical Example

Step 1: Compute Vector Norms

Query norm

$$|| q ||= \sqrt{0.85^2 + 0.65^2}$$
$$= \sqrt{0.7225 + 0.4225}$$
$$= \sqrt{1.145}$$
$$\approx 1.070$$

Document norms

Doc1

$$|| d_1 ||= \sqrt{0.9^2 + 0.7^2}$$
$$= \sqrt{0.81 + 0.49}$$
$$= \sqrt{1.30}$$
$$\approx 1.140$$

Doc2

$$|| d_2 ||= \sqrt{0.2^2 + 0.9^2}$$
$$= \sqrt{0.04 + 0.81}$$
$$= \sqrt{0.85}$$
$$\approx 0.922$$

Doc3

$$|| d_3 ||= \sqrt{0.8^2 + 0.6^2}$$
$$= \sqrt{0.64 + 0.36}$$
$$= \sqrt{1.00}$$
$$= 1.00$$

SBERT Numerical Example

Step 2: Cosine Similarities

Doc1 similarity

$$cos(q, d_1) = \frac{(0.85 \times 0.9 + 0.65 \times 0.7)}{(1.070 \times 1.140)}$$
$$= \frac{(0.765 + 0.455)}{1.220}$$
$$= \frac{1.22}{1.220}$$
$$\approx 0.999$$

Doc2 similarity

$$cos(q, d_2) = \frac{(0.85 \times 0.2 + 0.65 \times 0.9)}{(1.070 \times 0.922)}$$
$$= \frac{(0.17 + 0.585)}{0.986}$$
$$= \frac{0.755}{0.986}$$
$$\approx 0.77$$

Doc3 similarity

$$cos(q, d_3) = \frac{(0.85 \times 0.8 + 0.65 \times 0.6)}{(1.070 \times 1.00)}$$
$$= \frac{(0.68 + 0.39)}{1.070}$$
$$= \frac{1.07}{1.07}$$
$$\approx 1.00$$

Step 3: Ranking Results

Document	Cosine Similarity
Doc3: Neural networks deep learning	**1.00**
Doc1: AI and machine learning	**0.999**
Doc2: Climate change effects	**0.77**

Popular SBERT Models

higher dimension → better accuracy
lower dimension → faster

Model	Dimensions	Speed	Performance	Use Case
all-MiniLM-L6-v2	384	Very Fast	Good	General-purpose semantic search
all-mpnet-base-v2	768	Slower	Best	High-accuracy embeddings
multi-qa-mpnet-base-dot-v1	768	Slow	Excellent	Question-answer retrieval
paraphrase-MiniLM-L6-v2	384	Fast	Good	Paraphrase detection
msmarco-bert-base-dot-v5	768	Slow	Excellent	Search / document retrieval

Task: Semantic Textual Similarity (STS)

Model	Spearman Correlation	Speed (sentences/sec)
all-MiniLM-L6-v2	84.5	15,000
all-mpnet-base-v2	86.9	3,000
BERT (pairwise)	85.2	100

SBERT - Strengths and Limitations

Semantic understanding - catches synonyms and concepts
Fixed-size embeddings - efficient storage and comparison
Fast inference - O(1) per query after indexing
Pre-trained models - works out-of-the-box
Multilingual support - models for 50+ languages

- **Limitations:**
  Loss of token-level interaction - "cat sat on mat" vs "mat sat on cat" have same embedding
  Training data bias - inherits biases from training data
  Out-of-vocabulary - struggles with rare/domain-specific terms
  Context window - limited to 256-512 tokens
  Not interpretable - can't explain why documents matched

Analogy: SBERT is like having a **smart assistant who writes summaries**. The summaries capture the gist but lose **exact wording and word order nuances**.

COLBERT (Contextualized Late Interaction Over BERT)

COLBERT AND LATE INTERACTION

What is ColBERT?

Definition
ColBERT (Contextualized Late Interaction over BERT) is a neural retrieval model that combines the **efficiency of bi-encoders** with the **precision of cross-encoders**.

It achieves this by:

- Keeping **token-level embeddings**
- Using a **late interaction scoring mechanism**
- Computing similarity **after encoding**, instead of during encoding.

ColBERT is built on top of **BERT**.

COLBERT (Contextualized Late Interaction Over BERT)

COLBERT AND LATE INTERACTION

What is ColBERT? The Motivation

Cross-Encoder
High accuracy but extremely slow

```
[CLS] query [SEP] document [SEP] → relevance score
```

Problem:
- Must process **every query-document pair**
- Very expensive for large search systems.

Bi-Encoder
Fast but loses fine-grained interaction

```
query → single vector
document → single vector
cosine similarity → score
```

Problem:
- Compresses the entire sentence into **one vector**
- Loses **which words matched which words**.

COLBERT (Contextualized Late Interaction Over BERT)

COLBERT AND LATE INTERACTION

What is ColBERT?

ColBERT Approach (Late Interaction (Best of both)**)**
Instead of one vector, ColBERT keeps **token embeddings**.

$$\text{Query} \rightarrow [q_1, q_2, q_3, \ldots]$$
$$\text{Document} \rightarrow [d_1, d_2, d_3, \ldots]$$

- Similarity is computed between **all query tokens and document tokens**.
- Then ColBERT selects the **best match for each query token**.

Conceptually:

$$\text{Score(query, document)} = \Sigma \max \text{cosine}(q_i, d_j)$$

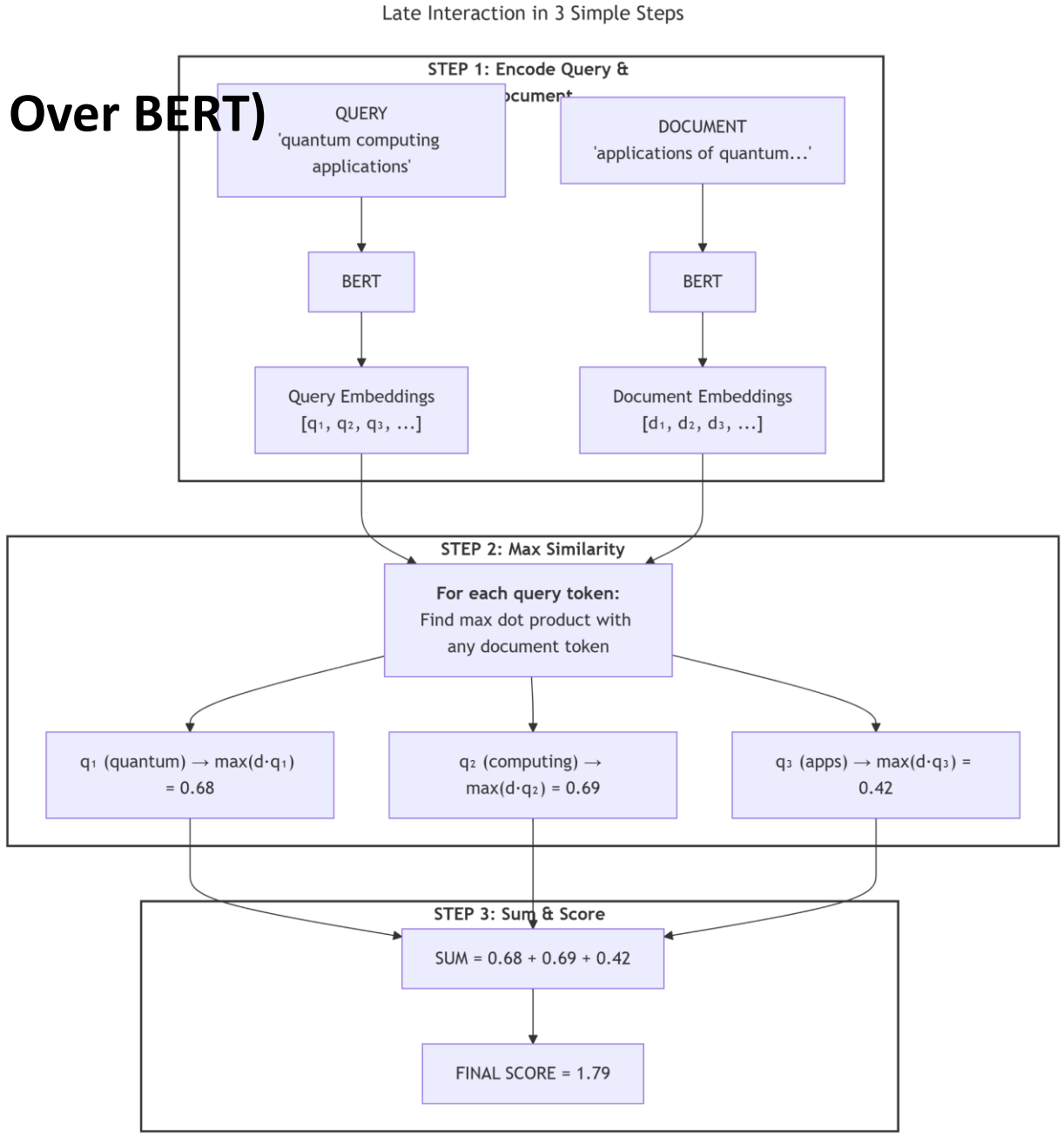
This preserves **fine-grained word matching** while still allowing **fast retrieval**.

Instead of single vector: Query Token and Doc Token , Each query token compares with all doc tokens.

COLBERT (Contextualized Late Interaction Over BERT)

COLBERT AND LATE INTERACTION ARCHITECTURE

What is ColBERT ARCHITECTURE?



COLBERT (Contextualized Late Interaction Over BERT)

ColBERT Scoring – The Math

In **ColBERT**, we compare **each query token with every document token**.

Let:

$Q = [q_1, q_2, \ldots, q_m]$ → query token embeddings

$D = [d_1, d_2, \ldots, d_n]$ → document token embeddings

For each query token:

$$\mathbf{max\_sim\_i = max\_j\ cosine(q\_i\ ,\ d\_j)}$$

Final document score:

$$\mathbf{Score(Q,D) = \Sigma\ max\_sim\_i}$$

This is called **MaxSim (maximum similarity)**.

COLBERT (Contextualized Late Interaction Over BERT)

ColBERT Scoring – Numerical Example

Query: **"quantum computing"**
Document: **"quantum physics applications"**
Token embeddings
q_quantum = [0.9, 0.3]
q_computing = [0.2, 0.8]

d_quantum = [0.85, 0.25]
d_physics = [0.1, 0.9]
d_applications= [0.3, 0.6]

COLBERT (Contextualized Late Interaction Over BERT)

ColBERT Scoring – Numerical Example

Step 1: Norms

$$||q\_quantum|| = \sqrt{(0.9^2 + 0.3^2)} = \sqrt{(0.81 + 0.09)} = \sqrt{0.90} = 0.949$$

$$||q\_computing|| = \sqrt{(0.2^2 + 0.8^2)} = \sqrt{(0.04 + 0.64)} = \sqrt{0.68} = 0.825$$

Document tokens:

$$||d\_quantum|| = \sqrt{(0.85^2 + 0.25^2)} = \sqrt{(0.7225 + 0.0625)} = \sqrt{0.785} = 0.886$$

$$||d\_physics|| = \sqrt{(0.1^2 + 0.9^2)} = \sqrt{0.82} = 0.905$$

$$||d\_applications|| = \sqrt{(0.3^2 + 0.6^2)} = \sqrt{0.45} = 0.671$$

COLBERT (Contextualized Late Interaction Over BERT)

ColBERT Scoring – Numerical Example

Step 2: MaxSim for q_quantum

Similarity with "quantum"

(0.9×0.85 + 0.3×0.25) / (0.949 × 0.886)

= (0.765 + 0.075) / 0.841
= 0.84 / 0.841
≈ 0.999

Similarity with "physics"

(0.9×0.1 + 0.3×0.9) / (0.949 × 0.905)

= (0.09 + 0.27) / 0.859
= 0.36 / 0.859
≈ 0.419

Similarity with "applications"

(0.9×0.3 + 0.3×0.6) / (0.949 × 0.671)

= (0.27 + 0.18) / 0.637
= 0.45 / 0.637
≈ 0.706

$max\_sim_1 = 0.999$

COLBERT (Contextualized Late Interaction Over BERT)

ColBERT Scoring – Numerical Example

Step 3: MaxSim for q_computing

With "quantum"

(0.2×0.85 + 0.8×0.25) / (0.825 × 0.886)

= (0.17 + 0.20) / 0.731
= 0.37 / 0.731
≈ 0.506

With "physics"

(0.2×0.1 + 0.8×0.9) / (0.825 × 0.905)
= (0.02 + 0.72) / 0.747
= 0.74 / 0.747
≈ 0.991

With "applications"

(0.2×0.3 + 0.8×0.6) / (0.825 × 0.671)
= (0.06 + 0.48) / 0.554
= 0.54 / 0.554
≈ 0.975

$max\_sim_2 = 0.991$

COLBERT (Contextualized Late Interaction Over BERT)

ColBERT Scoring – Numerical Example

Step 4: Final ColBERT Score

$$\text{Score}(Q,D) = \text{max\_sim}_1 + \text{max\_sim}_2$$
$$= 0.999 + 0.991$$
$$= 1.99$$

- ✓ *Unlike SBERT, which compares whole sentence embeddings,* ***ColBERT compares*** *individual tokens and keeps the best match for each query token.*
- ✓ *This preserves fine-grained word interactions while remaining much faster than cross-encoders.*

ColBERT vs SBERT vs Cross-Encoder

This comparison shows three major retrieval architectures built on **BERT**.

- **Cross-encoder** → highest accuracy but slow
- **SBERT** → fastest retrieval
- **ColBERT** → balance between speed and accuracy

Aspect	Cross-Encoder	SBERT	ColBERT
Scoring	Full attention over query + document	Pooled sentence vectors	Token-level MaxSim
Pre-computation	No	Yes (documents)	Yes (documents)
Query-time compute	O(n × BERT) forward passes	O(n × d) cosine similarity	O(m × n × d) token similarity
Accuracy	Very High	Medium	High
Speed	Very Slow	Very Fast	Moderate
Interpretability	High	Low	Medium
Storage per doc	None	~384–768 floats	~token_count × 128/768

COLBERT (Contextualized Late Interaction Over BERT)

ColBERT Problem

Problem

Rank documents for query:
"machine learning"

Query tokens:
["machine", "learning"]
Document 1
"deep learning algorithms"
→ ["deep", "learning", "algorithms"]
Document 2
"coffee machine repair"
→ ["coffee", "machine", "repair"]

Token Embeddings (2D simplified)

q_machine = [0.9, 0.1]
q_learning = [0.2, 0.8]

d1_deep = [0.8, 0.3]
d1_learning = [0.3, 0.7]
d1_algorithms = [0.4, 0.5]

d2_coffee = [0.1, 0.2]
d2_machine = [0.8, 0.2]
d2_repair = [0.1, 0.9]

COLBERT (Contextualized Late Interaction Over BERT)

PES UNIVERSITY

ColBERT PROBLEM

Step 1: Vector Norms

$$||q\_machine|| = \sqrt{(0.9^2 + 0.1^2)} = \sqrt{0.82} = 0.906$$
$$||q\_learning|| = \sqrt{(0.2^2 + 0.8^2)} = \sqrt{0.68} = 0.825$$

$$||d1\_deep|| = \sqrt{(0.8^2 + 0.3^2)} = \sqrt{0.73} = 0.854$$
$$||d1\_learning|| = \sqrt{(0.3^2 + 0.7^2)} = \sqrt{0.58} = 0.762$$
$$||d1\_algorithms|| = \sqrt{(0.4^2 + 0.5^2)} = \sqrt{0.41} = 0.640$$

$$||d2\_coffee|| = \sqrt{(0.1^2 + 0.2^2)} = \sqrt{0.05} = 0.224$$
$$||d2\_machine|| = \sqrt{(0.8^2 + 0.2^2)} = \sqrt{0.68} = 0.825$$
$$||d2\_repair|| = \sqrt{(0.1^2 + 0.9^2)} = \sqrt{0.82} = 0.906$$

COLBERT (Contextualized Late Interaction Over BERT)

ColBERT PROBLEM

Step 2: Score Document 1
For q_machine

sim(q_machine, d1_deep)

= (0.9×0.8 + 0.1×0.3) / (0.906 × 0.854)
= (0.72 + 0.03) / 0.774
= 0.75 / 0.774
≈ 0.969

sim(q_machine, d1_learning)

= (0.9×0.3 + 0.1×0.7) / (0.906 × 0.762)
= (0.27 + 0.07) / 0.690
= 0.34 / 0.690
≈ 0.493

sim(q_machine, d1_algorithms)

= (0.9×0.4 + 0.1×0.5) / (0.906 × 0.640)
= (0.36 + 0.05) / 0.580
= 0.41 / 0.580
≈ 0.707

$max\_sim_1 = 0.969$

COLBERT (Contextualized Late Interaction Over BERT)

ColBERT PROBLEM

Step 2: Score Document 1
For q_learning

sim(q_learning, d1_deep)

= (0.2×0.8 + 0.8×0.3) / (0.825 × 0.854)
= (0.16 + 0.24) / 0.704
= 0.40 / 0.704
≈ 0.568

sim(q_learning, d1_learning)
= (0.2×0.3 + 0.8×0.7) / (0.825 × 0.762)
= (0.06 + 0.56) / 0.628
= 0.62 / 0.628
≈ 0.987

sim(q_learning, d1_algorithms)
= (0.2×0.4 + 0.8×0.5) / (0.825 × 0.640)
= (0.08 + 0.40) / 0.528
= 0.48 / 0.528
≈ 0.909

$max\_sim_2 = 0.987$

Doc1 Score = 0.969 + 0.987 = 1.956

COLBERT (Contextualized Late Interaction Over BERT)

ColBERT PROBLEM

Step 3: Score Document 2

For q_machine

sim(q_machine, d2_coffee)

= (0.9×0.1 + 0.1×0.2) / (0.906 × 0.224)

= (0.09 + 0.02) / 0.203

= 0.11 / 0.203

≈ 0.542

sim(q_machine, d2_machine)

= (0.9×0.8 + 0.1×0.2) / (0.906 × 0.825)

= (0.72 + 0.02) / 0.748

= 0.74 / 0.748

≈ 0.989

sim(q_machine, d2_repair)

= (0.9×0.1 + 0.1×0.9) / (0.906 × 0.906)

= (0.09 + 0.09) / 0.821

= 0.18 / 0.821

≈ 0.219

$max\_sim_1 = 0.989$

COLBERT (Contextualized Late Interaction Over BERT)

ColBERT PROBLEM

Step 3: Score Document 2

For q_learning

sim(q_learning, d2_coffee)
= (0.2×0.1 + 0.8×0.2) / (0.825 × 0.224)
= (0.02 + 0.16) / 0.185
= 0.18 / 0.185
≈ 0.973

sim(q_learning, d2_machine)
= (0.2×0.8 + 0.8×0.2) / (0.825 × 0.825)
= (0.16 + 0.16) / 0.681
= 0.32 / 0.681
≈ 0.470

sim(q_learning, d2_repair)
= (0.2×0.1 + 0.8×0.9) / (0.825 × 0.906)
= (0.02 + 0.72) / 0.748
= 0.74 / 0.748
≈ 0.989

$max\_sim_2 = 0.989$

Doc2 Score = 0.989 + 0.989 = 1.978

COLBERT (Contextualized Late Interaction Over BERT)

ColBERT PROBLEM FINAL RANKING:

Document	Score
Document 2: "coffee machine repair"	1.978
Document 1: "deep learning algorithms"	1.956

This example reveals a known behavior of ColBERT:
The model may match query tokens with semantically similar but contextually different document tokens.

Example:
`learning ↔ repair`

This happens because MaxSim selects the best token match independently, without enforcing global context alignment.

ColBERT Strengths vs Weaknesses :

COLBERT (Contextualized Late Interaction Over BERT)

Strengths	Weaknesses
Preserves word-level interactions (knows which words matched)	Higher storage requirements (stores token embeddings)
Better than SBERT for fine-grained relevance	Slower than SBERT due to O(m × n) token comparisons
Faster than cross-encoders because document embeddings are pre-computed	May match unrelated words with similar embeddings
Provides interpretability by showing token-level matches	Limited positional understanding between tokens
Handles multi-word expressions effectively	Implementation complexity is higher than SBERT

Storage Comparison:

Model	Storage Formula	Storage for 1M Documents
SBERT	1M × 768 × 4 bytes	~3.07 GB
ColBERT (raw)	1M × 100 × 768 × 4 bytes	~307 GB
ColBERT (with compression)	Residual compression + binarization	~30–40 GB

ColBERTv2 - Improvements:

Technique	**Idea**	**Benefit**
Residual Compression	Store token embeddings as centroid ID + residual difference	Reduces storage drastically
Denoised Supervision	Use cross-encoder predictions as training labels (knowledge distillation)	Improves training quality
Centroid Interaction	Compare cluster centroids before full token comparison	Faster retrieval

ColBERTv2 - Improvements:

Residual Compression: This reduces storage while keeping most of the semantic information.

Step	Description
Original embedding	[0.234, -0.567, 0.891, ...]
Compression	Store centroid ID + small residual
Storage reduction	32-bit float → 2–4 bits per value

Denoised Supervision:

Step	**Description**
Teacher model	Cross-encoder generates relevance scores
Student model	ColBERT learns from these scores
Result	Cleaner training data and better ranking

ColBERTv2 - Improvements:

Centroid Interaction : This significantly reduces computation..

Step	Description
Token clustering	Document tokens grouped into clusters
Fast filtering	Compare query tokens with cluster centroids first
Final scoring	Full comparison only on top clusters

Performance Improvements

Metric	Original ColBERT	ColBERTv2
Storage	~307 GB	~16 GB
Speed	Baseline	~2× faster
Accuracy	Strong	Higher than ColBERT

HYBRID RETRIEVAL :

PES UNIVERSITY

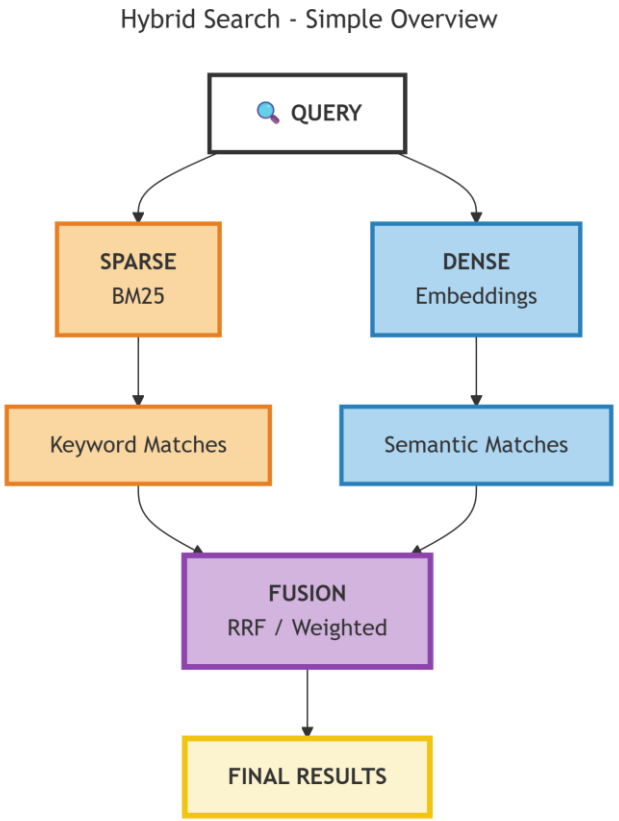
What is Hybrid Retrieval?

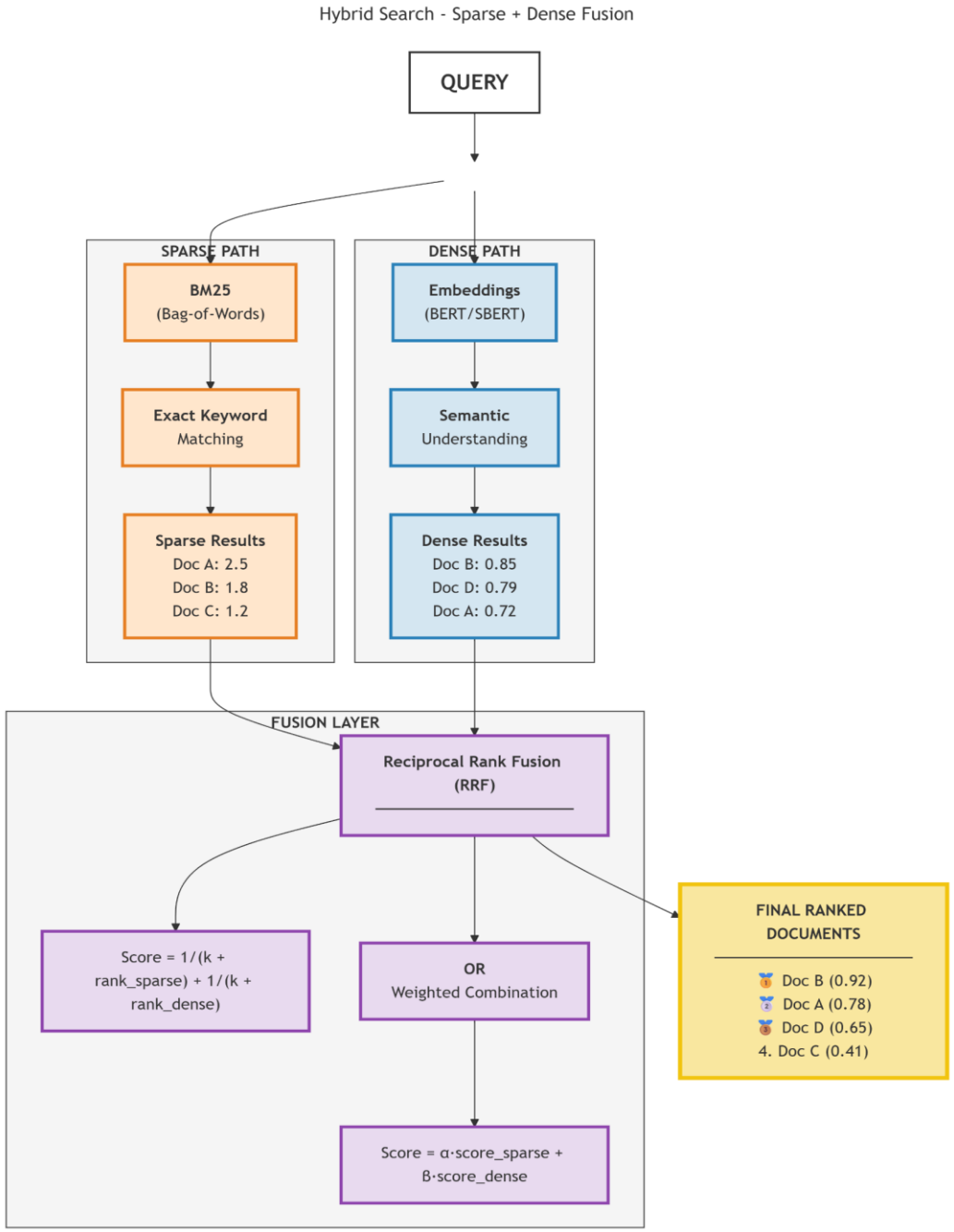
Hybrid Retrieval combines sparse retrieval (BM25) and dense retrieval (SBERT/ColBERT) to leverage the strengths of both approaches while mitigating their weaknesses.

Step	Sparse Path (BM25)	Dense Path (Embeddings)
Input	Query	Query
Retrieval	Exact keyword matching	Semantic understanding via embeddings
Output	Ranked documents	Ranked documents
Fusion	Combined using RRF (Reciprocal Rank Fusion) or weighted scores	Combined using RRF or weighted scores

Analogy: Hybrid retrieval is like having **two expert researchers**:
One finds **exact quotes** (BM25)
One understands **concepts and related ideas** (Dense)
Then you **combine their recommendations** for the best results.

HYBRID RETRIEVAL :





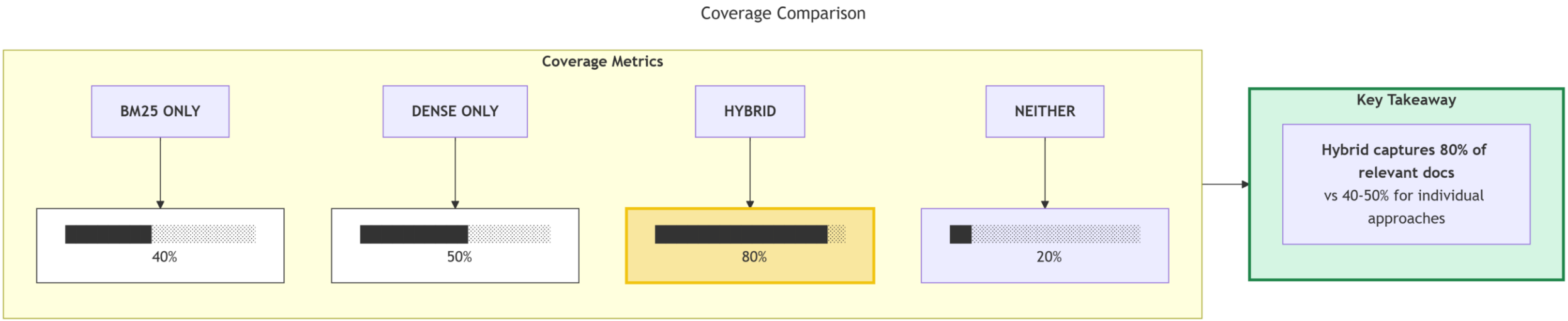
HYBRID RETRIEVAL :

When Each Method Excels

exact terms
rare words
product names

synonyms
concept search
semantic queries

Scenario	Example	BM25	Dense
Exact match needed	"iPhone 15 Pro Max price"	Perfect	May return related products
Synonyms	"automobile" vs "car"	Misses	Catches
Rare terms	"qubit superposition"	Good	May not understand
Ambiguous terms	"apple" (fruit vs company)	Confused	Context-aware
Long-tail queries	"1920s fashion trends"	Good	May hallucinate
Cross-lingual	"chat" (English/French)	Fails	Works



HYBRID RETRIEVAL : Fusion Methods – How to Combine Sparse & Dense Scores

Method 1: Reciprocal Rank Fusion (RRF)
Formula:

$$\text{score}(d) = \sum_{s} \frac{1}{k + rank_s(d)}$$

Where:
•$\text{rank}_s(d)$ =rank of document $d$in system s
•k= constant (typically 60)

Document	BM25 Rank	Dense Rank	RRF Calculation	RRF Score
A	3	5	1/(60+3) + 1/(60+5)	0.0159 + 0.0154 = 0.0313
B	10	2	1/(60+10) + 1/(60+2)	0.0143 + 0.0161 = 0.0304

Document A wins because it ranks well in both systems, even if Dense rank is slightly lower.

HYBRID RETRIEVAL :

Method 2: Weighted Sum

$$\mathbf{score}(\boldsymbol{d}) = \boldsymbol{\alpha} \times \boldsymbol{norm\_score\_bm25}(\boldsymbol{d}) + (\mathbf{1} - \boldsymbol{\alpha}) \times \boldsymbol{norm\_score\_dense}(\boldsymbol{d})$$

- αtypically 0.3–0.5
- Requires normalized scores from each system

Method 3: Score Normalization (pre-weighting)

- Min-Max: $\text{norm\_}score = \frac{score - min}{max - min}$
- Z-score: $\text{norm\_}score = \frac{score - mean}{std\_dev}$

HYBRID RETRIEVAL : RRF Numerical Example (k=60)

Doc	BM25 Rank	Dense Rank	RRF Calculation	RRF Score
D1	1	3	1/(60+1) + 1/(60+3)	0.0164 + 0.0159 = 0.0323
D2	2	5	1/(60+2) + 1/(60+5)	0.0161 + 0.0154 = 0.0315
D3	4	1	1/(60+4) + 1/(60+1)	0.0156 + 0.0164 = 0.0320
D4	3	8	1/(60+3) + 1/(60+8)	0.0159 + 0.0147 = 0.0306
D5	10	2	1/(60+10) + 1/(60+2)	0.0143 + 0.0161 = 0.0304

Ranking by RRF:

D1 (0.0323) → D3 (0.0320) → D2 (0.0315) → D4 (0.0306) → D5 (0.0304)

Key Insight: Documents performing well in both sparse and dense systems get the **highest RRF scores**.

HYBRID RETRIEVAL :

Problem

Find documents relevant to **“machine learning algorithms”** using **BM25 + Dense (SBERT)**.

HYBRID RETRIEVAL :

Step 1: BM25 Scores (Sparse)

Document	BM25 Score
Doc 1: "machine learning algorithms for beginners"	12.5
Doc 2: "coffee machine cleaning instructions"	3.2
Doc 3: "deep learning neural networks tutorial"	8.7
Doc 4: "algorithm design and analysis"	6.1
Doc 5: "machine learning in healthcare"	10.3

Step 2: Dense Scores (SBERT)

Query embedding: [0.7, 0.5, 0.3, ...]

Document 1 embedding: [0.8, 0.4, 0.2, ...] → cosine = 0.95
Document 2 embedding: [0.1, 0.2, 0.9, ...] → cosine = 0.45
Document 3 embedding: [0.6, 0.7, 0.3, ...] → cosine = 0.92
Document 4 embedding: [0.3, 0.2, 0.8, ...] → cosine = 0.56
Document 5 embedding: [0.7, 0.6, 0.2, ...] → cosine = 0.98

Document	Cosine Similarity
Doc 1	0.95
Doc 2	0.45
Doc 3	0.92
Doc 4	0.56
Doc 5	0.98

HYBRID RETRIEVAL :

Step 3: Normalize Scores (Min-Max)

$$\text{norm\_}score = \frac{score - min}{max - min}$$

BM25 min = 3.2, max = 12.5
Dense min = 0.45, max = 0.98

Document 1: BM25_norm = (12.5-3.2)/(12.5-3.2) = 1.00
Dense_norm = (0.95-0.45)/(0.98-0.45) = 0.50/0.53 = 0.94
Document 2: BM25_norm = (3.2-3.2)/(12.5-3.2) = 0.00
Dense_norm = (0.45-0.45)/(0.98-0.45) = 0.00
Document 3: BM25_norm = (8.7-3.2)/(12.5-3.2) = 5.5/9.3 = 0.59
Dense_norm = (0.92-0.45)/(0.98-0.45) = 0.47/0.53 = 0.89
Document 4: BM25_norm = (6.1-3.2)/(12.5-3.2) = 2.9/9.3 = 0.31
Dense_norm = (0.56-0.45)/(0.98-0.45) = 0.11/0.53 = 0.21
Document 5: BM25_norm = (10.3-3.2)/(12.5-3.2) = 7.1/9.3 = 0.76
Dense_norm = (0.98-0.45)/(0.98-0.45) = 1.00

HYBRID RETRIEVAL :

Step 4: Weighted Combination (α = 0.4 for BM25, 0.6 for Dense)

$$\text{Hybrid } Score = 0.4 \times BM25\_norm + 0.6 \times Dense\_norm$$

Document 1: 0.4 × 1.00 + 0.6 × 0.94 = 0.40 + 0.564 = 0.964
Document 2: 0.4 × 0.00 + 0.6 × 0.00 = 0.000
Document 3: 0.4 × 0.59 + 0.6 × 0.89 = 0.236 + 0.534 = 0.770
Document 4: 0.4 × 0.31 + 0.6 × 0.21 = 0.124 + 0.126 = 0.250
Document 5: 0.4 × 0.76 + 0.6 × 1.00 = 0.304 + 0.600 = 0.904

Document	Hybrid Score
Doc 1	0.964
Doc 2	0.000
Doc 3	0.770
Doc 4	0.250
Doc 5	0.904

HYBRID RETRIEVAL :

Step 5: Final Ranking

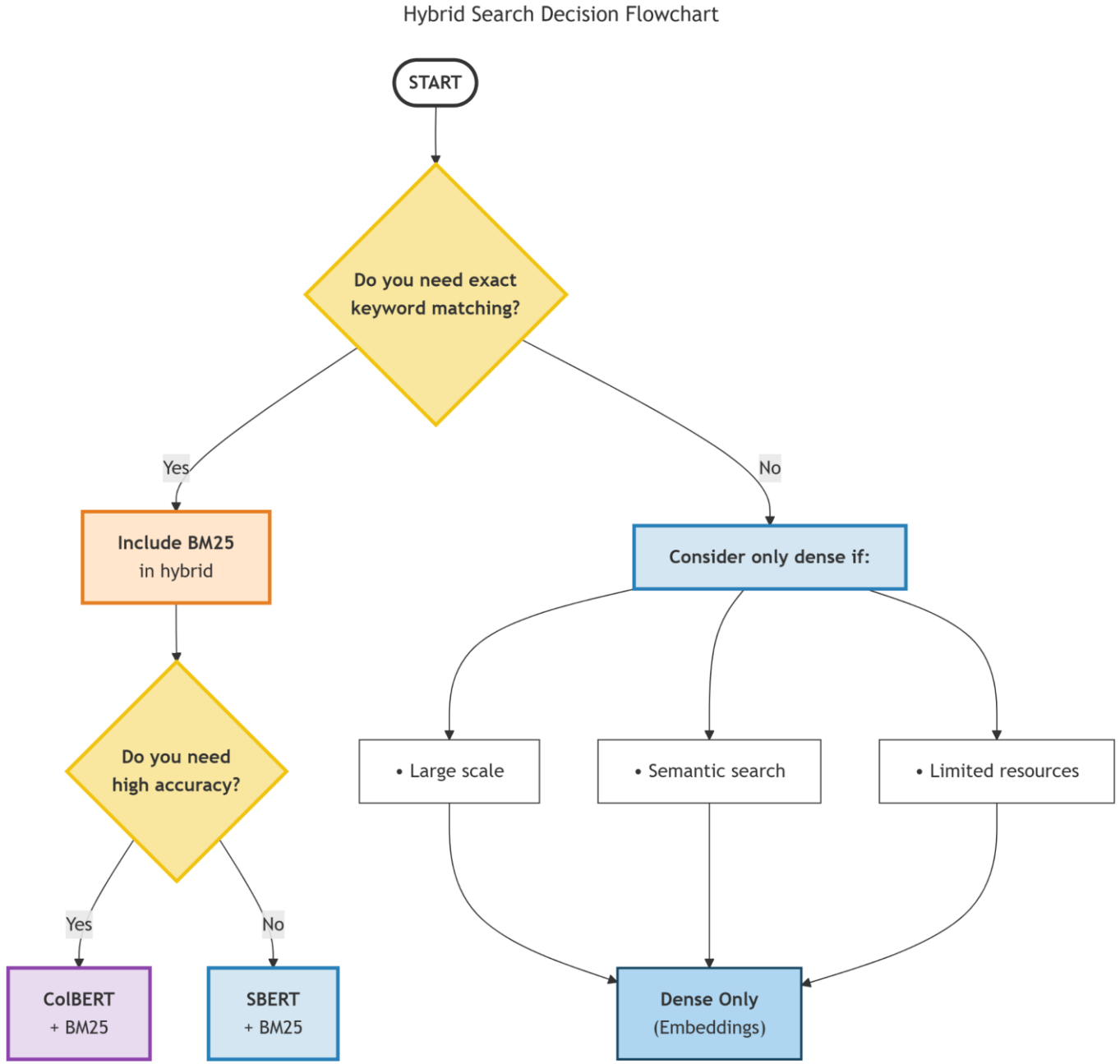
Rank	Document	Hybrid Score	BM25 Only Rank	Dense Only Rank
1	Doc 1	0.964	1	3
2	Doc 5	0.904	2	1
3	Doc 3	0.770	3	2
4	Doc 4	0.250	4	4
5	Doc 2	0.000	5	5

Key Observations:

- Doc 1 wins because it's strong in both
- Doc 5 (strong dense, good BM25) comes second
- Doc 3 (strong dense, moderate BM25) comes third
- Doc 4 (weak in both) is appropriately low

GenerativeAI and Its Applications

WHEN WHICH APPROACH?



WHEN WHICH APPROACH?

Problem 1: SBERT Similarity Calculation

Given these 2D embeddings (simplified):

- ✓ Query: "artificial intelligence" → [0.6, 0.8]
- ✓ Doc1: "machine learning" → [0.7, 0.6]
- ✓ Doc2: "climate change" → [0.2, 0.3]
- ✓ Doc3: "deep learning" → [0.8, 0.5]

Calculate cosine similarity for each document and rank them.

GenerativeAI and Its Applications

WHEN WHICH APPROACH?

Problem 2: ColBERT Scoring

Query tokens: ["neural", "networks"]
Document tokens: ["deep", "learning", "neural", "networks"]

Given embeddings:

- q_neural = [0.9, 0.2]
- q_networks = [0.3, 0.8]
- d_deep = [0.8, 0.3]
- d_learning = [0.2, 0.7]
- d_neural = [0.85, 0.25]
- d_networks = [0.35, 0.75]

Calculate ColBERT score.

Problem 3: Hybrid Fusion

BM25 ranks: Doc1(2), Doc2(4), Doc3(1), Doc4(3)
Dense ranks: Doc1(3), Doc2(1), Doc3(4), Doc4(2)

Calculate RRF scores with k=60 and determine final ranking.

THANK YOU

Generative AI and Its Applications

Unit 3

(UE23CS342BA4)

QUERY EXPANSION & RE-RANKING

Dr. Pooja Agarwal

Dept. of CSE, PES University, Bangalore

Acknowledgement

The slides are prepared from various resources from the Universities from abroad and India. Also, some material is taken from reliable resources from internet throughout this course.

The Query Problem

The Query Problem!

Original Query: "best laptop for programming"
Problems with this query:
1. Ambiguity: "best" means what? (price? performance?)
2. Missing context: What kind of programming?
3. Vocabulary gap: "laptop" vs "notebook" vs "macbook"
4. Underspecified: Budget? Screen size? Battery life?

Examples of what user might think:
"laptop with good processor for software development"
"long battery life laptop for coding on the go"
"affordable laptop for programming students"
"high-performance notebook for machine learning“

Analogy:
Query expansion is like a detective asking follow-up questions before searching for evidence. The initial clue might be vague, so you need to think of all possible interpretations.

The Query Problem

What is Query Expansion?!

Query Expansion is the process of reformulating and enriching the original user query with additional relevant terms to improve retrieval recall and precision.

Original Query: Q = "machine learning"

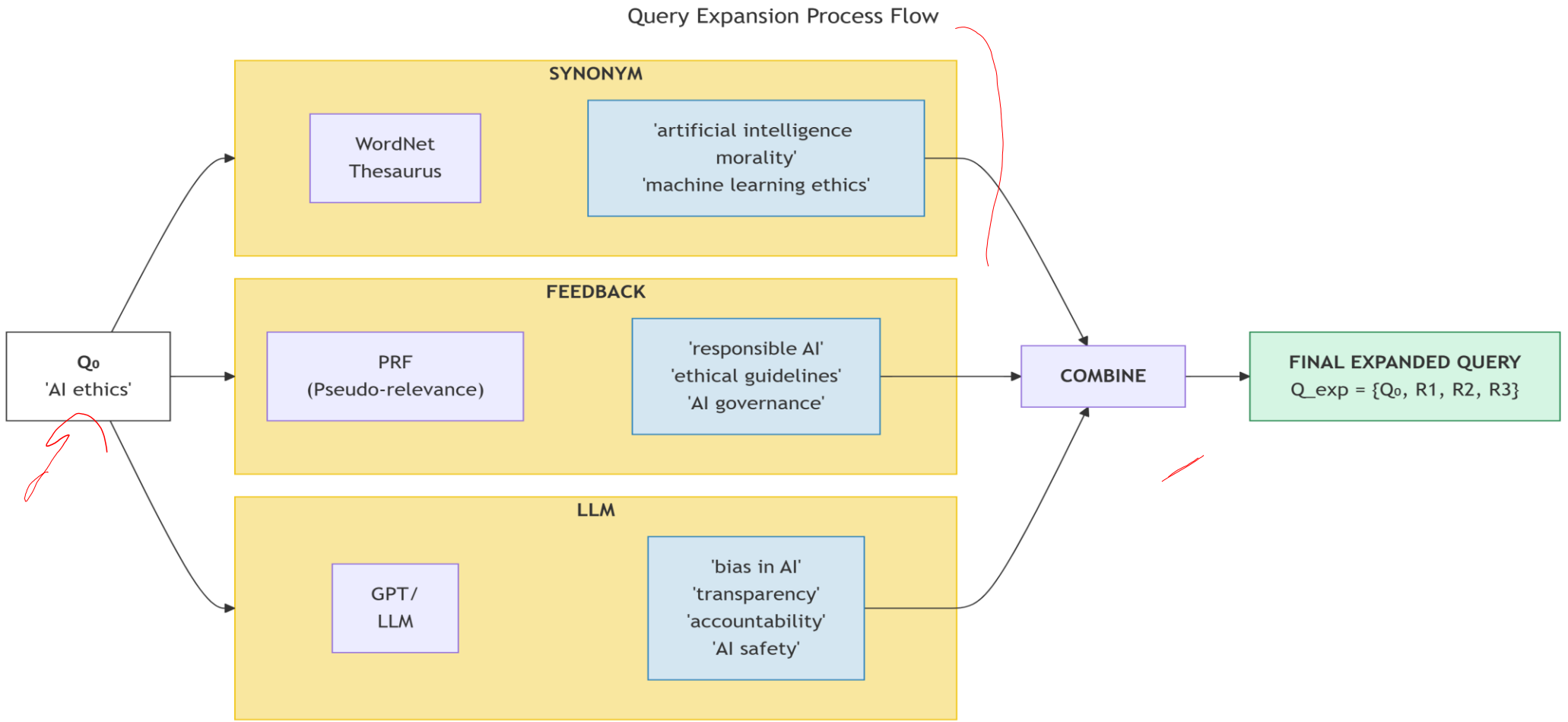
Expanded Query: Q' = "machine learning" OR "ML" OR

"artificial intelligence" OR "deep learning" OR

"neural networks" OR "supervised learning"

The Query Problem

What Query Expansion is and Visual Representation.



The Query Problem

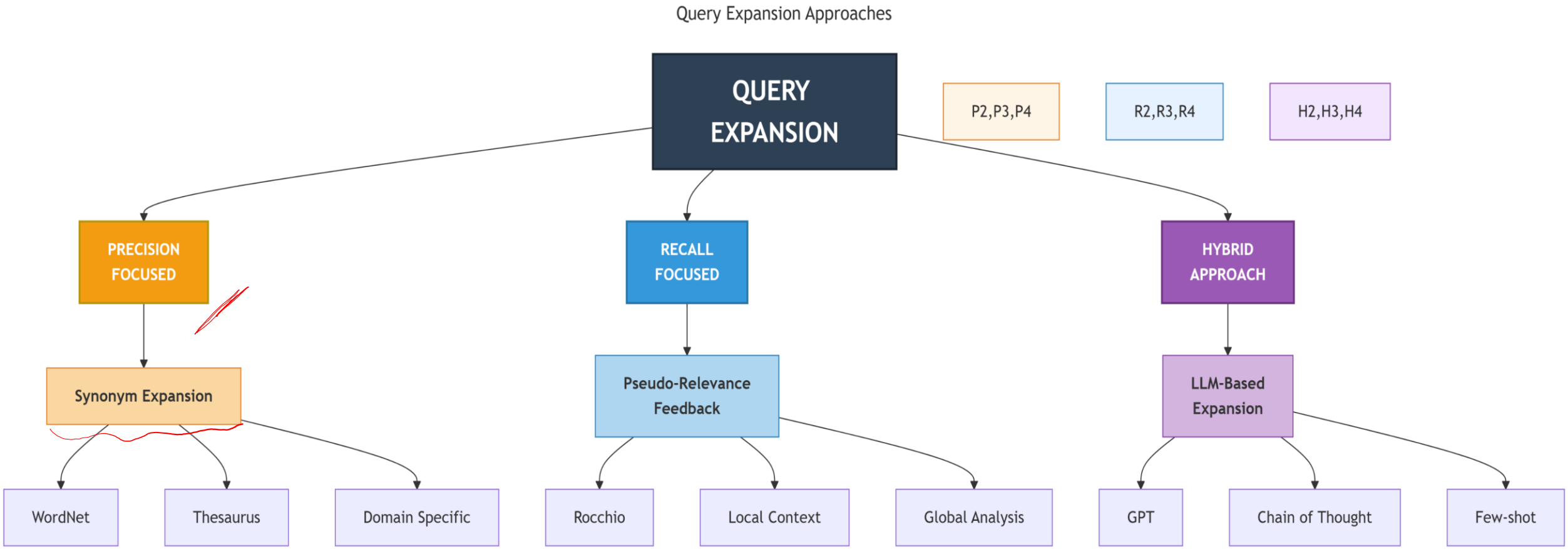
What Query Expansion Approaches.

Three Main Categories:

1.Precision-focused: Add synonyms to catch variations

2.Recall-focused: Learn from initial results to add terms

3.Hybrid: Use LLMs to understand intent and expand intelligently



The Query Problem

Technique 1 - Synonym-Based Expansion.

Original Query: "automobile repair manual"

Synonym Expansion:

Original Term	Expanded Synonyms
automobile	car, vehicle, motor vehicle
repair	fix, maintenance, service
manual	guide, handbook, instructions

Expanded Query:
("automobile" OR "car" OR "vehicle" OR "motor vehicle") AND
("repair" OR "fix" OR "maintenance" OR "service") AND
("manual" OR "guide" OR "handbook" OR "instructions")

Sources for Synonyms:

- **WordNet:** Lexical database of English
- **Domain-specific thesauri:** Medical, legal, technical
- **Embedding-based:** Find nearest neighbors in vector space

Technique 1 - Synonym-Based Expansion example.

Word: "car“

Synonyms: auto, automobile, machine, motorcar

Hyponyms: sedan, SUV, convertible, hatchback

Hypernyms: vehicle, motor vehicle, automotive

The Query Problem

Technique 1 - Synonym Expansion - Numerical Example.

Problem: Expand query and calculate new BM25 scores

Original Query: "car engine problem"

Doc1: "automobile engine troubleshooting guide"
Doc2: "vehicle maintenance tips"
Doc3: "motor repair handbook"
Doc4: "car engine oil change procedure"

The Query Problem

Technique 1 - Synonym Expansion - Numerical Example.

Step1: Synonym Expansion

Query Term	Expanded Terms
car	car, automobile, vehicle, motor
engine	engine, motor
problem	problem, issue, trouble, troubleshooting

Expanded query:

```
[car, automobile, vehicle, motor, engine, problem, issue, trouble, troubleshooting]
```

Technique 1 - Synonym Expansion - Numerical Example.

Step2: BM25 Scores **Before Expansion**

Only exact matches count.

Document	Matched Terms	Score
Doc4	car, engine	**8.5**
Doc1	engine	**5.2**
Doc2	—	**0**
Doc3	—	**0**

Observations:

- Only **2 of 4 documents retrieved**
- Many relevant documents use **different vocabulary**

Technique 1 - Synonym Expansion - Numerical Example.

Step3: BM25 Scores After Expansion

Now the query includes synonyms.

Document	Matched Terms	Score	Change
Doc1	automobile, engine, troubleshooting	**12.3**	+135%
Doc2	vehicle	**7.8**	New match
Doc3	motor	**6.5**	New match
Doc4	car, engine	**8.5**	Same

Why scores increase:

- More **term matches**
- BM25 increases weight when query terms appear in a document

The Query Problem

Technique 1 - Synonym Expansion - Numerical Example.

Step 4: Recall Improvement
Recall = relevant docs retrieved / total relevant docs
Before expansion:
`2 / 4 = 50%`
After expansion:
`4 / 4 = 100%`
So synonym expansion significantly improves **recall**.

Relevant documents in corpus: Doc1, Doc2, Doc3, Doc4 (4 documents)

Recall before: 2/4 = 50% (only Doc4 retrieved)
Recall after: 4/4 = 100% (all relevant retrieved)

1. Synonym expansion improves **recall** but may sometimes reduce **precision** because:
2. Some synonyms might introduce **noise**
3. For example, "motor" could refer to **electric motors**, not car engines
4. Therefore many modern search engines (e.g., **Elasticsearch**) apply:

- weighted synonyms
- contextual expansion
- query-time boosting

Technique 2 - Pseudo-Relevance Feedback (PRF).

The Rocchio Algorithm Approach

1.Retrieve top-k documents with original query

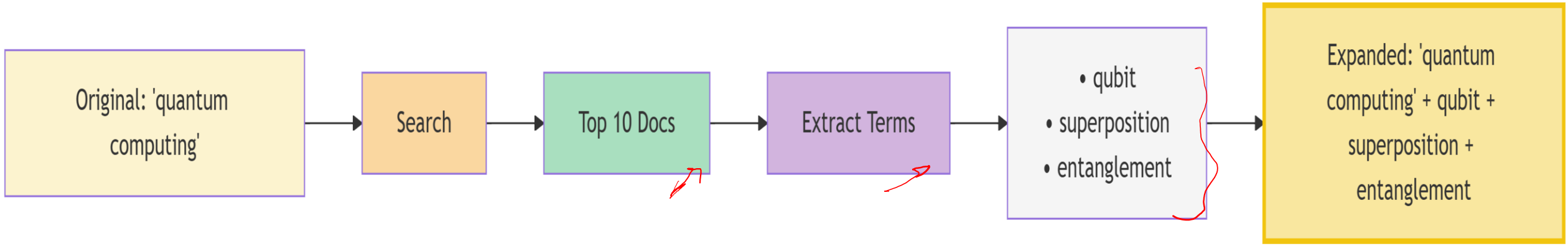
2.Assume these documents are relevant (pseudo-relevance)

3.Extract important terms from these documents

4.Add these terms to the original query

The Query Problem

Technique 2 - Pseudo-Relevance Feedback (PRF) Flow.



Technique 2 - Pseudo-Relevance Feedback (PRF) Flow.

The **Rocchio Algorithm** is a classic method in **Information Retrieval** used for **relevance feedback** and **query expansion**. It adjusts the original query vector by moving it **closer to relevant documents** and **away from non-relevant ones**.

$$Q_{new} = \alpha \cdot Q_{original} + \beta \cdot (1/|R|) \cdot \Sigma(d \in R) - \gamma \cdot (1/|NR|) \cdot \Sigma(d \in NR)$$

Symbol	Meaning
Q_original	Original query vector
Q_new	Updated query vector
R	Set of relevant documents
NR	Set of non-relevant documents
**	R
**	NR
α (alpha)	Weight of the original query
β (beta)	Weight for relevant documents
γ (gamma)	Penalty weight for non-relevant documents

Technique 2 - Pseudo-Relevance Feedback (PRF) Numerical.

Step 1: Initial Retrieval with Query "machine learning"

Top 5 Documents:

Doc1: "machine learning algorithms for beginners"

Doc2: "deep learning neural networks tutorial"

Doc3: "supervised learning techniques"

Doc4: "python machine learning library scikit-learn"

Doc5: "artificial intelligence applications"

Technique 2 - Pseudo-Relevance Feedback (PRF) Numerical.

Step 2: Extract Important Terms (TF-IDF based)

Document	Top Terms (excluding stopwords)
Doc1	algorithms, beginners
Doc2	deep, neural, networks
Doc3	supervised, techniques
Doc4	python, library, scikit-learn
Doc5	artificial, intelligence, applications

Technique 2 - Pseudo-Relevance Feedback (PRF) Numerical.

Step 3: Term Frequency in Pseudo-Relevant Set)

Term	Frequency	Select?
algorithms	1	Yes
beginners	1	Yes
deep	1	Yes
neural	1	Yes
networks	1	Yes
supervised	1	Yes
python	1	Yes
scikit-learn	1	Yes
artificial	1	Yes
intelligence	1	Yes

Technique 2 - Pseudo-Relevance Feedback (PRF) Numerical.

Step 4: Expanded Query)

Q' = "machine learning" OR algorithms OR beginners OR deep OR neural OR networks OR supervised OR python OR scikit-learn OR artificial OR intelligence

Term	Frequency	Select?
algorithms	1	Yes
beginners	1	Yes
deep	1	Yes
neural	1	Yes
networks	1	Yes
supervised	1	Yes
python	1	Yes
scikit-learn	1	Yes
artificial	1	Yes
intelligence	1	Yes

Technique 2 - Numerical PRF - Weight Adjustment Example.

Rocchio Algorithm in Action

Initial State: **1. Initial Query Vector**

```
q = [0.8, 0.6]
```

This represents the **original query position** in a 2-dimensional vector space.

2. Relevant Document Centroid

Relevant documents:

```
d1 = [0.9, 0.7]
d2 = [0.7, 0.8]
d3 = [0.8, 0.9]
```

Centroid:

```
R_centroid =
[(0.9 + 0.7 + 0.8)/3 , (0.7 + 0.8 + 0.9)/3]
= [2.4/3 , 2.4/3]
= [0.8 , 0.8]
```

This represents the **average position of relevant documents**.

Technique 2 - Numerical PRF - Weight Adjustment Example.

Rocchio Algorithm in Action

Initial State: **3. Non-Relevant Document Centroid**

Non-relevant documents:

d4 = [0.2, 0.1]

d5 = [0.3, 0.2]

Centroid:

NR_centroid =[(0.2 + 0.3)/2 , (0.1 + 0.2)/2] = [0.25 , 0.15]

This represents the **average location of irrelevant documents**.

.

Technique 2 - Numerical PRF - Weight Adjustment Example.

4. Apply the Rocchio Formula

Parameters:

$\alpha = 1.0$

$\beta = 0.8$

$\gamma = 0.2$

Formula:

$q\_new = \alpha q + \beta R\_centroid - \gamma NR\_centroid$

Substitute values:

$q\_new = 1.0[0.8,0.6] + 0.8[0.8,0.8] - 0.2[0.25,0.15]$

Calculate each component:

Original query weight

$[0.8, 0.6]$

Relevant influence

$0.8 \times [0.8,0.8] = [0.64,0.64]$

Non-relevant penalty

$0.2 \times [0.25,0.15] = [0.05,0.03]$

Final vector

$q\_new = [0.8,0.6] + [0.64,0.64] - [0.05,0.03] = [1.39, 1.21]$

Technique 2 - Numerical PRF - Weight Adjustment Example.

5. Normalization

Magnitude:

$||q\_new|| = \sqrt{(1.39^2 + 1.21^2)}$

$= \sqrt{(1.93 + 1.46)}$

$= \sqrt{3.39}$

≈ 1.84

Normalized vector:

$q\_norm = [1.39/1.84 , 1.21/1.84] \approx [0.76 , 0.66]$

6. Interpretation The new query vector:

`[0.76 , 0.66]`

has moved:

toward the relevant centroid `[0.8,0.8]`

away from the non-relevant centroid `[0.25,0.15]`

This means the refined query will retrieve documents **more similar to relevant ones**.

7. Geometric Intuition

In vector space:

- Original query → somewhere between document clusters
- Relevant docs → upper region
- Non-relevant docs → lower region
- Rocchio **pulls the query toward the relevant cluster** and **pushes it away from irrelevant ones**.

The Query Problem

Technique 3 - LLM-Based Query Expansion

Using Large Language Models for Intelligent Expansion

You are a query expansion expert. Given a user query, generate 5 alternative phrasings or related terms that would help retrieve relevant documents.

Original Query: {user_query}

Generate expanded queries in these categories:

- 1. Synonym replacement
- 2. More specific version
- 3. More general version
- 4. Related concepts
- 5. Common misspellings/variations

Expanded Queries:

The Query Problem

Technique 3 - LLM-Based Query Expansion example:

Original: "AI ethics guidelines"

LLM Output:
1. Synonym: "artificial intelligence moral principles"
2. Specific: "EU AI Act compliance requirements"
3. General: "technology ethical frameworks"
4. Related: "responsible AI development standards"
5. Variations: "AI ethic guidelines", "AI ethical guide lines"

Final Expanded Query Set:
"AI ethics guidelines" OR "artificial intelligence moral principles" OR
"EU AI Act compliance requirements" OR "technology ethical frameworks" OR
"responsible AI development standards" OR "AI ethic guidelines"

The Query Problem

Technique 3 - LLM Query Expansion - Chain of Thought

Step-by-step reasoning for better expansion

Query: "best programming language for data science"

Chain of Thought:
1. What is the user's intent?
   → Wants to learn data science, needs language recommendation

2. What are the actual data science languages?
   → Python, R, Julia, SQL

3. What factors matter for "best"?
   → Libraries (pandas, numpy, scikit-learn), community, learning curve

4. What related concepts should we include?
   → data analysis, machine learning, statistical computing, visualization

5. Generate expansion:
   Python data science
   R programming for statistics
   Julia for scientific computing
   data analysis programming languages
   machine learning libraries
   statistical computing tools
   data visualization programming

Resulting Query: ("programming language" OR "coding language") AND ("data science" OR "data analysis" OR "machine learning" OR "statistics" OR "scientific computing") AND ("Python" OR "R" OR "Julia" OR "SQL")

Query Expansion – Comparative Analysis

Effectiveness Comparison

Technique	Precision	Recall	Computation Cost	Best For
Synonym Expansion	Medium	Medium	Very Low	General domains with known vocabulary
Pseudo Relevance Feedback (PRF)	Medium	High	Medium	Domain-specific queries with good initial results
LLM-based Expansion	High	High	High	Complex queries requiring semantic understanding

The Query Problem

Techniques Example:

Test Query
Query: *"climate change effects"*

Synonym Expansion
Recall@10: **0.65**
Precision@10: **0.70**
Added Terms:
•global warming impacts

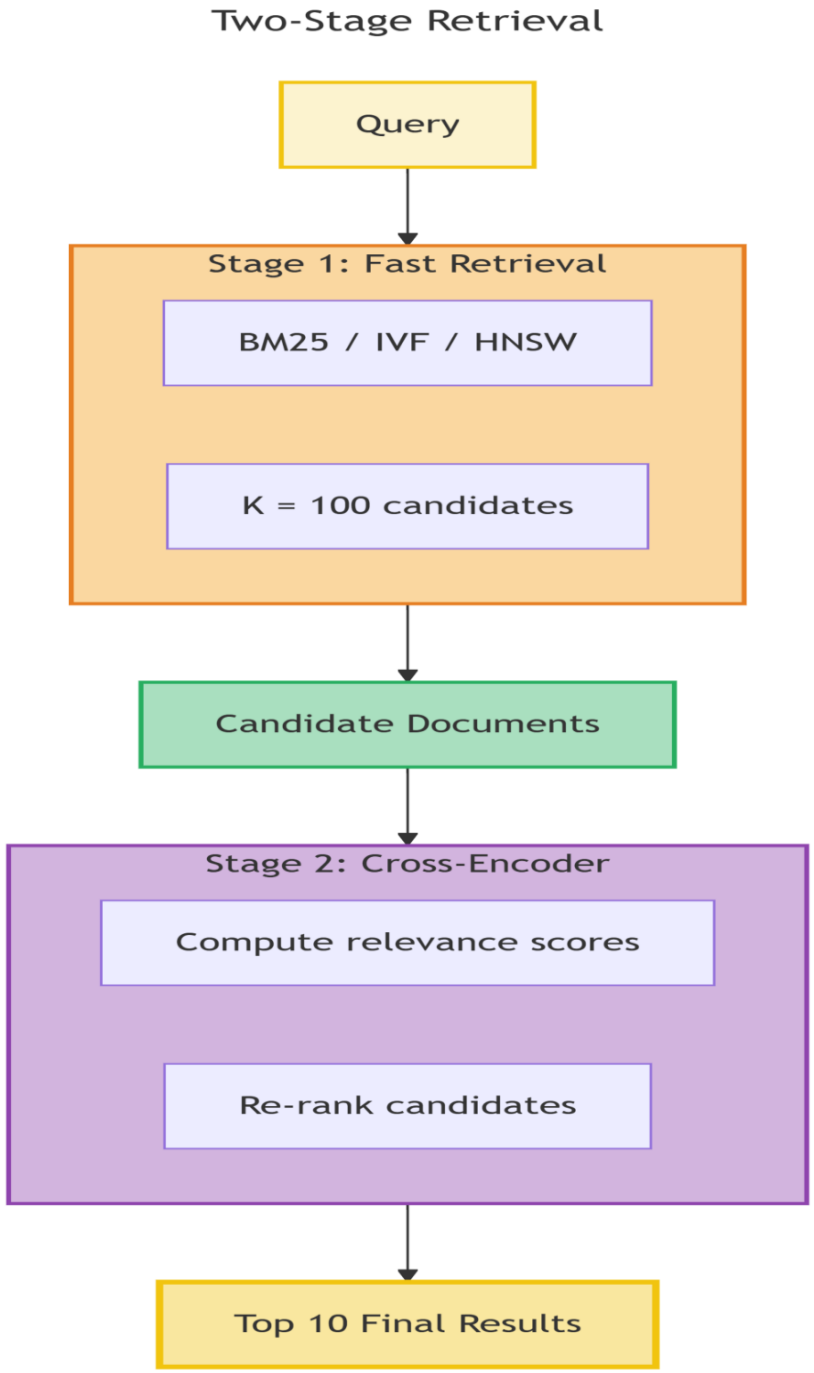
Pseudo Relevance Feedback (PRF)
Recall@10: **0.82**
Precision@10: **0.68**
Added Terms:
•sea level rise
•extreme weather

LLM-based Expansion
Recall@10: **0.88**
Precision@10: **0.75**
Added Terms:
•carbon emissions
•greenhouse gas
•Paris Agreement
•climate policy

RE-RANKING

What is Re-Ranking?

Re-ranking is a two-stage retrieval process where an initial fast retrieval method gets top-K candidates, followed by a more accurate (but slower) model to reorder these candidates for final results.



RE-RANKING

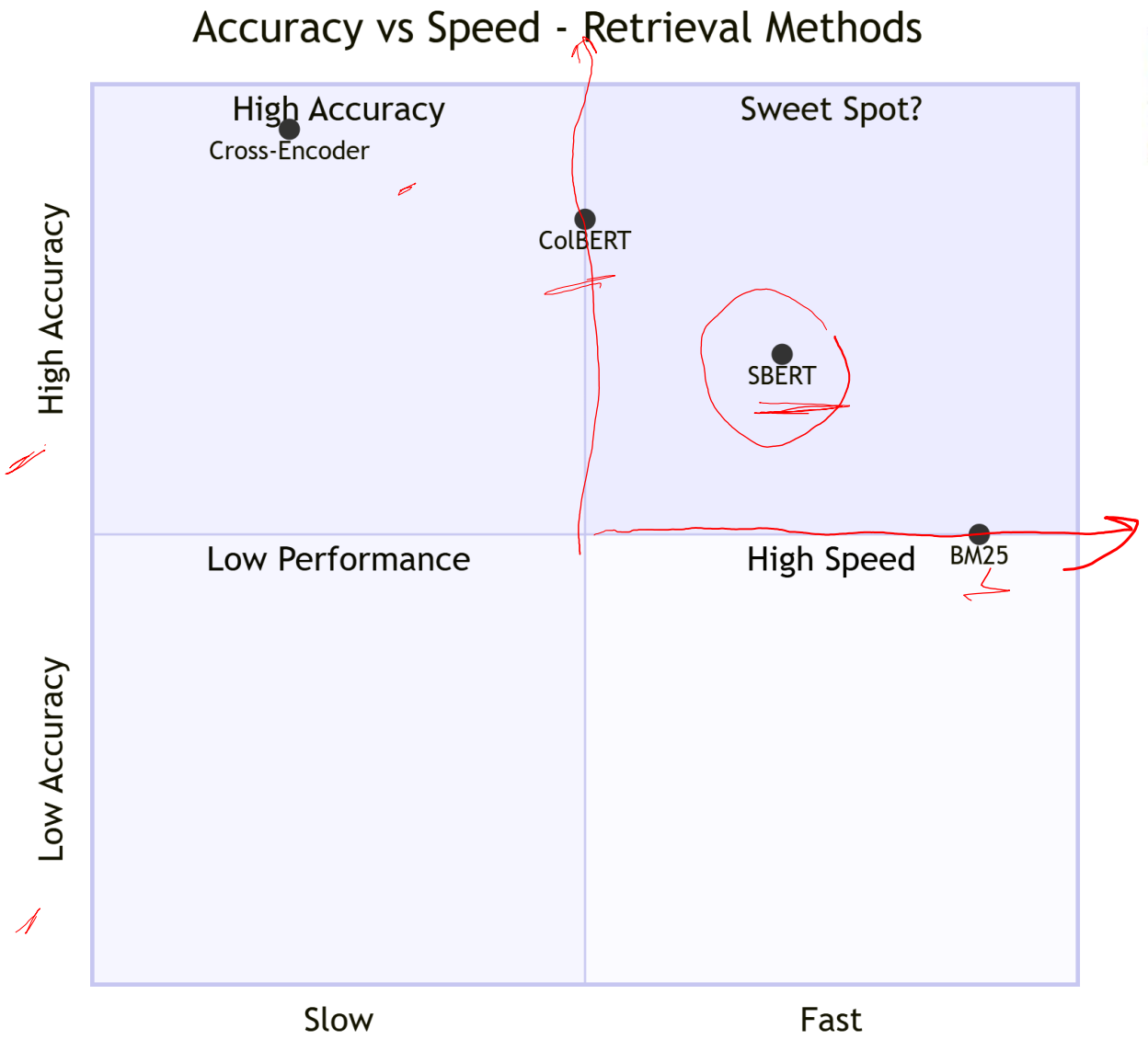
Why Re-Ranking? The Efficiency-Accuracy Trade-off?

The Solution: Two-Stage Retrieval

Stage 1 (Fast Retrieval):
BM25 / IVF / HNSW → Retrieve top 100 candidate documents
Time: ~10 ms

Stage 2 (Accurate Re-ranking):
Cross-Encoder model → Re-rank the 100 candidates
Time: ~100 ms

Total Time ≈ 110 ms



RE-RANKING Speed Comparison?

Comparison: Full corpus scan with cross-encoder:
1M documents × 50ms = 50,000,000ms = 50,000 seconds ≈ 14 hours (impossible.)
Two-stage approach:
Stage 1: 100 candidates × 0.1ms = 10ms
Stage 2: 100 candidates × 50ms = 5,000ms = 5 seconds
Total ≈ 5 seconds(possible)
Running a cross-encoder over the full corpus of 1M documents would take **~10,000 ms**, so the two-stage approach dramatically reduces computation while maintaining high accuracy.

Method	Time for 1M Documents	Accuracy
BM25 only	~10 ms	Low–Medium
Cross-Encoder only	~10,000 ms	Very High
Two-stage Retrieval (BM25 + Cross-Encoder)	~110 ms	High

Two-stage retrieval systems balance **efficiency and accuracy** by:
•Using fast retrieval models to generate candidates
•Applying expensive neural models only on a small subset of documents
This architecture is widely used in modern search systems and recommendation pipelines.

What are Re-Ranking Architectures?

Three Main Approaches:

1. Cross-Encoder Re-ranking

Query: "machine learning"
Document: "deep neural networks tutorial"

[CLS] query [SEP] document [SEP] → Transformer → [CLS] → Score

Pros: Most accurate, full interaction
Cons: Cannot pre-compute, must run for each query-doc pair

What are Re-Ranking Architectures?

Three Main Approaches:

2. Late Interaction Re-ranking (ColBERT style)

Query tokens → [q1, q2, ...]
Document tokens → [d1, d2, ...] (pre-computed)

$$\text{Score} = \Sigma \max(q\_i \cdot d\_j)$$

Pros: Pre-computed doc vectors, faster than cross-encoder
Cons: Less accurate than full cross-encoder

What are Re-Ranking Architectures?

Three Main Approaches:

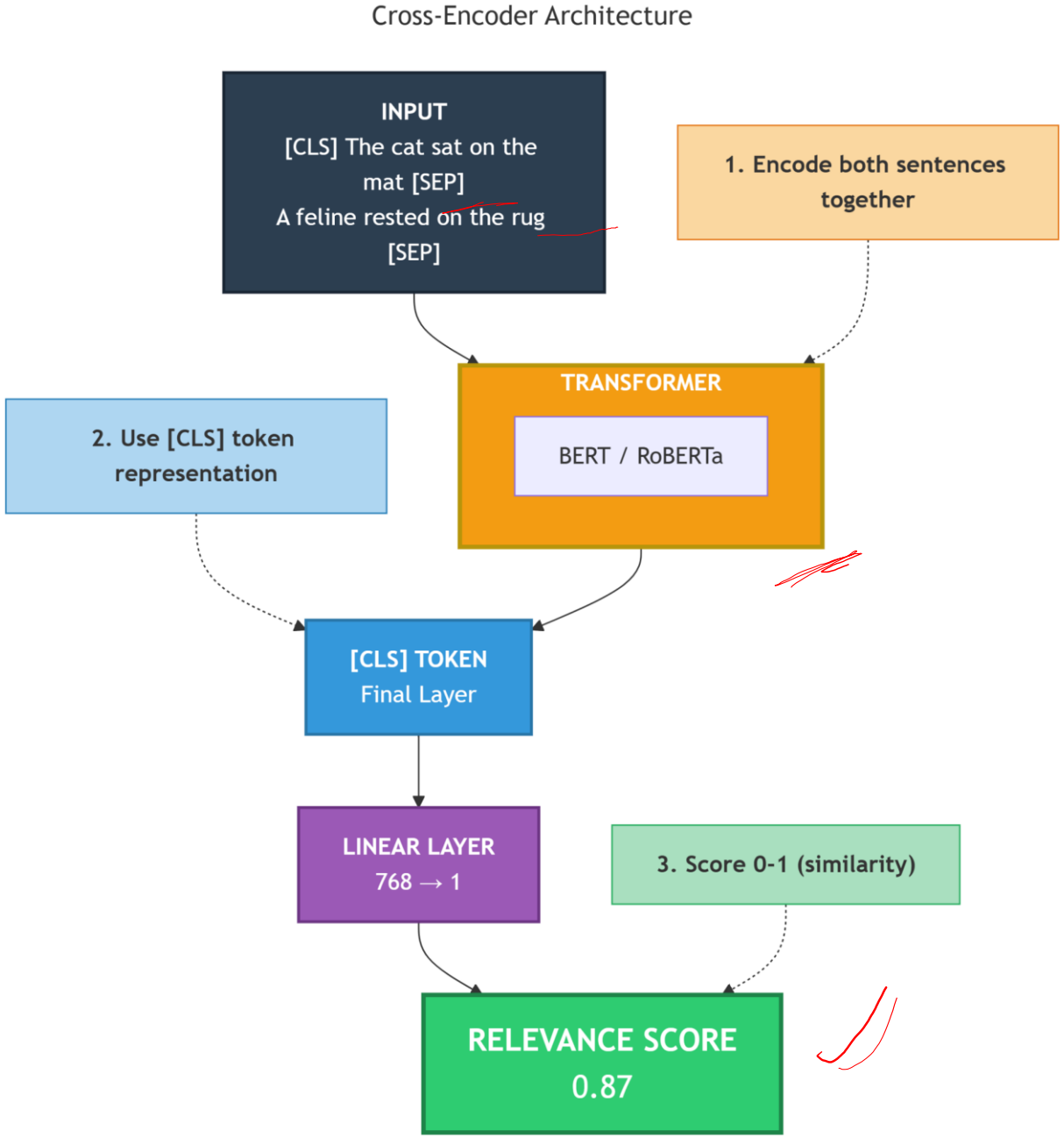
3. Ensemble Re-ranking

Score = w1·BM25_score + w2·Dense_score + w3·ColBERT_score

Pros: Combines multiple signals
Cons: Weight tuning required

Cross-Encoder Architecture

Key Characteristic: Query and document interact in **every layer** through self-attention.



Cross-Encoder Problem:

Query: "solar energy"
Document: "renewable power sources"

Token embeddings (simplified):
q_solar = [0.8, 0.3]
q_energy = [0.4, 0.7]
d_renewable = [0.7, 0.4]
d_power = [0.5, 0.6]
d_sources = [0.3, 0.2]

Cross-encoder processes ALL interactions:
Attention between q_solar and d_renewable
Attention between q_solar and d_power
Attention between q_solar and d_sources
Attention between q_energy and d_renewable
... and so on

CALCULATION:

Final Score Computation

[CLS] token captures the combined representation:
h_cls = f(q_solar, q_energy, d_renewable, d_power, d_sources)
score = σ(W·h_cls + b) = 0.92

Compare with Bi-encoder (SBERT):

q_vector = pool(q_solar, q_energy) = [0.6, 0.5]
d_vector = pool(d_renewable, d_power, d_sources) = [0.5, 0.4]
cosine = 0.98/(1.0×0.64) = 0.86

Cross-encoder (0.92) > Bi-encoder (0.86) because it captures fine-grained interactions!

Cross-Encoder Problem:

Step 1: Initial Retrieval (BM25)
Query: **“quantum computing applications”**
Top documents retrieved using BM25:

Rank	Document	BM25 Score
1	Quantum computing for machine learning	12.5
2	Applications of quantum mechanics	10.2
3	Quantum physics fundamentals	9.8
4	Computing in the quantum era	9.5
5	Machine learning algorithms	8.7

Cross-Encoder Problem:

Step 2: Cross-Encoder Scoring
Each query–document pair is scored by a cross-encoder model.

Document	Cross-Encoder Score
Quantum computing for machine learning	0.95
Applications of quantum mechanics	0.87
Quantum physics fundamentals	0.45
Computing in the quantum era	0.82
Machine learning algorithms	0.23

Cross-Encoder Problem:

Step 3: Re-Rank by Cross-Encoder Score

New Rank	Document	Cross-Encoder Score	Original BM25 Rank
1	Doc1	0.95	1
2	Doc2	0.87	2
3	Doc4	0.82	4
4	Doc3	0.45	3
5	Doc5	0.23	5

Key Insight:
Document 4 moved up because it is more relevant than BM25 initially estimated.
Document 3 moved down because it is less relevant.

Cross-Encoder Problem:

Re-Ranking with MonoT5

T5-Based Cross-Encoder for Ranking

Architecture:

Input:
Query: quantum computing applications
Document: quantum computing for machine learning
Processing:
T5 Model

Output:
Classification result → **"true" or "false"**

How MonoT5 Works:

MonoT5 is trained to answer the question:
“Does this document answer the query?”

Training Format:
Input Query: {q} Document: {d}

Target:
"true" if relevant
"false" if not relevant

Cross-Encoder Problem:

Re-Ranking with MonoT5 The model generates tokens and the probability of **"true"** is used as the relevance score.

Example1:

Input
Query: quantum computing applications
Document: quantum computing for ML

Output
"true" with probability **0.97**
Relevance Score = **0.97**

Example 2:

Input
Query: quantum computing applications
Document: quantum physics fundamentals

Output
"false" with probability **0.89**
Relevance Score = **0.11**

MonoT5 Numerical Example:

Re-Ranking with MonoT5

Scoring with Logits
Vocabulary (simplified):
true, false, yes, no

Document 1 Example
T5 output logits:

Token	Logit
true	5.2
false	-2.1
yes	3.1
no	-1.5

Softmax Calculation
P(true) = exp(5.2) / [exp(5.2) + exp(-2.1) + exp(3.1) + exp(-1.5) + ...]
exp(5.2) = 181.27
exp(-2.1) = 0.12
exp(3.1) = 22.20
exp(-1.5) = 0.22
P(true) = 181.27 / (181.27 + 0.12 + 22.20 + 0.22)
P(true) = 181.27 / 204.56
P(true) ≈ **0.886**
Final relevance score = **0.886**

MonoT5 Numerical Example:

Re-Ranking with MonoT5

Ranking Multiple Documents

Document	"true" logit	"false" logit	P(true)	Rank
Doc1	5.2	-2.1	0.886	2
Doc2	6.1	-3.0	0.941	1
Doc3	2.3	1.2	0.524	4
Doc4	4.5	-1.0	0.822	3

DuoT5 – Pairwise Re-Ranking

Re-Ranking with DuoT5

Even More Accurate: Compare Documents Directly.

Architecture:

Input:
Query: q
Document A: d1
Document B: d2

Processing:
T5 Model

Output:
Prediction → **"A" or "B"** (which document is more relevant)

The model evaluates **both documents together**, allowing it to capture interactions **between documents**, not just between query and document.

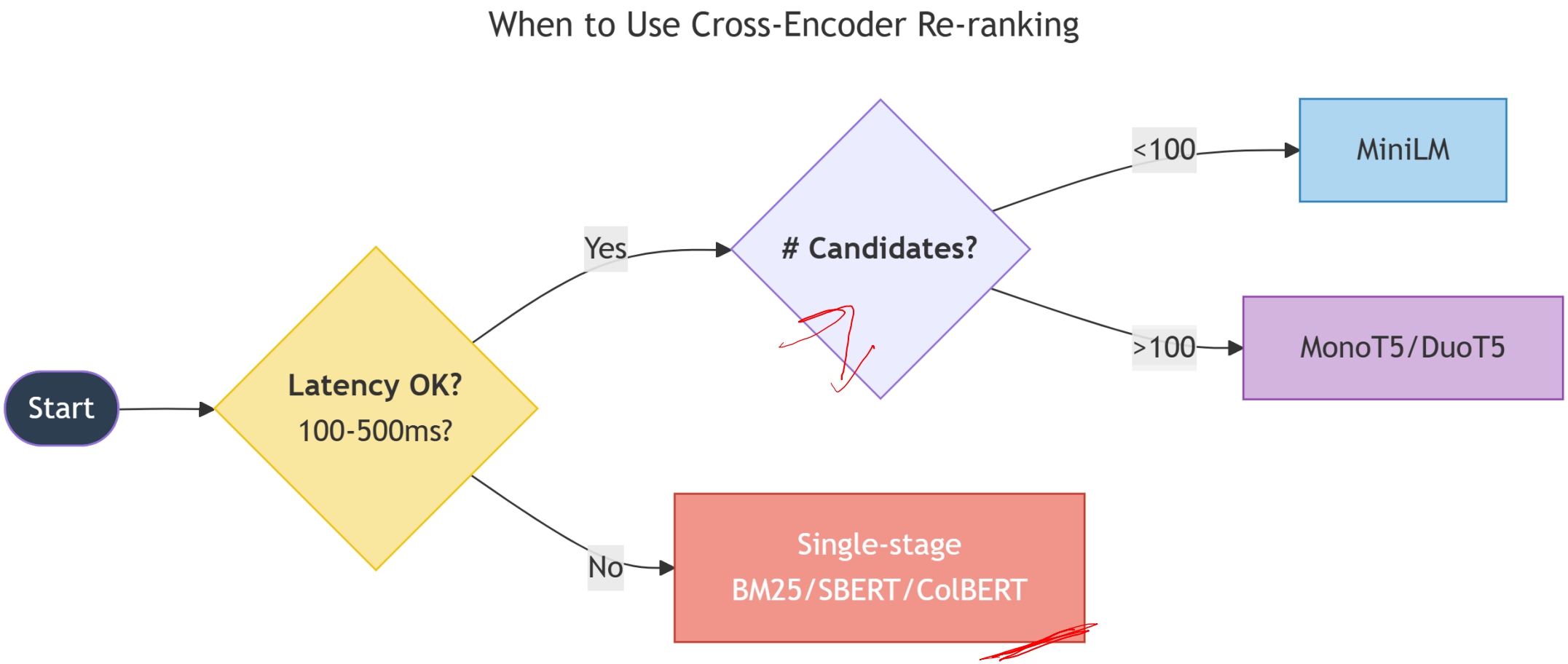
Side by side DuoT5 Working example:
Instead of scoring each document independently, DuoT5 performs **pairwise comparisons**.

Example Input:
Query: quantum computing applications
Document A: quantum computing for machine learning
Document B: quantum physics fundamentals

Target Output:
"A" (because Document A is more relevant to the query)

Training
Training examples follow this format:
Input
Query: {q}
Document A: {d1}
Document B: {d2}
Target
"A" if Document A is more relevant
"B" if Document B is more relevant
The model learns to **choose the better document**.

When to use what?



NUMERICAL PROBLEMS ON HYBRID SEARCH

Problem 1 - Hybrid Search with BM25 + SBERT

Problem Statement: 1

You have a corpus of 5 documents. Query: **"machine learning algorithms"**

Document Collection:

Doc1: "Machine learning for beginners tutorial"
Doc2: "Deep learning neural networks guide"
Doc3: "Supervised learning algorithms explained"
Doc4: "Python machine learning library scikit-learn"
Doc5: "Data science with R programming"

COMPUTE:

1.Normalize BM25 scores (min-max normalization)

2.Normalize dense scores

3.Combine with weights $\alpha=0.4$ (BM25) and $\beta=0.6$ (dense)

4.Calculate RRF scores with $k=60$

5.Compare weighted vs RRF rankings

NUMERICAL PROBLEMS ON HYBRID SEARCH

Min-Max normalization formula:

$$BM25_{norm} = \frac{BM25 - BM25_{min}}{BM25_{max} - BM25_{min}}$$

- BM25_min = 3.4
- BM25_max = 9.1
- Range = 9.1 - 3.4 = 5.7

Normalized BM25:

Doc	Calculation	Normalized BM25
Doc1	(8.5-3.4)/5.7	5.1/5.7 ≈ 0.895
Doc2	(6.2-3.4)/5.7	2.8/5.7 ≈ 0.491
Doc3	(9.1-3.4)/5.7	5.7/5.7 = 1.0
Doc4	(7.8-3.4)/5.7	4.4/5.7 ≈ 0.772
Doc5	(3.4-3.4)/5.7	0/5.7 = 0.0

NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 2: Normalize Dense Scores (Cosine Similarities)

Min-max normalization formula is same:

- Min = 0.62
- Max = 0.98
- Range = 0.36

Normalized Dense:

Doc	Calculation	Normalized Dense
Doc1	(0.98-0.62)/0.36 = 0.36/0.36	1.0
Doc2	(0.76-0.62)/0.36 = 0.14/0.36	0.389
Doc3	(0.97-0.62)/0.36 = 0.35/0.36	0.972
Doc4	(0.95-0.62)/0.36 = 0.33/0.36	0.917
Doc5	(0.62-0.62)/0.36 = 0	0.0

NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 3: Weighted Combination (α BM25 + β Dense)

Weights:

- α = 0.4
- β = 0.6

Weighted score formula:

$$Score = \alpha \cdot BM25_{norm} + \beta \cdot Dense_{norm}$$

Weighted Ranking Order:

1. Doc3 → 0.983
2. Doc1 → 0.958
3. Doc4 → 0.859
4. Doc2 → 0.429
5. Doc5 → 0.0

Doc	Calculation	Weighted Score
Doc1	0.40.895 + 0.61.0 = 0.358 + 0.6	0.958
Doc2	0.40.491 + 0.60.389 = 0.196 + 0.233	0.429
Doc3	0.41.0 + 0.60.972 = 0.4 + 0.583	0.983
Doc4	0.40.772 + 0.60.917 = 0.309 + 0.550	0.859
Doc5	0.40.0 + 0.60.0 = 0	0.0

NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 4: Reciprocal Rank Fusion (RRF)

RRF formula:

$$RRF\_score = \sum_i \frac{1}{k + rank_i}$$

- Here, **k = 60** (standard small constant to dampen high ranks)
- **rank_i** = position of document in each ranking (BM25 & Dense separately)

Step 4a: Assign ranks

- BM25 ranking: Doc3(1), Doc1(2), Doc4(3), Doc2(4), Doc5(5)
- Dense ranking: Doc1(1), Doc3(2), Doc4(3), Doc2(4), Doc5(5)

Step 4b: Compute RRF score for each doc

$$RRF = \frac{1}{60 + rank_{BM25}} + \frac{1}{60 + rank_{Dense}}$$

Example for Doc1:

$$RRF_{Doc1} = \frac{1}{60 + 2} + \frac{1}{60 + 1} = 1/62 + 1/61 \approx 0.01613 + 0.01639 \approx 0.03252$$

NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 4: Reciprocal Rank Fusion (RRF)

Doc	BM25 Rank	Dense Rank	RRF
Doc1	2	1	0.01613 + 0.01639 = 0.03252
Doc2	4	4	1/64 + 1/64 = 0.03125
Doc3	1	2	1/61 + 1/62 ≈ 0.01639 + 0.01613 = 0.03252
Doc4	3	3	1/63 + 1/63 ≈ 0.03175
Doc5	5	5	1/65 + 1/65 ≈ 0.03077

Step 4c: RRF Ranking Order

1.Doc1 / Doc3 → tie (0.03252)
2.Doc4 → 0.03175
3.Doc2 → 0.03125
4.Doc5 → 0.03077

RRF tends to **boost documents that appear high in both rankings**.

NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 5: Comparison

Document	Weighted Score	Weighted Rank	RRF Score	RRF Rank
Doc1	0.958	2	0.0325	1 (tie)
Doc2	0.429	4	0.0312	4
Doc3	0.983	1	0.0325	1 (tie)
Doc4	0.859	3	0.0318	3
Doc5	0.000	5	0.0308	5

GenerativeAI and Its Applications

NUMERICAL PROBLEMS ON HYBRID SEARCH

Problem 2 - Hybrid Search with Query Expansion

Problem Statement: 2

Original Query: **"AI ethics"**

Step 1: Apply Query Expansion using LLM

Expanded terms:

- "artificial intelligence morality"
- "machine learning guidelines"
- "responsible AI principles"
- "AI bias fairness"
- "ethical AI frameworks"

Step 2: Retrieve documents with expanded query

Documents:

DocA: "Guidelines for ethical artificial intelligence development"
DocB: "Machine learning bias and fairness in healthcare"
DocC: "Principles of responsible innovation in tech"
DocD: "AI regulation and policy frameworks"
DocE: "Data privacy in machine learning systems"

Step 3: Compute relevance scores

Task: Calculate hybrid scores (α=0.3 BM25, β=0.7 Dense) and rank documents.

NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 1: Query Expansion (LLM)

Expanded terms:
- artificial intelligence morality
- machine learning guidelines
- responsible AI principles
- AI bias fairness
- ethical AI frameworks

Step 2: Retrieve Documents with Expanded Query

Document	Text
DocA	Guidelines for ethical artificial intelligence development
DocB	Machine learning bias and fairness in healthcare
DocC	Principles of responsible innovation in tech
DocD	AI regulation and policy frameworks
DocE	Data privacy in machine learning systems

NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 3: Relevance Scores

Document	Contains Original	Contains Expanded	BM25	Dense (Cosine)
DocA	AI, ethics	artificial intelligence, guidelines, ethical	12.5	0.95
DocB	none	machine learning, bias, fairness	8.2	0.92
DocC	none	principles, responsible	6.7	0.78
DocD	none	regulation, frameworks	5.1	0.85
DocE	none	data privacy, machine learning	7.3	0.88

NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 4: Normalize Scores (Min-Max)
BM25 Normalized (min=5.1, max=12.5):

Doc	BM25_norm
DocA	1.000
DocB	0.419
DocC	0.216
DocD	0.000
DocE	0.297

Dense Normalized (min=0.78, max=0.95):

Doc	Dense_norm
DocA	1.000
DocB	0.824
DocC	0.000
DocD	0.412
DocE	0.588

NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 5: Weighted Combination (α=0.3 BM25, β=0.7 Dense)

$$Score = 0.3 * BM25_{norm} + 0.7 * Dense_{norm}$$

Doc	Weighted Score	Calculation
DocA	1.000	0.31 + 0.71 = 1.0
DocB	0.703	0.30.419 + 0.70.824 = 0.703
DocC	0.065	0.30.216 + 0.70 = 0.065
DocD	0.288	0.30 + 0.70.412 = 0.288
DocE	0.501	0.30.297 + 0.70.588 = 0.501

NUMERICAL PROBLEMS ON HYBRID SEARCH

Step 6: Final Ranking

Rank	Document	Score	Title
1	DocA	1.000	Guidelines for ethical AI development
2	DocB	0.703	Machine learning bias and fairness in healthcare
3	DocE	0.501	Data privacy in machine learning systems
4	DocD	0.288	AI regulation and policy frameworks
5	DocC	0.065	Principles of responsible innovation in tech

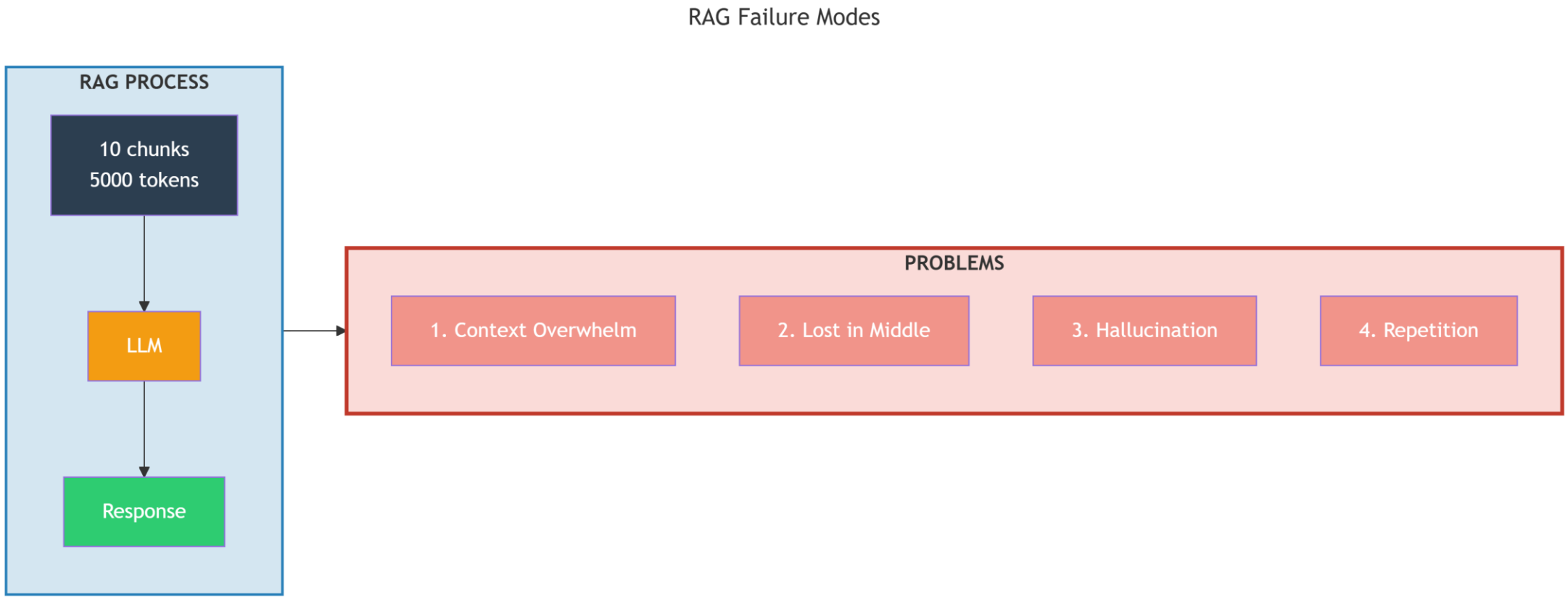
Query expansion and hybrid scoring significantly boosts relevant documents that do not contain the exact original query terms.

GENERATION ENHANCEMENT TECHNIQUES

Problems:
1. LLM gets overwhelmed by too much context
2. LLM ignores middle chunks (lost in the middle)
3. LLM hallucinates when context is insufficient
4. LLM repeats information from multiple chunks

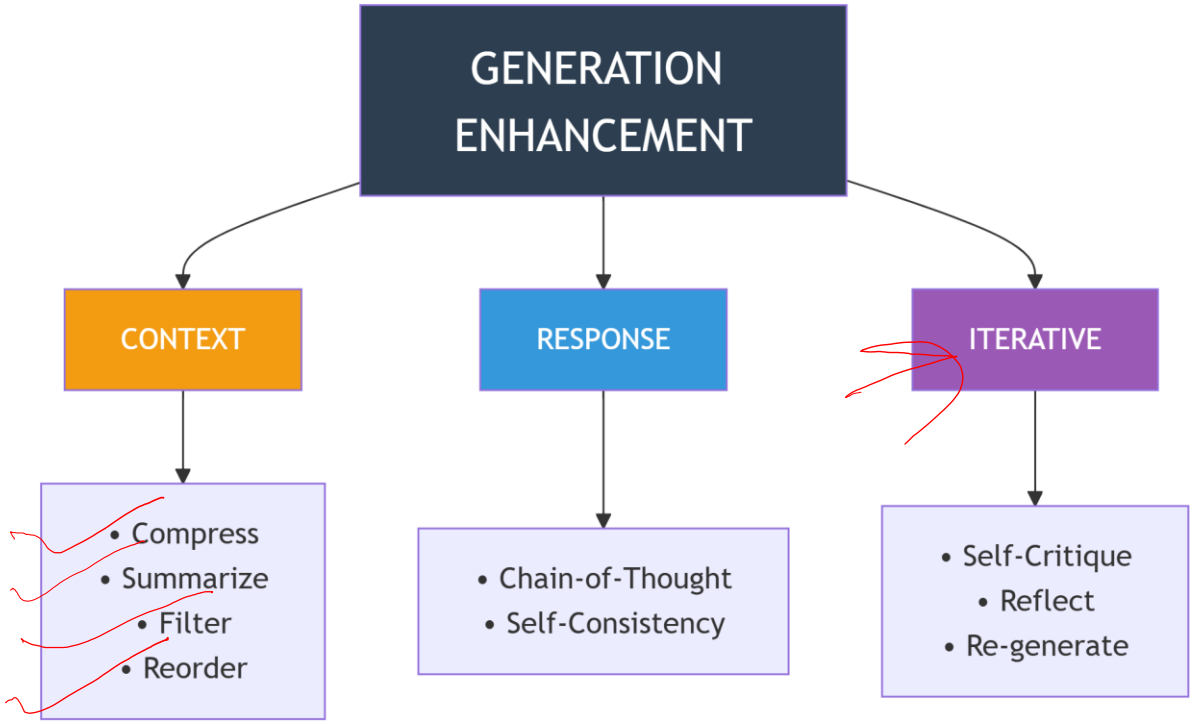
What is Generation Enhancement?

Generation Enhancement refers to techniques applied after retrieval to improve how the LLM uses retrieved context to generate better, more accurate responses.



GenerativeAI and Its Applications

GENERATION ENHANCEMENT TECHNIQUES



Three Main Categories:

1.Context Optimization: Make retrieved info more usable

2.Response Synthesis: Better prompting strategies

3.Iterative Refinement: Generate, evaluate, improve

GENERATION ENHANCEMENT TECHNIQUES

Technique 1 - Context Compression

Problem: Too many tokens = expensive + confusing

Solution 1: Summarization

text

Before: 5 chunks, 2500 tokens

Chunk 1: "Quantum computing uses qubits which can exist in superposition..."

Chunk 2: "Superposition allows qubits to be 0 and 1 simultaneously..."

Chunk 3: "Entanglement is another key principle where qubits are correlated..."

Chunk 4: "Quantum gates manipulate qubits to perform computations..."

Chunk 5: "Shor's algorithm uses quantum effects to factor large numbers..."

After Summarization (LLM-generated):

"Quantum computing leverages qubit properties like superposition and entanglement, manipulated via qu antum gates, to perform computations such as Shor's factoring algorithm." (150 tokens)

Compression ratio: 2500 → 150 (16.7x reduction!)

Solution 2: Extractive Compression

text

Keep only the most relevant sentences based o

Original: [sent1, sent2, sent3, sent4, sent5, ...]

Score each: [0.9, 0.2, 0.8, 0.1, 0.7, ...]

Keep top-K: [sent1, sent3, sent5]

GENERATION ENHANCEMENT TECHNIQUES

Technique 1 - Context Compression

After Summarization (LLM-generated):

"Quantum computing leverages qubit properties like superposition and entanglement, manipulated via qu antum gates, to perform computations such as Shor's factoring algorithm." (150 tokens)

Compression ratio: 2500 → 150 (16.7x reduction!)

Solution 2: Extractive Compression

Keep only the most relevant sentences based on query similarity:

Original: [sent1, sent2, sent3, sent4, sent5, ...]

Score each: [0.9, 0.2, 0.8, 0.1, 0.7, ...]

Keep top-K: [sent1, sent3, sent5]

GENERATION ENHANCEMENT TECHNIQUES

Technique 2 - Context Reordering

The "Lost in the Middle" Problem:

Research shows LLMs pay most attention to the **beginning** and **end** of context, ignoring the middle.

Solution: Put most relevant chunks at beginning and end!

Before (random order):

[Chunk3 (rel=0.5), Chunk1 (rel=0.9), Chunk5 (rel=0.3),

Chunk2 (rel=0.8), Chunk4 (rel=0.4)] → After (reordered by relevance, with best at ends):

[Chunk1 (0.9), Chunk3 (0.5), Chunk4 (0.4), Chunk5 (0.3), Chunk2 (0.8)]

Best at start Worst in middle 2nd best at end

Improvement: 15-20% better answer quality!

GENERATION ENHANCEMENT TECHNIQUES

Technique 3 - Chain-of-Thought with Context

Standard Prompt:

Context: {retrieved_chunks}

Question: {query}

Answer:

Enhanced with Chain-of-Thought:

Context: {retrieved_chunks}

Question: {query}

1. First, what are the key pieces of information from the context?

   [LLM extracts relevant facts]

2. How do these pieces relate to each other?

   [LLM identifies connections]

3. What additional reasoning is needed?

   [LLM applies logic]

4. Based on the above, the answer is:

   [LLM provides final answer]

GENERATION ENHANCEMENT TECHNIQUES

EXAMPLE: Context: Articles about climate change effects
Question: How will rising temperatures affect crop yields?

1. Key information:
   - Temperature increases of 2°C reduce wheat yields by 6%
   - Corn yields drop 7% per degree above 30°C
   - CO2 fertilization could boost some crops by 15%

2. Relationships:
   - Temperature effect is negative and non-linear
   - CO2 effect partially offsets but varies by crop

3. Reasoning:
   - Net effect depends on region and crop type
   - Tropical regions will see more negative impacts

4. Answer: Rising temperatures will generally decrease crop yields, with impacts varying by region and crop...

GENERATION ENHANCEMENT TECHNIQUES

Technique 4 - Self-Consistency and Reflection

Self-Consistency: Generate multiple answers and vote

Round 1: Generate answer with temperature=0.7

Round 2: Generate answer with temperature=0.7

Round 3: Generate answer with temperature=0.7

Answers:

1. "The capital of France is Paris"

2. "Paris is the capital city of France"

3. "France's capital is Paris“

Consensus: Paris (3/3 votes)

Self-Reflection: Generate, critique, improve

Step 1: Generate initial answer

Step 2: Ask LLM to critique its own answer

Step 3: Generate improved answer based on critique

Example:

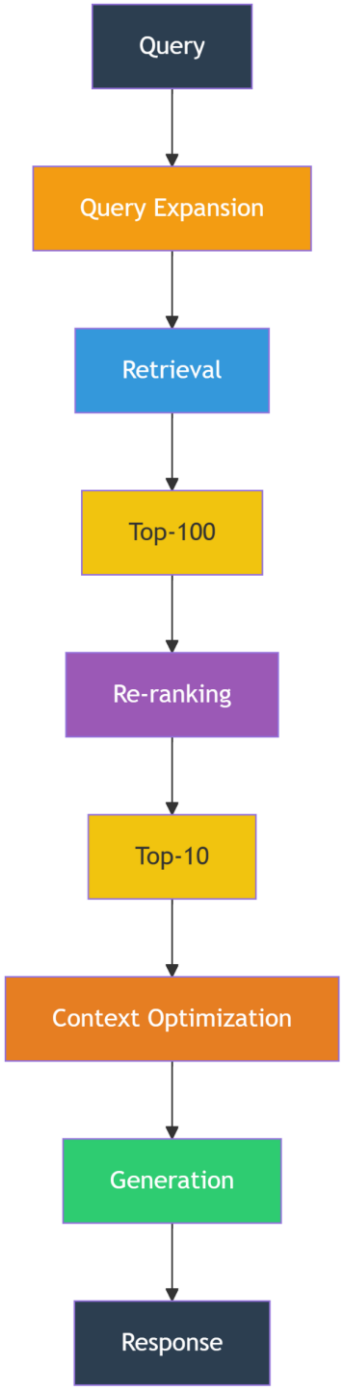
Initial: "Quantum computing is faster than classical computing."

Critique: "Vague - faster at what? Need to specify task types."

Improved: "Quantum computing provides exponential speedup for specific tasks like factoring large numbers, but is not universally faster than classical computing."

GenerativeAI and Its Applications

Complete RAG Pipeline with All Enhancements



PES UNIVERSITY

THANK YOU

Generative AI and Its Applications

Unit 3

(UE23CS342BA4)

Fine-Tuning, Data Resolution/Precision, Quantization, MoE

Dr. Pooja Agarwal
Dept. of CSE, PES University, Bangalore

Acknowledgement

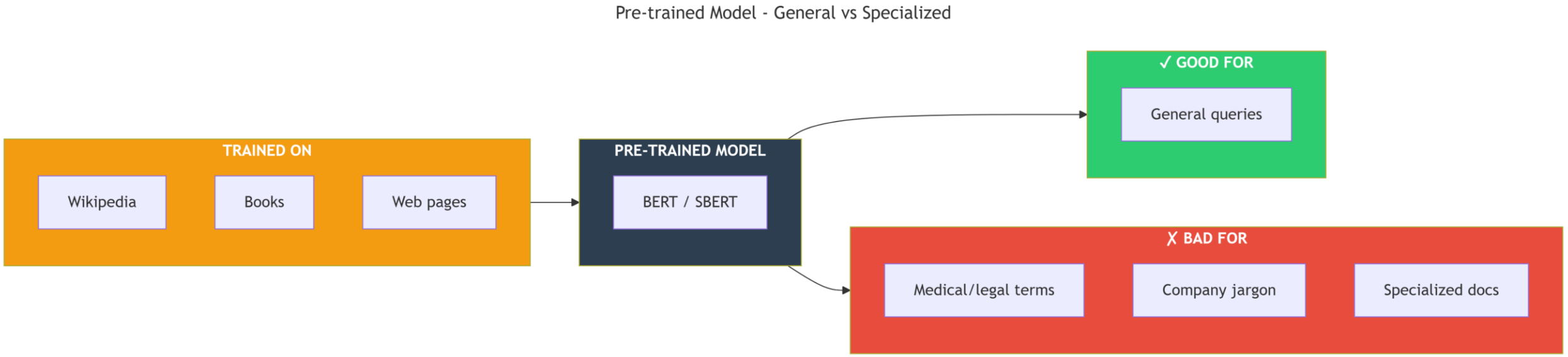
The slides are prepared from various resources from the Universities from abroad and India. Also, some material is taken from reliable resources from internet throughout this course.

FINE-TUNING FOR RAG

how do we make LLM systems accurate, efficient, and scalable in real-world applications?

Why Fine-Tune for RAG?!

The Problem with Pre-trained Models:

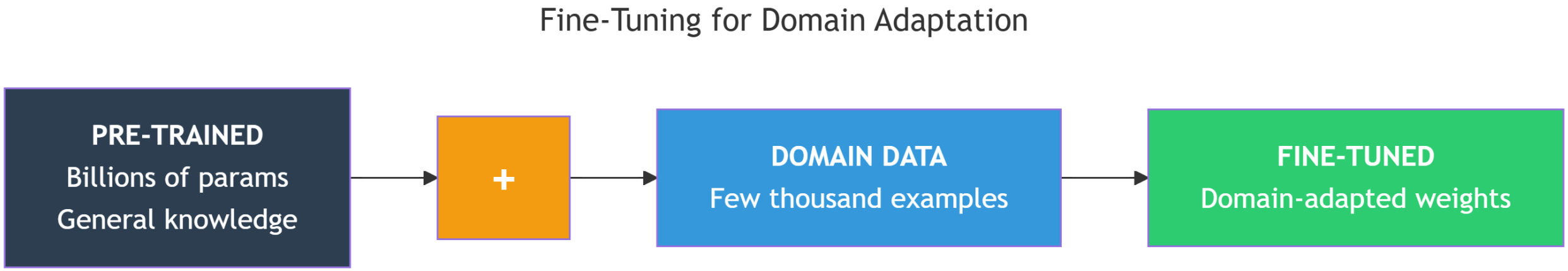


A pre-trained model is like a **medical school graduate** – they know general medicine but need **residency training** to become a specialist surgeon. Fine-tuning is that specialized training for your specific domain.

FINE-TUNING FOR RAG

Why Fine-Tune for RAG?!

What is Fine-Tuning? : **Fine-Tuning** is the process of taking a pre-trained model and further training it on a smaller, domain-specific dataset to adapt its knowledge and representations to a particular task or domain.



Loss Before Fine-tuning: — high —▶
Loss After Fine-tuning: — low —▶

The model updates weights to minimize loss on your specific data while retaining general knowledge.

FINE-TUNING FOR RAG

Why Fine-Tune for RAG?!

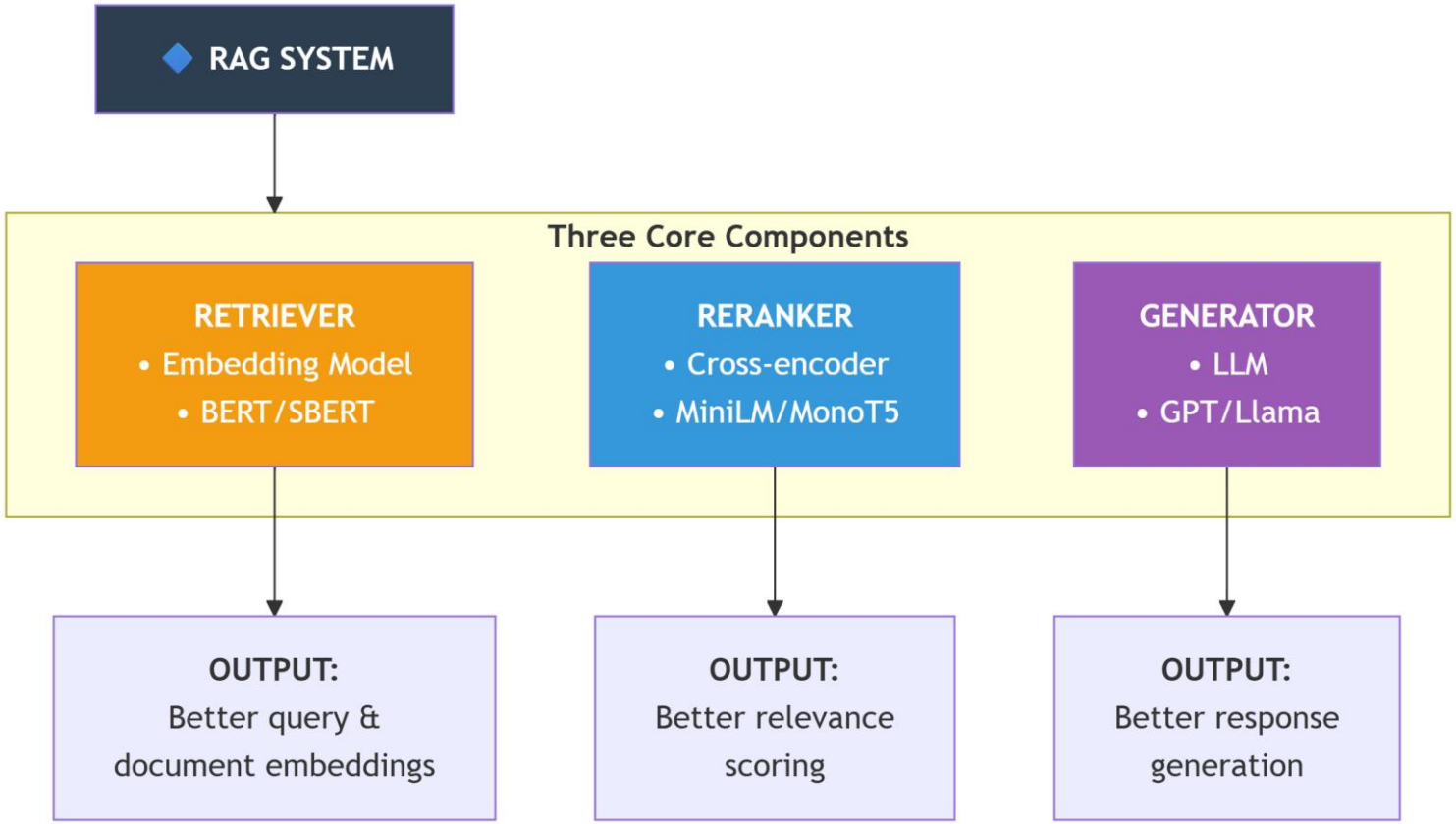
RAG systems have three fine-tunable components::

Which to Fine-Tune?

- **Retriever:** When your documents have domain-specific terminology
- **Reranker:** When you need high-precision relevance judgments
- **Generator:** When responses need specific style/format/domain knowledge

FINE-TUNING FOR RAG

What to Fine-Tune in RAG?!



Why Fine-Tune for RAG?!

Fine-Tuning the Retriever (Embedding Model):

Goal: Make embeddings place similar domain documents closer together.

Training Data Format:

Triplets: (Query, Positive Document, Negative Document)

Example (Medical Domain):
Query: "What are the side effects of Metformin?"
Positive: "Metformin may cause gastrointestinal disturbances including diarrhea..."
Negative: "Lifestyle modifications for diabetes management..."

FINE-TUNING FOR RAG

We maximize similarity of correct pair vs all wrong pairs

Why Fine-Tune for RAG?!

Loss Function: Multiple Negative Ranking Loss (MNRL):

$$\text{Loss} = -\log\left(e^{(\text{sim}(q,d^{+})/\tau)} / \Sigma\, e^{(\text{sim}(q,d_j)/\tau)} \right)$$

Where:
- sim(q,d) = cosine similarity
- τ = temperature parameter (typically 0.01-0.1)
- denominator includes 1 positive + N negatives

Numerical Example:

Initial Model Prediction
Suppose a pretrained model predicts the probability for a binary classification task.

Input example: **“This movie is great”**
True label: **1 (Positive)**
Model prediction before fine-tuning:

$$\hat{y} = 0.40$$

Loss function: **Binary Cross-Entropy**

$$\boldsymbol{Loss} = -\big(\boldsymbol{y}\mathbf{log}(\hat{\boldsymbol{y}}) + (\mathbf{1} - \boldsymbol{y})\mathbf{log}(\mathbf{1} - \hat{\boldsymbol{y}})\big)$$

FINE-TUNING FOR RAG

Numerical Example:

Step 1: Loss Before Fine-Tuning

$$y = 1, \hat{y} = 0.40$$
$$Loss = -\big(1 \cdot \log(0.40)\big)$$
$$Loss = -(-0.916)$$
$$Loss \approx 0.916$$

Loss is **high**, meaning the prediction is far from the true label.

Step 2: After Fine-Tuning

After training on domain-specific data, the model updates its weights.

New prediction:

$\hat{y} = 0.90$

Step 3: Loss After Fine-Tuning

$$Loss = -\big(1 \cdot \log(0.90)\big)$$
$$Loss = -(-0.105)$$
$$Loss \approx 0.105$$

Stage	Prediction	Loss
Before Fine-Tuning	0.40	0.916
After Fine-Tuning	0.90	0.105

Loss Before Fine-tuning: 0.916

Loss After Fine-tuning: 0.105

Retriever finds candidates, reranker decides the winner

Fine-Tuning the Reranker (Cross-Encoder)

Goal: Make cross-encoder better at judging relevance for your domain.

Training Data Format: **Training Data Format**

Models are trained on **query–document pairs with relevance scores**.

Example:
Query: **“diabetes treatment guidelines”**
Document: **“ADA 2023 guidelines recommend Metformin as first-line therapy”**
Label: **5 (highly relevant)**
Query: **“diabetes treatment guidelines”**
Document: **“History of insulin discovery”**
Label: **1 (barely relevant)**

Fine-Tuning the Reranker (Cross-Encoder)

Goal: Make cross-encoder better at judging relevance for your domain.

Loss Function: Binary Cross-Entropy or MSE:

Binary Classification (Relevant / Not Relevant)

Binary Cross-Entropy Loss:

$$Loss = -[y \cdot \log(\hat{y}) + (1 - y) \cdot \log(1 - \hat{y})]$$

Where:

- **y** = true label (0 or 1)
- **ŷ** = predicted probability

Score Regression (Relevance Scores)

Mean Squared Error:

$$Loss = \left(y - \hat{y}\right)^2$$

Used when labels are **continuous scores (0–5)**.

Numerical Example (Binary Cross-Entropy):

True label:

$y = 1$ (relevant)

Model prediction:

$\hat{y} = 0.85$

Loss calculation:

$$\begin{aligned} \text{Loss} &= -[1 \cdot \log(0.85) + 0 \cdot \log(0.15)] \\ &= -\log(0.85) \\ &= -(-0.1625) \\ &= 0.1625 \end{aligned}$$

Loss is **low**, meaning the prediction is close to the correct label.

Example with Poor Prediction

If the model predicts:

$\hat{y} = 0.40$

Loss becomes:

$$\begin{aligned} \text{Loss} &= -\log(0.40) \\ &= 0.916 \end{aligned}$$

Prediction	Loss	Interpretation
0.85	0.1625	Good prediction
0.40	0.916	Poor prediction

- Lower loss means **better model predictions**.
- Training updates model weights to **minimize this loss over all query–document pairs**.

Numerical Example (Binary Cross-Entropy):

Fine-Tuning the Reranker – Implementation:

https://huggingface.co/blog/train-reranker#:~:text=Through%20finetuning%2C%20the%20model%20can,those%20baselines%20are%20much%20bigger.

Fine-Tuning the Generator (LLM) Instruction Fine-Tuning and PEFT

Goal

Make a large language model generate responses in a **desired style, format, and domain knowledge**.

Training Data Format:

Instruction Fine-Tuning uses:

Instruction + Context + Response

Example (Medical QA):

Instruction:

Answer the medical question based on the context.

Context:

Metformin is contraindicated in patients with eGFR < 30 mL/min.

Response:

Metformin should not be used in patients with severe kidney disease (eGFR <30 mL/min) due to the risk of lactic acidosis.

Fine-Tuning the Generator (LLM)

Goal : Make a large language model generate responses in a **desired style, format, and domain knowledge**.

Parameter-Efficient Fine-Tuning (PEFT):
Full Fine-Tuning

Update **all model parameters**.

Example:
Llama2 7B → 7 billion parameters must be updated.
This requires **large GPU memory and high cost**.

LoRA (Low-Rank Adaptation) ***We don't change the brain, we add a small adapter***
Full Fine-Tuning is expensive
Solution:

- Freeze model
- Train small matrices

Pretrained weights are **frozen** and only small adapter layers are trained.
Pretrained Weight Matrix:

$$W \in \mathbb{R}^{dxk} \text{ (frozen)}$$

LoRA Update

$$\Delta W = B \cdot A,\ W + BA \text{ (only A \& B trained)}$$

Where

$$A \in \mathbb{R}^{rxk}$$

$$B \in \mathbb{R}^{dxr}$$

$$r << \min(d,k)$$

Typical rank r = 8–64

Trainable parameters are **less than 1% of the original model**.

LoRA – Detailed Explanation

How LoRA Works

Original Layer:

Input x

→ Weight W

→ Output y

LoRA Modified Layer:

Input x

→ W · x (frozen)

→ B · A · x (trainable)

Final Output

$$y = W \cdot x + B \cdot A \cdot x$$

Matrix Definitions

W : Original pretrained weights (d × k) – Frozen

A : Low-rank matrix (r × k) – Trainable

B : Low-rank matrix (d × r) – Trainable

r : Rank (8–64), much smaller than d or k

Parameter Savings Example:

For a transformer layer:

d = 4096

k = 4096

Full Fine-Tuning

4096 × 4096 = 16.7 million trainable parameters

LoRA with rank r = 8

A parameters = 8 × 4096 = 32,768

B parameters = 4096 × 8 = 32,768

Total trainable parameters = 65,536

This is **0.39% of the original parameters**.

LoRA Numerical Example

Input vector
x = [1.0, 2.0]
Original weight matrix
W =[0.5 0.3
0.2 0.4]

Step 1: Original Forward Pass

W · x

= [0.5×1.0 + 0.3×2.0,0.2×1.0 + 0.4×2.0]
= [1.1, 1.0]

Step 2: LoRA Contribution

LoRA matrices (rank r = 1)
A = [0.1 0.2]
B =[0.3 0.4]
Compute: **A · x**
= [0.1×1.0 + 0.2×2.0]
= [0.5]
Then: **B · (A · x)**
= [0.3×0.5, 0.4×0.5]
= [0.15, 0.20]

Step 3: Combined Output

y = W·x + B·A·x

y = [1.1, 1.0] + [0.15, 0.20]
y = [1.25, 1.20]

During Training

W remains **fixed**
W =[0.5 0.3
0.2 0.4]

- Only **A and B matrices update using gradients**.
- This allows efficient fine-tuning with **very few trainable parameters**.

LoRA Numerical Example

Fine-Tuning an LLM for Command Generation

https://huggingface.co/docs/transformers/en/training

FINE-TUNING FOR RAG

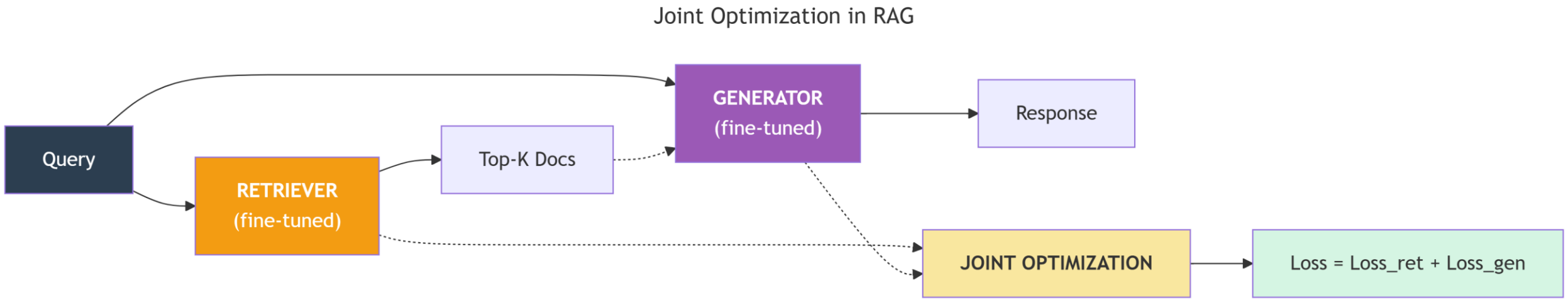
Instead of training retriever and generator separately, train both together.

RA-DIT - Retrieval-Augmented Dual Instruction Tuning

State-of-the-art: Fine-tune both retriever and generator together

Fine-tune **both the retriever and the generator together** for better retrieval-augmented generation.
This approach improves:

- Document retrieval quality
- Response generation accuracy
- End-to-end RAG system performance



RA-DIT - Retrieval-Augmented Dual Instruction Tuning

Training Objective:

Total loss = retrieval + generation

Joint loss combines **retrieval loss** and **generation loss**.

$$L_{total} = \alpha \cdot L_{retriever} + \beta \cdot L_{generator}$$

Where:

- **$L_{retriever}$** → improves document retrieval
- **$L_{generator}$** → improves response generation
- **α, β** → weighting parameters

Example:

$\alpha = 0.5$

$\beta = 0.5$

RA-DIT – Numerical Example

Step 1: Retriever Loss (Multiple Negatives Ranking Loss)
Query: "diabetes treatment“

Positive document: "Metformin is first-line for type 2 diabetes"
Negative documents: History of insulin, Dietary guidelines, Exercise benefits

Similarity scores:

Pair	Similarity
$sim(q,d^{+})$	0.92
$sim(q,d^{-1})$	0.45
$sim(q,d^{-2})$	0.38
$sim(q,d^{-3})$	0.41

Temperature:
τ = 0.05
Retriever loss:

```
L_ret = -log( e^(0.92/0.05) /
                (e^(0.92/0.05) + e^(0.45/0.05)
                 + e^(0.38/0.05) + e^(0.41/0.05)))
= -log( e^18.4 /(e^18.4 + e^9 + e^7.6 + e^8.2))
                    ≈ -log(0.9998)
                      ≈ 0.0002
```

Retriever loss is **very small**, meaning the model correctly ranks the relevant document highest.

RA-DIT – Numerical Example Generator Loss

Step 2: Generator Loss
(Cross-Entropy)
Target response:
`"Metformin is prescribed as first-line treatment..."`
Predicted logits for first token:

Token	Logit
Metformin (correct)	5.2
Insulin	2.1
Diet	1.8

Softmax probability:

```
P(target) = e^5.2/(e^5.2 + e^2.1 + e^1.8 +...)
          = 181.3/(181.3 + 8.2 + 6.0 +...)
                                ≈ 0.92
```

Generator loss:

```
L_gen = -log(0.92)
      ≈ 0.083
```

RA-DIT – Numerical Example

Step 3: Joint Loss

Weights:

$\alpha = 0.5$

$\beta = 0.5$

$$L_{total} = \alpha \cdot L_{ret} + \beta \cdot L_{gen}$$

$L_{total} = 0.5 \times 0.0002 + 0.5 \times 0.083$

$L_{total} \approx 0.0416$

RA-DIT jointly optimizes:

- **Retriever accuracy**
- **Generator response quality**

This leads to **better retrieval-augmented LLM systems**.

FINE-TUNING FOR RAG

Rule of Thumb

Fine-tune the component **closest to the specific requirement**:

1.Start with the **retriever** to improve document retrieval.

2.Fine-tune the **reranker** if higher ranking precision is needed.

3.Fine-tune the **generator** only when specific response styles, formats, or reasoning behaviors are required.

When to Fine-Tune – Decision Matrix

Scenario	Retriever	Reranker	Generator
Domain-specific vocabulary (medical, legal, technical)	Required	Helpful	Optional
Specific response format (JSON, structured tables, templates)	Not required	Not required	Required
High precision retrieval required	Optional	Required	Not required
Low-resource domain with limited training data	Helpful	Helpful	Optional
Multilingual applications	Required	Helpful	Helpful
Company internal documents or proprietary knowledge	Required	Helpful	Optional

PES UNIVERSITY

CELEBRATING 50 YEARS

DATA RESOLUTION & PRECISION

DATA RESOLUTION & PRECISION

WHAT IS DATA RESOLUTION & PRECISION?

Definition Precision = how accurately numbers are stored

Data resolution (precision) refers to the numerical accuracy used to represent **model weights, activations, and embeddings**.

Higher precision provides more accurate numerical representation but requires **more memory and computation**.

Common Numeric Precisions:

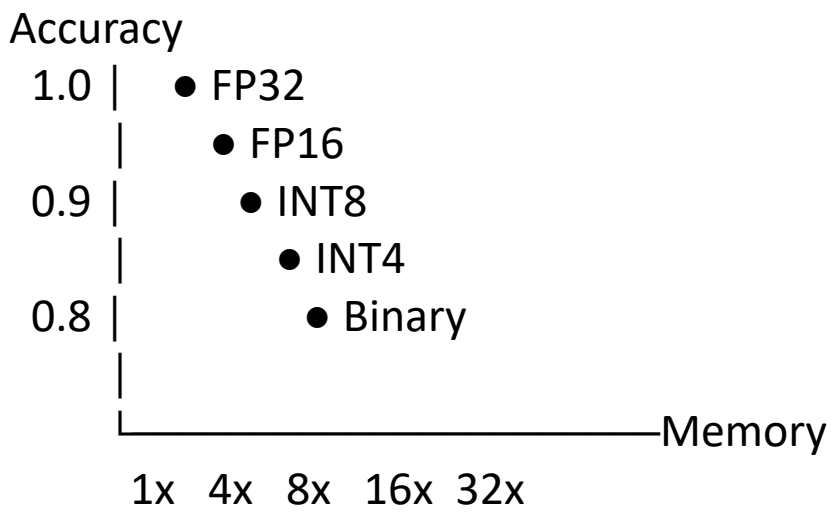
- FP32 → very accurate
- INT8 → compressed

Precision	Bits	Numerical Range	Typical Use
FP32	32	±1.4e-45 to 3.4e38	Training (master weights)
FP16	16	±5.96e-8 to 65504	Faster Training
BF16	16	±1.18e-38 to 3.4e38	Stable training
INT8	8	-128 to 127	Deployment
INT4	4	-8 to 7	Extreme compression
Binary	1	-1, 0, 1	Specialized hardware

DATA RESOLUTION & PRECISION

Precision vs Memory Trade-off

Precision	Relative Accuracy	Memory Usage
FP32	Highest	1×
FP16	Very high	2× reduction
INT8	High	4× reduction
INT4	Moderate	8–16× reduction
Binary	Lowest	32× reduction



PES UNIVERSITY

CELEBRATING 50 YEARS

Quantization

QUANTIZATION TECHNIQUES

QUANTIZATION TECHNIQUES What is Quantization?

Definition

Quantization is the process of **reducing the numerical precision** of model parameters and activations from high precision formats (such as FP32) to lower precision formats (such as INT8 or INT4).

Purpose:

- Reduce memory usage
- Accelerate inference
- Enable deployment on limited hardware

Analogy

Quantization is similar to rounding a value such as:

3.14159 → 3.14

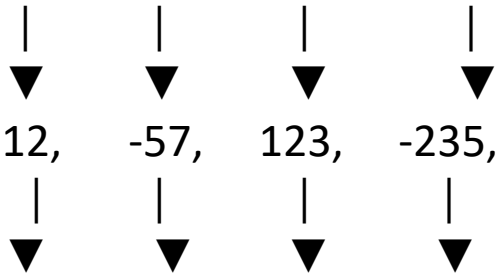
The number becomes easier to store and compute with, but introduces a small approximation error.

QUANTIZATION TECHNIQUES

QUANTIZATION TECHNIQUES What is Quantization?

Process
1.Divide by scale
2.Round
3.Store as integer
4.Reconstruct

FP32 Weights: [0.1234, -0.5678, 1.2345, -2.3456, ...]



INT8 Quantized: [12, -57, 123, -235, ...]

Scale Factor: 0.01 0.01 0.01 0.01 (Divide by scale)

Reconstructed: [0.12, -0.57, 1.23, -2.35, ...]

Error: 0.0034 0.0022 0.0045 0.0044

QUANTIZATION TECHNIQUES

Quantization Mathematics What is Quantization?

Linear Quantization (Symmetric)

Quantization formula:

$$x\_int = (round(x / s))$$

Where:

s = scale factor

b = number of bits

Scale factor:

$$s = max(|x|) / (2^{(b-1)} - 1)$$

Dequantization

$$x\_dequant = x\_int \times s$$

Numerical Example:

FP32 weights:
[0.5, -1.2, 2.3, -0.8, 1.7]

Maximum absolute value:
2.3

INT8 range:
-127 to 127

Scale factor:
s = 2.3 / 127 = 0.01811

QUANTIZATION TECHNIQUES

Quantization Mathematics

Quantization

FP32	Calculation	INT8
0.5	0.5 / 0.01811	28
-1.2	-1.2 / 0.01811	-66
2.3	2.3 / 0.01811	127
-0.8	-0.8 / 0.01811	-44
1.7	1.7 / 0.01811	94

Quantized values:
[28, -66, 127, -44, 94]

QUANTIZATION TECHNIQUES

Quantization Mathematics

Dequantization

INT8	Dequantized	Error
28	0.507	0.007
-66	-1.195	0.005
127	2.300	0
-44	-0.797	0.003
94	1.702	0.002

QUANTIZATION TECHNIQUES Quantization Methods

Trained Model (FP32) —▶ Calibration Data —▶ Quantized Model (INT8)

- Trained Model (FP32) —▶ Calibration Data: No training, just calibration
- Quantized Model (INT8): Fast, some accuracy loss

Post-Training Quantization (PTQ)

Pipeline:

Trained Model (FP32) → Calibration Data → Quantized Model (INT8)

Characteristics:

- **No retraining** required
- **Very fast** conversion
- Slight accuracy loss possible

Quantization-Aware Training (QAT)

Pipeline:

Training with simulated quantization → Final Quantized Model

Characteristics:

- Quantization simulated **during training**
- **Higher final accuracy**
- Requires additional training time

Train with simulated quantization —▶ Final Quantized Model

- Train with simulated quantization: Higher accuracy, slower training
- Final Quantized Model: Best accuracy

QUANTIZATION TECHNIQUES Quantization Methods

Accuracy Comparison

Method	Accuracy	Time	Use Case
FP32 baseline	100%	Training complete	Reference model
PTQ INT8	98.5%	Minutes	Rapid deployment
QAT INT8	99.2%	Hours	Production systems
PTQ INT4	95%	Minutes	Edge devices
QAT INT4	97%	Hours	Mobile deployment

PES UNIVERSITY

CELEBRATING 50 YEARS

Mixture of Experts

MIXTURE OF EXPERTS (MoE)

What is Mixture of Experts (MoE)?

Definition:

Mixture of Experts (MoE) is a neural network architecture that uses **multiple specialized sub-networks** ("experts") with a gating mechanism that routes each input to the most appropriate expert(s).

Analogy:

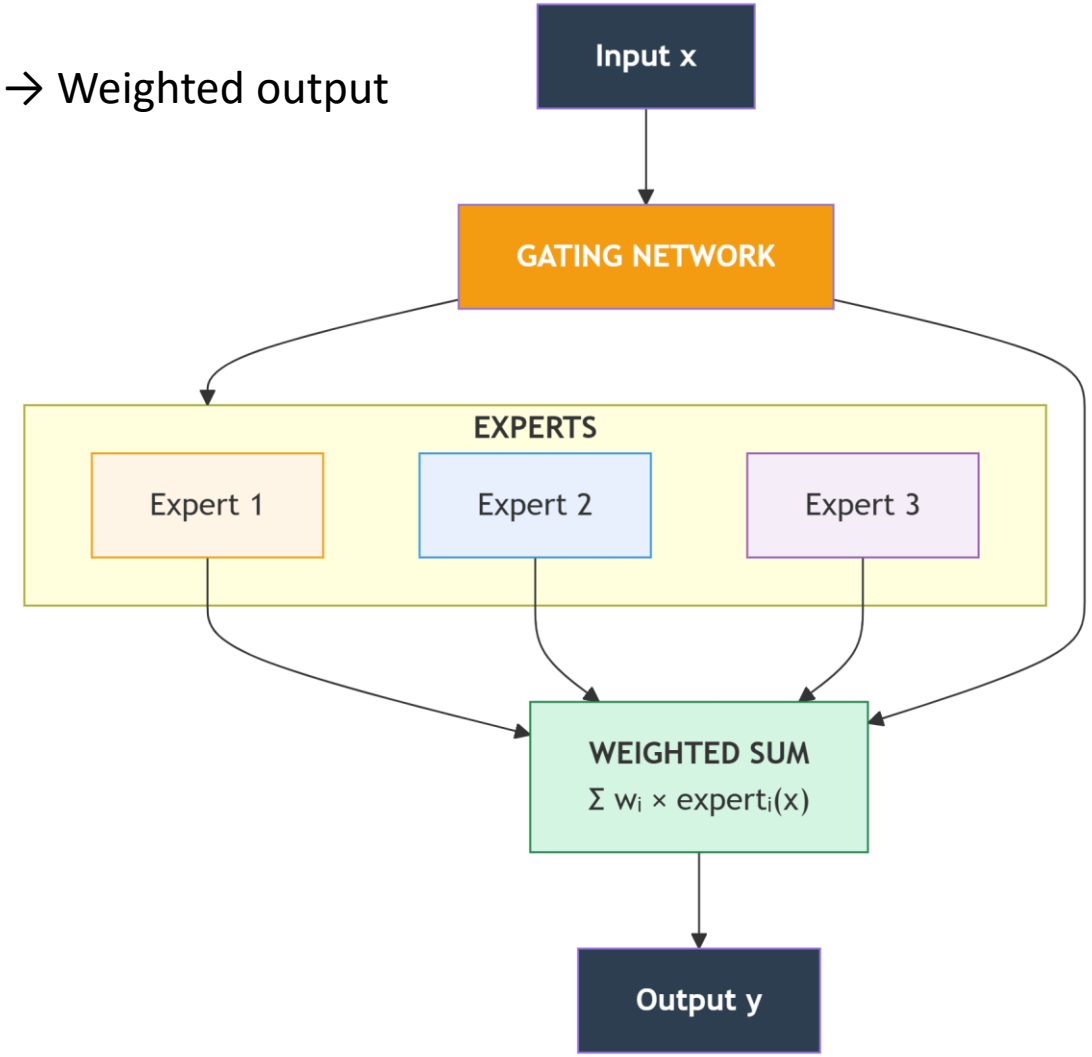
MoE is like a **hospital with specialist doctors**:

- General physician (gate) routes patients to specialists
- Cardiologist (expert 1) handles heart issues
- Neurologist (expert 2) handles brain issues
- Orthopedist (expert 3) handles bone issues
- Each patient sees only relevant specialists, not all doctors!

MIXTURE OF EXPERTS (MoE)

Visual Architecture:

Input → Gate → Experts → Weighted output



MIXTURE OF EXPERTS (MoE)

MoE Architecture – Mathematical Formulation:

Gating Function

$$G(x) = \text{softmax}(W_g \cdot x + \varepsilon)$$

Where:

- W_g = gating weight matrix
- x = input vector
- ε = small noise added during training

Final Output

$$y = \Sigma\, G_i(x) \cdot E_i(x)$$

Where:

- $G_i(x)$ = probability assigned to expert i
- Sum of all gate probabilities equals 1

Expert Outputs

Each expert computes:

$E_i(x)$

for i = 1 … N experts.

Sparse MoE (Top-K Routing)

Instead of using all experts:

I. Select top K experts with highest gate scores
II. Set other gate probabilities to zero
III. Renormalize the remaining probabilities

Final output:

$$y = \Sigma\, (i \in \text{Top-K})\, G_i(x) \cdot E_i(x)$$

Typical value: **K = 2**

Benefits

- Only a few experts are active per input
- Large model capacity
- Efficient computation
- Scales to very large parameter counts

MIXTURE OF EXPERTS (MoE)

MoE Numerical Example

Input vector:
x = [1.0, 0.5]

Expert Outputs
Expert 1
E1(x) = 0.8x1 + 0.3x2
= 0.8(1.0) + 0.3(0.5)
= 0.95
Expert 2
E2(x) = 0.4x1 + 0.9x2
= 0.4(1.0) + 0.9(0.5)
= 0.85
Expert 3
E3(x) = 0.2x1 + 0.5x2
= 0.2(1.0) + 0.5(0.5)
= 0.45

Gating Network
Weight matrix:
W_g = [[0.5, 0.3], (weights for expert1)
[0.2, 0.7], (weights for expert2)
[0.1, 0.4]] (weights for expert3)
Gate logits: Wg · x
[0.5×1.0+0.3×0.5, 0.2×1.0+0.7×0.5, 0.1×1.0+0.4×0.5]

= [0.65, 0.55, 0.30]

Gate Probabilities
softmax([0.65, 0.55, 0.30])
≈ [0.38, 0.34, 0.28]

Final Output
y = 0.38(0.95) + 0.34(0.85) + 0.28(0.45)
y = 0.361 + 0.289 + 0.126
y = **0.776**

MIXTURE OF EXPERTS (MoE)

Sparse MoE Example (Top-K = 2):

Gate logits:[0.65, 0.55, 0.30]
Top-2 experts: Expert1 and Expert2

Renormalized Gate Probabilities
G1 = 0.65 / (0.65 + 0.55) = 0.542
G2 = 0.55 / (0.65 + 0.55) = 0.458
Expert3 probability = 0

Final Output
y = 0.542(0.95) + 0.458(0.85)
y = 0.515 + 0.389
y = **0.904**

Computational Savings:
Dense: 3 expert computations (100% compute)
Sparse: 2 expert computations (67% compute)
With 32 experts, K=2: only 6.25% compute!

Method	Experts Used	Compute
Dense MoE	3/3	100%
Sparse MoE	2/3	67%

Example with 32 experts:
Top-2 routing uses only **6.25% of experts per token**.

Idea:
•Use only top-K experts
Benefit:
•Same performance
•Less computation

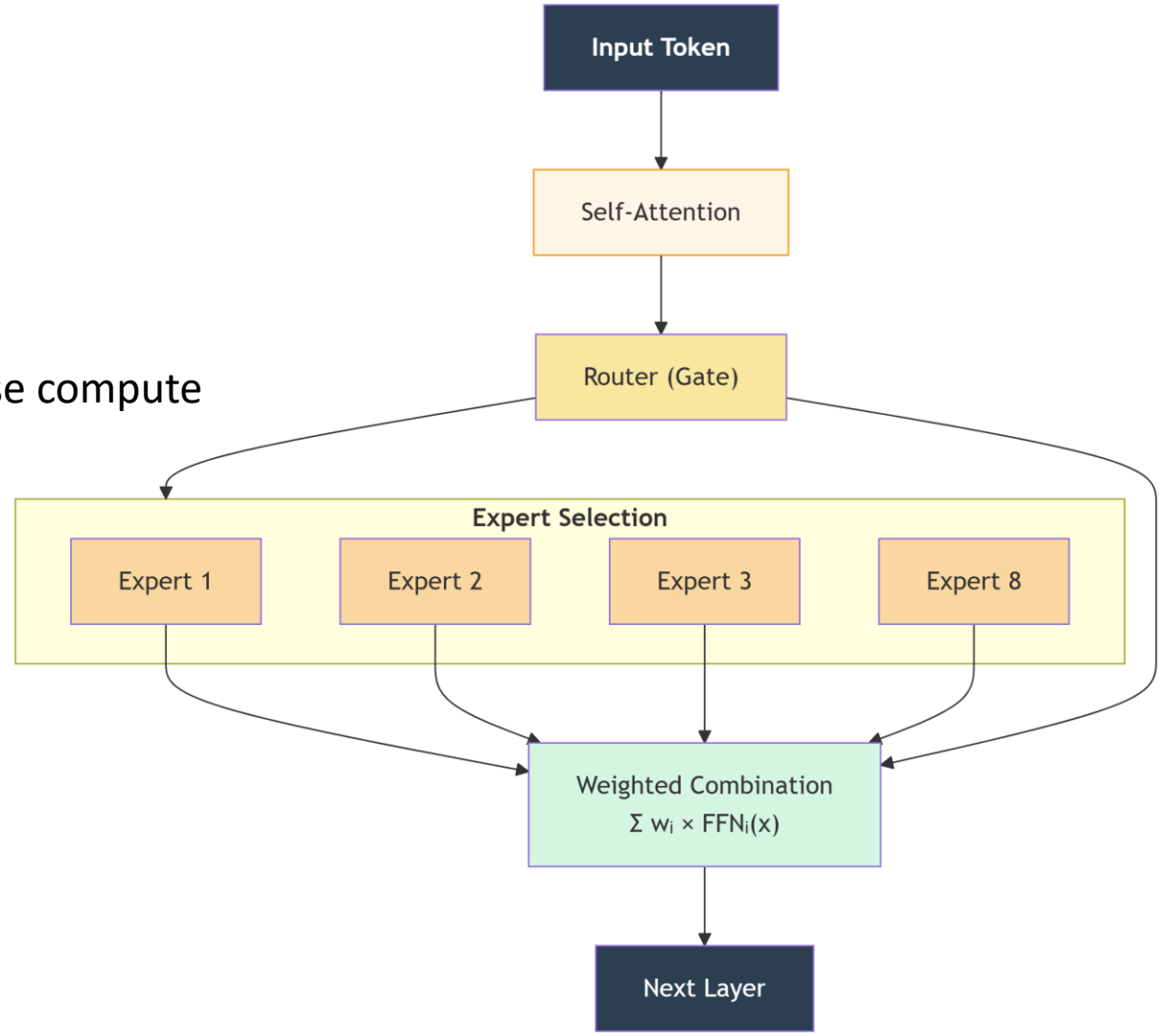
MIXTURE OF EXPERTS (MoE)

MoE in Large Language Models

MoE is commonly used in large transformer architectures.

Key Characteristics

- Each token routed to 1–2 experts
- Different tokens activate different experts
- Experts learn specialized representations
- Enables trillion-parameter models with sparse compute



MIXTURE OF EXPERTS (MoE)

MoE Numerical Example in LLM

MoE is commonly used in large transformer architectures.

Input sentence: "The cat sat on the mat"

Token 1: "The" → Gate probabilities: [0.1, 0.7, 0.1, 0.1] → Expert2
Token 2: "cat" → Gate probabilities: [0.8, 0.1, 0.05, 0.05] → Expert1
Token 3: "sat" → Gate probabilities: [0.2, 0.2, 0.5, 0.1] → Expert3
Token 4: "on" → Gate probabilities: [0.15, 0.15, 0.6, 0.1] → Expert3
Token 5: "the" → Gate probabilities: [0.1, 0.7, 0.1, 0.1] → Expert2
Token 6: "mat" → Gate probabilities: [0.05, 0.1, 0.8, 0.05] → Expert3

Expert Specialization:
- Expert1: Nouns (cat, dog, house)
- Expert2: Articles/determiners (the, a, an)
- Expert3: Verbs/actions (sat, ran, jumped)
- Expert4: Prepositions (on, in, under) - lightly used

To prevent all tokens going to same expert:

$$L_{balance} = \alpha \cdot \Sigma (f_i \times P_i)$$

where:
f_i = fraction of tokens routed to expert i
P_i = average gate probability for expert i
α = balancing coefficient (typically 0.01)

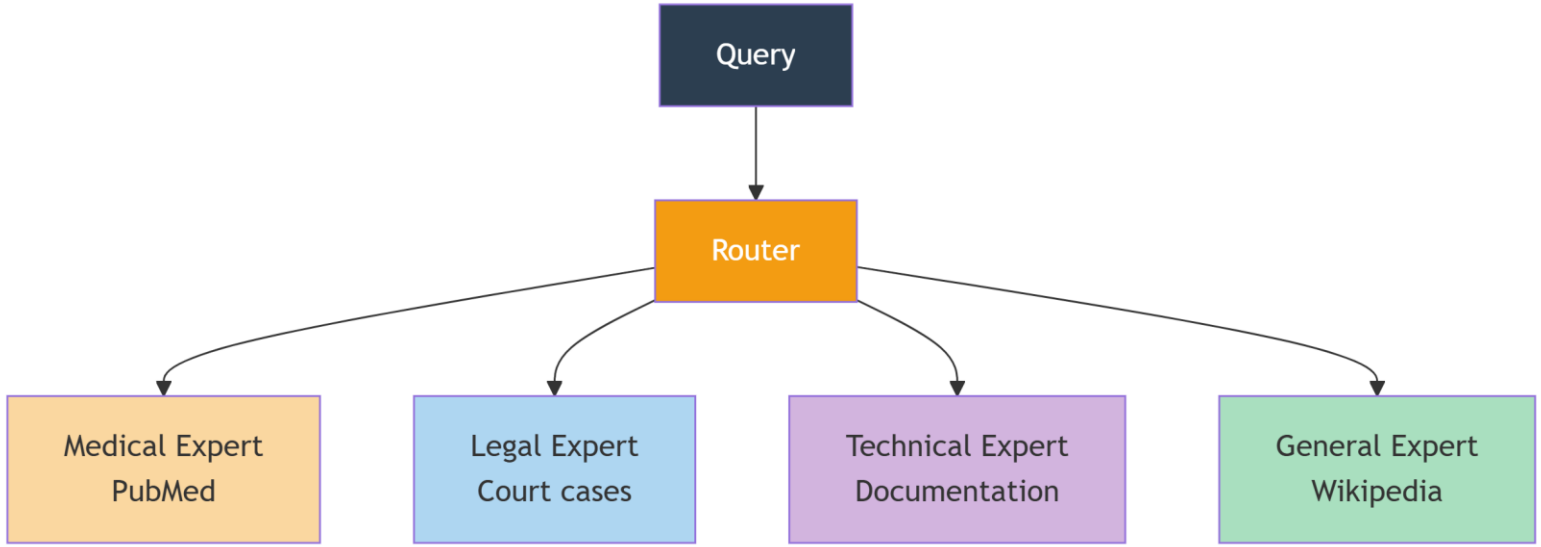
If f_i deviates from 1/N, loss increases!

MIXTURE OF EXPERTS (MoE)

MoE in RAG - Potential Applications

MoE is commonly used in large transformer architectures.

1. Multi-Domain Retrieval:



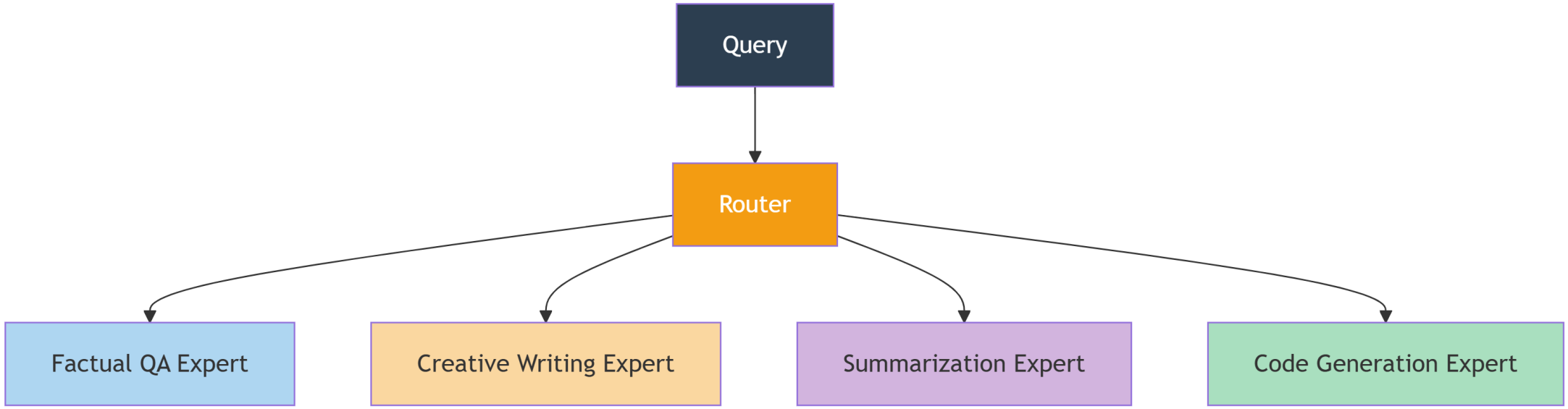
Query: "What are the side effects of Metformin?"
→ Routed to Medical Expert (90%), General Expert (10%)

MIXTURE OF EXPERTS (MoE)

MoE in RAG - Potential Applications

MoE is commonly used in large transformer architectures.

2. Multi-Query Type Generation:



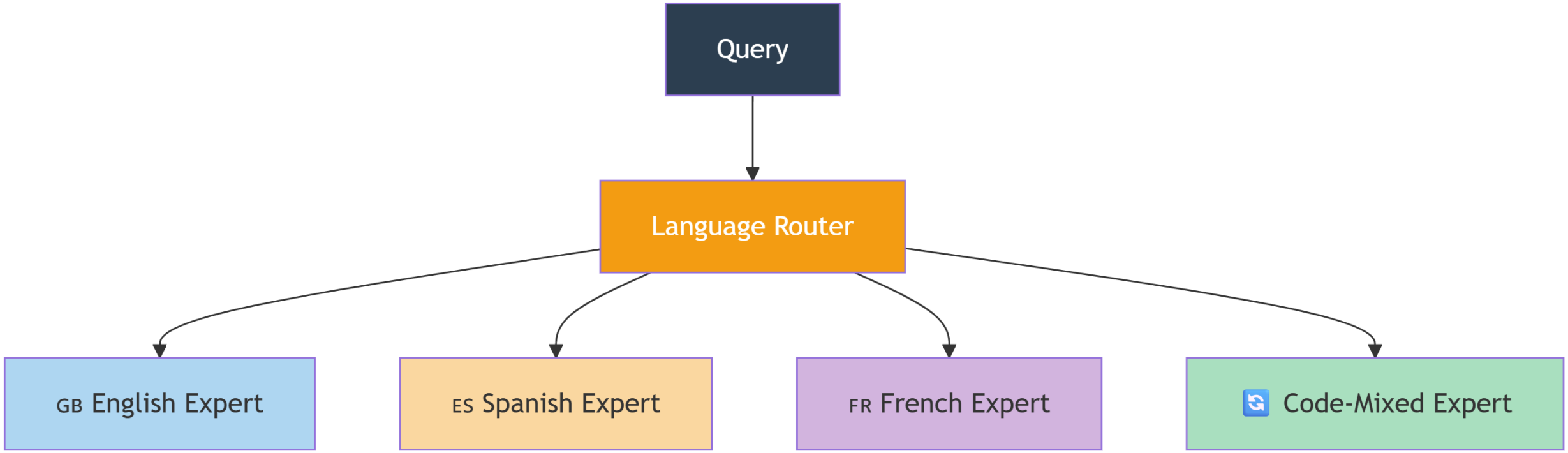
Query: "Write a poem about AI"
→ Routed to Creative Expert (95%), Factual Expert (5%)

MIXTURE OF EXPERTS (MoE)

MoE in RAG - Potential Applications

MoE is commonly used in large transformer architectures.

3. Multi-Language Support:



GenerativeAI and Its Applications

Practice Problems

Problem 1: LoRA Parameter Calculation
Given a layer with input_dim=1024, output_dim=1024, calculate:
Original parameter count
LoRA parameters with r=16
Percentage of trainable parameters

Problem 2: Quantization Error
FP32 weights: [0.876, -1.234, 2.567, -3.145, 0.432]
Quantize to INT8 (symmetric) and calculate MSE error.

Problem 3: MoE Routing
Input x = [0.2, 0.8]
Gate weights W_g = [[0.3,0.7], [0.6,0.4], [0.2,0.5], [0.1,0.9]]
Expert outputs: [0.5, 0.8, 0.3, 0.6]
Calculate:
Gate logits and probabilities
Sparse output with K=2
Compare with dense output

Problem 4: Fine-Tuning Strategy
Design a fine-tuning strategy for a legal document Q&A system with:
10,000 legal documents
1,000 query-answer pairs
Need high precision (95%+)
Must run on 8GB GPU

GenerativeAI and Its Applications

Practice Problems

Problem 1: LoRA Parameter Calculation

Given:

- Layer: input_dim = 1024, output_dim = 1024
- LoRA rank r = 16

Step 1: Original Parameter Count

Original weight matrix $W \in \mathbb{R}^{1024x1024}$

Parameters = 1024 × 1024 = 1,048,576

Step 2: LoRA Parameters

LoRA uses two matrices: $A \in \mathbb{R}^{rxk}$ and $B \in \mathbb{R}^{dxr}$

where d = output_dim = 1024, k = input_dim = 1024, r = 16

Matrix A parameters = r × k = 16 × 1024 = 16,384

Matrix B parameters = d × r = 1024 × 16 = 16,384

Total LoRA parameters = 16,384 + 16,384 = 32,768

Step 3: Percentage of Trainable Parameters

Percentage = (LoRA parameters / Original parameters) × 100

= (32,768 / 1,048,576) × 100

= 3.125%

- Original parameters: **1,048,576**
- LoRA parameters: **32,768**
- Trainable percentage: **3.125%**

PES UNIVERSITY

Problem 2: Quantization Error

Given: FP32 weights = [0.876, -1.234, 2.567, -3.145, 0.432]

Step 1: Find Scale Factor (Symmetric Quantization)

Max absolute value = max(|-3.145|, |2.567|, ...) = 3.145

INT8 range: -127 to 127

Scale factor s = max_abs / 127 = 3.145 / 127 = 0.02476

Step 2: Quantize to INT8

Formula: x_int = round(x / s)

0.876 / 0.02476 = 35.38 → 35

-1.234 / 0.02476 = -49.84 → -50

2.567 / 0.02476 = 103.68 → 104

-3.145 / 0.02476 = -127.02 → -127

0.432 / 0.02476 = 17.45 → 17

Quantized INT8: [35, -50, 104, -127, 17]

Practice Problems Problem 2: Quantization Error

Step 3: Dequantize

Formula: x_dequant = x_int × s

35 × 0.02476 = 0.8666

-50 × 0.02476 = -1.2380

104 × 0.02476 = 2.5750

-127 × 0.02476 = -3.1445

17 × 0.02476 = 0.4209

Dequantized: [0.8666, -1.2380, 2.5750, -3.1445, 0.4209]

Step 4: Calculate MSE (Mean Squared Error)

Squared errors:

$(0.876 - 0.8666)^2 = (0.0094)^2 = 0.000088$

$(-1.234 - (-1.2380))^2 = (0.004)^2 = 0.000016$

$(2.567 - 2.5750)^2 = (-0.008)^2 = 0.000064$

$(-3.145 - (-3.1445))^2 = (-0.0005)^2 = 0.00000025$

$(0.432 - 0.4209)^2 = (0.0111)^2 = 0.000123$

MSE = (0.000088 + 0.000016 + 0.000064 + 0.00000025 + 0.000123) / 5

= 0.00029125 / 5

= 0.00005825

Answer: MSE = **5.825×10^{-5}**

GenerativeAI and Its Applications

Practice Problems **Problem 3: MoE Routing**

Given:

•Input x = [0.2, 0.8]

•Gate weights W_g = [[0.3, 0.7], [0.6, 0.4], [0.2, 0.5], [0.1, 0.9]]

•Expert outputs: [0.5, 0.8, 0.3, 0.6]

Step 1: Calculate Gate Logits

$Logit_1 = 0.3 \times 0.2 + 0.7 \times 0.8 = 0.06 + 0.56 = 0.62$

$Logit_2 = 0.6 \times 0.2 + 0.4 \times 0.8 = 0.12 + 0.32 = 0.44$

$Logit_3 = 0.2 \times 0.2 + 0.5 \times 0.8 = 0.04 + 0.40 = 0.44$

$Logit_4 = 0.1 \times 0.2 + 0.9 \times 0.8 = 0.02 + 0.72 = 0.74$

Gate logits = [0.62, 0.44, 0.44, 0.74]

Step 2: Calculate Gate Probabilities (Softmax)

Denominator = $e^{0.62} + e^{0.44} + e^{0.44} + e^{0.74}$

= 1.859 + 1.553 + 1.553 + 2.096 = 7.061

$P_1 = 1.859 / 7.061 = 0.263$
$P_2 = 1.553 / 7.061 = 0.220$
$P_3 = 1.553 / 7.061 = 0.220$
$P_4 = 2.096 / 7.061 = 0.297$

Gate probabilities = [0.263, 0.220, 0.220, 0.297]

Step 3: Dense Output (All Experts)

Dense output = $\Sigma P_i \times E_i$

= 0.263×0.5 + 0.220×0.8 + 0.220×0.3 + 0.297×0.6

= 0.1315 + 0.176 + 0.066 + 0.1782

= 0.5517

Practice Problems Problem 3: MoE Routing

Step 4: Sparse Output with K=2

Top-2 logits: indices 3 (0.74) and 0 (0.62)

Renormalize top-2 probabilities:

Sum_top = 0.297 + 0.263 = 0.560

P_3_sparse = 0.297 / 0.560 = 0.530

P_0_sparse = 0.263 / 0.560 = 0.470

P_1_sparse = P_2_sparse = 0

Sparse output = 0.470×0.5 + 0×0.8 + 0×0.3 + 0.530×0.6

= 0.235 + 0 + 0 + 0.318

= 0.553

Comparison:

- Dense output: **0.5517**
- Sparse output (K=2): **0.5530**
- Difference: **0.0013** (0.24%)

Answer: Sparse routing with K=2 achieves **99.76%** of dense output quality with **50% fewer expert computations!**

Practice Problems

Problem 4: Fine-Tuning Strategy for Legal Document Q&A

Requirements Analysis:

- 10,000 legal documents (for retrieval)
- 1,000 query-answer pairs (for fine-tuning)
- 95%+ precision needed
- 8GB GPU constraint

Phase 1: Retriever Fine-Tuning

Model: all-MiniLM-L6-v2 (80MB) or legal-bert (340MB)

Method: Multiple Negatives Ranking Loss

Data: Create triplets from 1,000 QA pairs

- Query + positive answer + negative (other answers)

Hard Negatives: Use BM25 to find similar but irrelevant docs

Batch Size: 32 (fits in 8GB)

Epochs: 3-5

Expected Improvement: +5-8% recall@10

Practice Problems

Problem 4: Fine-Tuning Strategy for Legal Document Q&A

Phase 2: Reranker Fine-Tuning (Critical for 95% precision)
Model: cross-encoder/ms-marco-MiniLM-L-6-v2 (120MB)
Method: Binary cross-entropy
Data: 1,000 QA pairs × 10 negatives = 10,000 examples
- Label 1 for positive pairs
- Label 0 for negative pairs

Quantization: INT8 post-training (30MB)
Batch Size: 16
Epochs: 2-3
Expected Improvement: +10-15% precision@5

Phase 3: Generator (Optional - if response format matters)
Model: Llama-2-7B (FP16 = 14GB → too large)
Solution: Use 4-bit quantized version (3.5GB)
Method: LoRA (r=8) with PEFT
Trainable params: ~0.1% (saves memory)
Data: 1,000 QA pairs formatted as:
"### Instruction: {query}\n### Context: {retrieved_docs}\n### Response: {answer}"
Batch Size: 4 with gradient accumulation
Epochs: 2

Component	Before Fine-tuning	After Fine-tuning	Improvement
Retriever Recall@10	82%	89%	+7%
Reranker Precision@5	78%	94%	+16%
End-to-End Accuracy	75%	92%	+17%

THANK YOU